LAPORAN ARSITEKTUR BACK-END PROJECTMATCH AI

ProjectMatch AI: Link-and-Match Pendidikan dengan Kebutuhan Industri, Riset, dan Masyarakat

Diusulkan Oleh :

'Abdullah Abil Abi Yoso	V3925001	(TI B)
Khoirulbarie Kholifatul Anshori	V3925028	(TI B)

PROGRAM STUDI D3 TEKNIK INFORMATIKA

KAMPUS KAB. MADIUN

SEKOLAH VOKASI

UNIVERSITAS SEBELAS MARET

2026

DAFTAR ISI

RINGKASAN DAN STACK 7
Stack yang ditetapkan 7
1. Struktur Arsitektur Sistem 9
1.1 Gambaran Tiga Lapis 9
1.2 Komponen dan Fungsi Utama 10
1.3 Pilihan Arsitektur dan Alasannya 11
1.4 Alternatif yang Tidak Dipilih 12
2. Kebutuhan Fungsional pada Backend 13
3. Autentikasi dan Keamanan 15
4. Alur Fungsional Utama 16
4.1 Pola Pemrosesan Asinkron 16
4.2 Registrasi, Login, dan Penautan GitHub (FR-01, FR-02) 18
4.3 Pembuatan Portofolio dari GitHub (FR-03) 18
4.4 Agregasi Lowongan dan Ekstraksi Kriteria (FR-04, FR-05) 18
4.5 Pencocokan dan Visualisasi (FR-06, FR-07) 19
4.6 Dasbor Lowongan, Bookmark, dan Pengalihan Lamaran (FR-08) 19
4.7 Notifikasi Real-time (FR-09) 19
4.8 Penjadwalan dan Sinkronisasi Lowongan (FR-10) 20
4.9 Administrasi Akun dan Statistik (FR-11, FR-12) 20
5. Pipeline AI: Ekstraksi, Embedding, dan Pencocokan 21
5.1 Prinsip Rancangan 21
5.2 Tahapan Pipeline 21
5.3 Pemicu Pencocokan 21
5.4 Pemisahan Status 22
5.5 Penanganan Kegagalan 22
6. Integrasi Layanan Pihak Ketiga 23
6.1 Integrasi Groq Cloud API 24
6.2 Integrasi GitHub REST API 25
6.3 Integrasi API Lowongan Eksternal (LinkedIn, Upwork, Fiverr) 26
6.4 Kanal WebSocket 26
7. Peran dan Hak Akses 28
8. Aturan Data dan Konsistensi 29
9. Endpoint Inti Backend 30
9.1 Kode Respons yang Digunakan 32
10. Non-Fungsional dan Cara Verifikasi 33
11. Penempatan Komponen dan Infrastruktur 35

12. Model Data....37
13. Diagram UML....39
13.1 Use Case Diagram....39
13.2 Activity Diagram....44
13.3 Class Diagram....56
13.4 Sequence Diagram....65
14. Catatan Penyelarasan terhadap SKPL....75
15. Keterbatasan dan Mitigasi....77
16. Penutup....78
16.1 Kesimpulan....78
16.2 Saran Pengembangan....78
DAFTAR PUSTAKA....79

DAFTAR GAMBAR

Gambar 1. Arsitektur three-tier backend ProjectMatch AI 10
Gambar 2. Alur pemrosesan asinkron backend ProjectMatch AI 17
Gambar 3. Peta integrasi backend ProjectMatch AI 23
Gambar 4. Sequence diagram integrasi Groq API 25
Gambar 5. Sequence diagram integrasi GitHub API 26
Gambar 6. Sequence diagram integrasi WebSocket 27
Gambar 7. Peta endpoint REST API per role 31
Gambar 8. Deployment diagram ProjectMatch AI 35
Gambar 9. Entity Relationship Diagram (ERD) ProjectMatch AI 37
Gambar 10. Use Case Diagram FR-01: Registrasi & Login 39
Gambar 11. Use Case Diagram FR-02: Manajemen Profil & Penautan GitHub 39
Gambar 12. Use Case Diagram FR-03: Automated GitHub Portfolio Generator 40
Gambar 13. Use Case Diagram FR-04: Agregasi Lowongan Pekerjaan Multi-Platform 40
Gambar 14. Use Case Diagram FR-05: Ekstraksi Kriteria Lowongan Otomatis (LLM) 41
Gambar 15. Use Case Diagram FR-06: AI Semantic Matching 41
Gambar 16. Use Case Diagram FR-07: Visualisasi Radar Chart Interaktif 41
Gambar 17. Use Case Diagram FR-08: Dasbor Eksplorasi Lowongan & Pengalihan Lamaran 42
Gambar 18. Use Case Diagram FR-09: Notifikasi Real-time 42
Gambar 19. Use Case Diagram FR-10: Automated Job Feed Scheduler & Data Sync 43
Gambar 20. Use Case Diagram FR-11: Manajemen Akun & Hak Akses 43
Gambar 21. Use Case Diagram FR-12: Monitoring & Statistik Agregasi 44
Gambar 22. Activity Diagram FR-01: Registrasi & Login 45
Gambar 23. Activity Diagram FR-02: Manajemen Profil & Penautan GitHub 46
Gambar 24. Activity Diagram FR-03: Automated GitHub Portfolio Generator 47
Gambar 25. Activity Diagram FR-04: Agregasi Lowongan Pekerjaan Multi-Platform 48
Gambar 26. Activity Diagram FR-05: Ekstraksi Kriteria Lowongan Otomatis (LLM) 49
Gambar 27. Activity Diagram FR-06: AI Semantic Matching 50
Gambar 28. Activity Diagram FR-07: Visualisasi Radar Chart Interaktif 51
Gambar 29. Activity Diagram FR-08: Dasbor Eksplorasi Lowongan & Pengalihan Lamaran 52
Gambar 30. Activity Diagram FR-09: Notifikasi Real-time 53
Gambar 31. Activity Diagram FR-10: Automated Job Feed Scheduler & Data Sync 54
Gambar 32. Activity Diagram FR-11: Manajemen Akun & Hak Akses 55
Gambar 33. Activity Diagram FR-12: Monitoring & Statistik Agregasi 56
Gambar 34. Class Diagram FR-01: Registrasi & Login 57
Gambar 35. Class Diagram FR-02: Manajemen Profil & Penautan GitHub 58
Gambar 36. Class Diagram FR-03: Automated GitHub Portfolio Generator 59

Gambar 37. Class Diagram FR-04: Agregasi Lowongan Pekerjaan Multi-Platform 59
Gambar 38. Class Diagram FR-05: Ekstraksi Kriteria Lowongan Otomatis (LLM) 60
Gambar 39. Class Diagram FR-06: AI Semantic Matching 60
Gambar 40. Class Diagram FR-07: Visualisasi Radar Chart Interaktif 61
Gambar 41. Class Diagram FR-08: Dasbor Eksplorasi Lowongan & Pengalihan Lamaran 62
Gambar 42. Class Diagram FR-09: Notifikasi Real-time 63
Gambar 43. Class Diagram FR-10: Automated Job Feed Scheduler & Data Sync 63
Gambar 44. Class Diagram FR-11: Manajemen Akun & Hak Akses 64
Gambar 45. Class Diagram FR-12: Monitoring & Statistik Agregasi 64
Gambar 46. Sequence Diagram FR-01: Registrasi & Login 65
Gambar 47. Sequence Diagram FR-02: Manajemen Profil & Penautan GitHub 66
Gambar 48. Sequence Diagram FR-03: Automated GitHub Portfolio Generator 67
Gambar 49. Sequence Diagram FR-04: Agregasi Lowongan Pekerjaan Multi-Platform 68
Gambar 50. Sequence Diagram FR-05: Ekstraksi Kriteria Lowongan Otomatis (LLM) 69
Gambar 51. Sequence Diagram FR-06: AI Semantic Matching 70
Gambar 52. Sequence Diagram FR-07: Visualisasi Radar Chart Interaktif 70
Gambar 53. Sequence Diagram FR-08: Dasbor Eksplorasi Lowongan & Pengalihan Lamaran 71
Gambar 54. Sequence Diagram FR-09: Notifikasi Real-time 72
Gambar 55. Sequence Diagram FR-10: Automated Job Feed Scheduler & Data Sync 72
Gambar 56. Sequence Diagram FR-11: Manajemen Akun & Hak Akses 73
Gambar 57. Sequence Diagram FR-12: Monitoring & Statistik Agregasi 74

DAFTAR TABEL

Tabel 1. Stack yang digunakan 7
Tabel 2. Komponen dan fungsi utama 10
Tabel 3. Alternatif arsitektur yang tidak dipilih 12
Tabel 4. Kebutuhan fungsional dan perilaku backend 13
Tabel 5. Autentikasi dan keamanan 15
Tabel 6. Pemisahan status pada sistem 22
Tabel 7. Antarmuka integrasi backend 23
Tabel 8. Peran dan hak akses 28
Tabel 9. Aturan data dan konsistensi 29
Tabel 10. Endpoint inti backend 30
Tabel 11. Kode respons HTTP yang muncul pada alur backend 32
Tabel 12. Kebutuhan non-fungsional dan cara verifikasinya 33
Tabel 13. Penempatan komponen 36
Tabel 14. Ringkasan entitas dan relasi 38
Tabel 15. Keterbatasan sistem dan mitigasi pada rancangan 77

RINGKASAN DAN STACK

ProjectMatch AI adalah platform otomatisasi karier digital berbasis web yang mempertemukan keahlian riil mahasiswa pada repositori GitHub dengan lowongan freelance dan part-time dari LinkedIn, Fiverr, dan Upwork. Pencocokan dilakukan secara semantik dengan bantuan AI, bukan sekadar mencocokkan kata kunci atau melihat IPK. Backend bertugas menyediakan REST API (antarmuka yang dipanggil aplikasi web) untuk akun dan login, pengelolaan profil dan penautan akun GitHub, pembuatan portofolio dari repositori, agregasi lowongan multi-platform, pencocokan dan rekomendasi, bookmark dan pengalihan ke tautan lamaran, notifikasi real-time, serta administrasi akun dan statistik agregasi.

Backend dibangun dengan pendekatan tiga lapis (three-tier): klien di browser, aplikasi FastAPI di server, dan data di PostgreSQL dengan ekstensi pgvector. Proses yang berat, yaitu pengambilan data GitHub, sintesis portofolio dan ekstraksi kriteria lowongan lewat LLM, penarikan lowongan dari API platform, dan pembuatan embedding, tidak dikerjakan di dalam permintaan pengguna. Proses itu dijalankan di latar belakang dan statusnya dicatat pada kolom `processing_status`. Inferensi LLM dialihkan ke layanan Groq Cloud API sehingga server tidak perlu menanggung beban komputasi model bahasa. Bila Groq gagal merespons atau terkena batas kuota, sistem mencoba ulang maksimal tiga kali, lalu menandai proses sebagai `failed` agar dapat diproses ulang. Kegagalan atau batas kuota pada salah satu API platform lowongan juga tidak memutus aliran kerja, karena sistem mencatat galat dan tetap memproses platform yang tersedia. Fitur lain seperti login, profil, dan pencocokan dari embedding yang sudah tersimpan tetap berjalan.

Stack yang ditetapkan

Tabel 1. Stack yang digunakan

Komponen	Teknologi
Backend	Python, FastAPI (dijalankan dengan Uvicorn), REST API berawalan `/api/v1`, WebSocket
Autentikasi dan keamanan	JWT, RBAC (Role-Based Access Control), Argon2id, HTTPS/TLS lewat Nginx
Data	PostgreSQL 15+ dengan pgvector 0.5+ (indeks HNSW, cosine distance), SQLAlchemy, Alembic, pgvector-python
AI dan NLP	Groq Cloud API (`Llama-3.1-8b-instant`), Sentence Transformers (PyTorch) model multibahasa 384 dimensi (misalnya `paraphrase-multilingual-MiniLM-L12-v2`)
Pemrosesan asinkron	FastAPI BackgroundTasks, scheduler atau worker latar belakang (cron) untuk penarikan lowongan berkala (opsional Celery + Redis)
Penyimpanan data	Hanya basis data dan cache metadata (kolom `github_data`); tidak ada penyimpanan berkas PDF maupun citra e-KYC
Integrasi eksternal	GitHub REST API, LinkedIn API atau RapidAPI Job Search, Upwork API (OAuth 2.0 / GraphQL Job Search), Fiverr API (Gigs / Buyer Request), Groq Cloud API

Komponen	Teknologi
Klien dan infrastruktur	Next.js dengan Recharts, Nginx (reverse proxy), Ubuntu 22.04 LTS (server), Windows 10/11 (pengembangan)

Dokumen ini adalah rancangan arsitektur backend yang disusun berdasarkan panduan perubahan SKPL ProjectMatch AI. Seluruh target kinerja di dalamnya, yaitu akurasi pencocokan minimal 80%, waktu rekomendasi kurang dari 1 menit, sintesis portofolio GitHub kurang dari 10 detik per akun, penguraian satu lowongan oleh LLM kurang dari 3 detik, dan uptime minimal 99%, merupakan sasaran yang dibuktikan lewat pengujian pada tahap implementasi, bukan capaian yang sudah diukur.

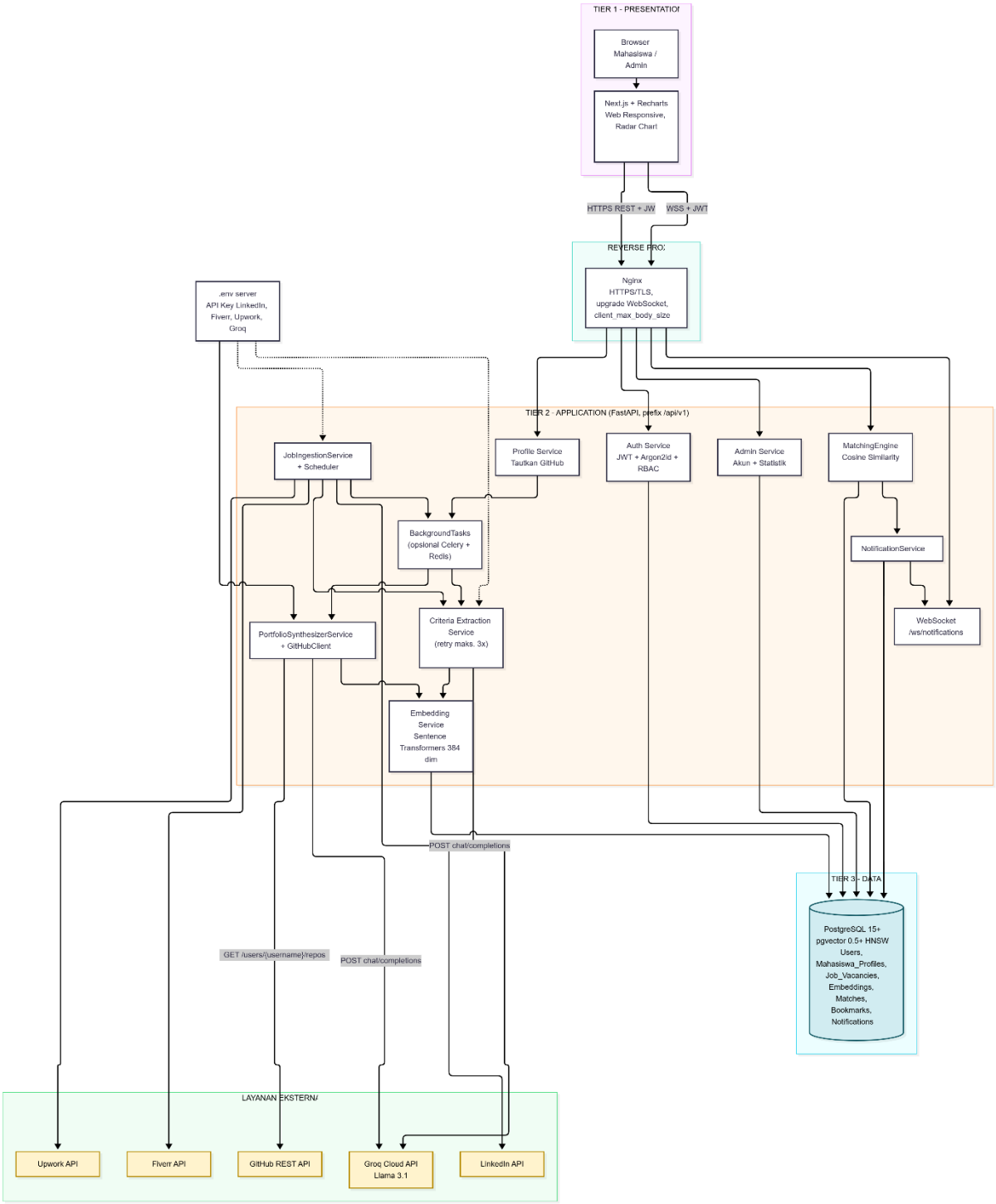
1. Struktur Arsitektur Sistem

1.1 Gambaran Tiga Lapis

Arsitektur backend ProjectMatch AI terbagi menjadi tiga lapis. Lapis pertama adalah presentasi, yaitu aplikasi web Next.js di browser pengguna. Lapis ini memuat tampilan responsif, komponen Recharts untuk radar chart overlay dan drill-down, tautan pengalihan ke halaman lowongan di platform asal, serta klien WebSocket untuk menerima notifikasi. Klien berkomunikasi dengan server lewat HTTPS pada awalan `/api/v1` dan lewat WSS pada `/ws/notifications`.

Lapis kedua adalah aplikasi yang berjalan pada server Ubuntu 22.04. Seluruh lalu lintas masuk lebih dulu melewati Nginx, yang menangani HTTPS/TLS, meneruskan permintaan ke FastAPI, dan mendukung upgrade WebSocket. Di belakangnya, FastAPI memuat router untuk autentikasi, profil, lowongan, pencocokan, notifikasi, dan admin, ditambah endpoint WebSocket serta middleware JWT dan RBAC. Komponen AI dan integrasi berjalan dalam proses aplikasi yang sama: GitHubClient dan PortfolioSynthesizerService untuk membangun portofolio, JobIngestionService untuk menarik lowongan dari API platform, Sentence Transformers untuk embedding, Matching Engine untuk menghitung cosine similarity, dan NotificationService untuk pengiriman notifikasi.

Lapis ketiga adalah data. PostgreSQL dengan pgvector menyimpan data relasional sekaligus vektor embedding, termasuk cache metadata GitHub dan lowongan hasil agregasi. Server tidak lagi menyimpan berkas CV, video briefing, maupun foto KTM. Di luar tiga lapis itu, backend memanggil layanan eksternal, yaitu Groq Cloud API untuk inferensi LLM, GitHub REST API untuk repositori mahasiswa, serta API LinkedIn, Upwork, dan Fiverr untuk lowongan.



Gambar 1. Arsitektur three-tier backend ProjectMatch AI

1.2 Komponen dan Fungsi Utama

Tabel 2. Komponen dan fungsi utama

Komponen	Fungsi utama
Klien (Next.js, Recharts, WebSocket client)	Menampilkan antarmuka, memanggil REST API lewat HTTPS, menerima notifikasi lewat WSS, dan mengalihkan mahasiswa ke tautan lamaran di platform asal lowongan.

Komponen	Fungsi utama
Nginx	Reverse proxy, terminasi HTTPS/TLS, dan meneruskan upgrade WebSocket.
FastAPI (router)	Menerima permintaan, memvalidasi input, lalu meneruskannya ke service. Router yang ada: Auth, Profile, Job, Matching, Notification, Admin, dan WebSocket endpoint.
JWT Middleware dan RBAC	Memverifikasi token pada setiap permintaan terlindungi dan memeriksa peran pengguna sebelum request masuk ke router.
GitHubClient	Mengambil repositori publik, language breakdown, dan README.md mahasiswa lewat GitHub REST API, lalu menyimpannya sebagai cache di `github_data`.
PortfolioSynthesizerService	Mengirim agregasi data GitHub ke Groq dan menyusun portofolio kompetensi terstruktur (JSON) yang disimpan di `extracted_skills`.
JobIngestionService (scheduler / worker)	Memanggil API LinkedIn, Fiverr, dan Upwork secara terjadwal, melakukan deduplikasi lewat `external_job_id`, dan menandai lowongan yang sudah kedaluwarsa.
LLMExtractionService	Mengirim deskripsi lowongan ke Groq untuk dinormalisasi menjadi kualifikasi kunci, kategori keahlian, durasi, dan tipe kontrak (freelance atau part-time) dalam format JSON.
Embedding Service (Sentence Transformers)	Mengubah teks portofolio GitHub mahasiswa dan kriteria lowongan menjadi vektor 384 dimensi.
Matching Engine	Menghitung cosine similarity dari vektor yang tersimpan di pgvector dan menyusun rincian skor per skill.
Pemrosesan latar belakang (BackgroundTasks, scheduler, opsional Celery + Redis)	Menjalankan sinkronisasi GitHub, sintesis portofolio, penarikan lowongan, ekstraksi LLM, dan pembuatan embedding di luar siklus permintaan pengguna.
NotificationService dan ConnectionManager	Menyimpan notifikasi ke basis data dan mendorongnya ke pengguna yang sedang terhubung lewat WebSocket.
PostgreSQL 15+ dengan pgvector	Data utama (users, profil mahasiswa, lowongan, embedding, match, bookmark, notifikasi) dan indeks HNSW untuk pencarian vektor.
Groq Cloud API	Menjalankan model Llama 3.1 untuk merangkum data GitHub menjadi portofolio kompetensi dan mengekstrak kriteria dari deskripsi lowongan dalam format JSON.
GitHub REST API	Menyediakan repositori publik, language breakdown, dan README mahasiswa dari username GitHub yang ditautkan.
API LinkedIn, Upwork, dan Fiverr	Menyediakan data lowongan freelance dan part-time (judul, deskripsi, anggaran, tautan lamaran) untuk diagregasi.

1.3 Pilihan Arsitektur dan Alasannya

Backend ProjectMatch AI dipilih berbentuk satu aplikasi FastAPI dengan tiga lapis, bukan kumpulan layanan terpisah. Ada beberapa pertimbangan yang mendasarinya.

Pertama, hampir seluruh komponen yang dibutuhkan SKPL tersedia di ekosistem Python: klien HTTP untuk GitHub REST API dan API platform lowongan, pustaka klien Groq, serta

Sentence Transformers untuk embedding. Dengan FastAPI, komponen-komponen itu bisa dipanggil langsung dari kode backend tanpa jembatan antarbahasa.

Kedua, FastAPI mendukung REST API dan WebSocket dalam satu kerangka kerja. Itu cukup untuk dua kanal komunikasi yang dibutuhkan, yaitu transaksi data utama dan notifikasi real-time (FR-09).

Ketiga, PostgreSQL dengan pgvector menjadikan satu basis data melayani dua kebutuhan: data relasional dan pencarian kemiripan vektor. Skor kecocokan dihitung dari jarak cosine yang langsung ditanyakan ke basis data.

Keempat, inferensi LLM dialihkan ke Groq Cloud API. Server cukup berspesifikasi ringan (RAM 4–8 GB, CPU 2–4 core, SSD 20 GB) karena model bahasa tidak dijalankan secara lokal.

Kelima, tim pengembang berjumlah dua orang dengan metode Agile-Scrum dalam rentang waktu perkuliahan. Satu paket aplikasi lebih mudah dibangun, diuji, dan dicari kesalahannya dibandingkan banyak layanan yang masing-masing harus dideploy. Tumpukan teknologi utamanya open-source atau memakai Free Tier. Kuota dan ketentuan akses API LinkedIn, Upwork, dan Fiverr perlu dipastikan pada tahap implementasi karena dapat berbeda dari Free Tier.

Beban proses berat tidak menahan permintaan pengguna karena dipisahkan lewat pemrosesan asinkron. Dengan begitu, penautan akun GitHub dapat langsung dijawab sementara pengambilan repositori, sintesis, dan embedding berjalan di belakang, sedangkan penarikan lowongan berjalan terjadwal (NFR-02, NFR-07).

1.4 Alternatif yang Tidak Dipilih

Tabel 3. Alternatif arsitektur yang tidak dipilih

Alternatif	Alasan tidak dipilih
Microservices (parser, embedding, matching, dan notifikasi dipisah menjadi layanan sendiri)	Cakupan sistem hanya 12 fitur dengan tim dua orang. Pemisahan layanan menambah beban deployment, komunikasi antarlayanan, dan pemantauan tanpa manfaat yang sepadan pada skala saat ini.
LLM lokal yang dijalankan sendiri di server	Membutuhkan perangkat keras jauh lebih besar daripada spesifikasi target. SKPL memilih Groq Cloud API agar server tetap ringan. LLM lokal hanya dicatat sebagai saran pengembangan bila volume permintaan sangat besar.
Basis data vektor terpisah dari PostgreSQL	Menambah satu sistem yang harus dirawat dan disinkronkan dengan data relasional. pgvector sudah cukup untuk menyimpan embedding dan mencari berdasarkan cosine distance dengan indeks HNSW.
Pencocokan berbasis kata kunci	Tidak menangkap makna dan konteks. SKPL secara tegas meminta pencocokan semantik agar portofolio dari repositori GitHub dapat dicocokkan dengan kebutuhan lowongan.

2. Kebutuhan Fungsional pada Backend

Kode FR dan NFR pada bab ini mengikuti SKPL ProjectMatch AI versi terbaru. Kolom NFR terkait diambil dari Matriks Keterkaitan FR dan NFR pada SKPL.

Tabel 4. Kebutuhan fungsional dan perilaku backend

FR	Perilaku backend	NFR terkait	Komponen dan teknologi
FR-01	Registrasi (`POST /auth/register`) dan login (`POST /auth/login`) dengan email dan password. Password di-hash dengan Argon2id, login yang berhasil menghasilkan token JWT. Penautan akun GitHub tersedia sebagai opsi.	NFR-05, NFR-06	AuthRouter, AuthService, PasswordHasher, TokenService
FR-02	Mahasiswa menautkan username GitHub sebagai sumber utama portofolio lewat `PUT /profiles/me`. `POST /profiles/github/sync` memvalidasi akun GitHub, mengatur `processing_status` menjadi `pending`, lalu mengantrekan pemrosesan. Tidak ada unggahan berkas.	NFR-03, NFR-04, NFR-05	ProfileRouter, GitHubClient, BackgroundTasks
FR-03	Sistem mengambil repositori publik, bahasa pemrograman, riwayat commit, dan README.md lewat GitHub REST API, menyusun portofolio kompetensi terstruktur (JSON) lewat Groq, lalu membuat embedding. Target kurang dari 10 detik per akun.	NFR-03, NFR-01	GitHubClient, PortfolioSynthesizerService, Groq API, Sentence Transformers, pgvector
FR-04	Sistem mengambil dan mengagregasi lowongan freelance dan part-time dari API LinkedIn, Fiverr, dan Upwork. Lowongan disimpan ke `job_vacancies` berstatus `open` dengan `processing_status pending`, dan duplikasi dicegah lewat `external_job_id`.	NFR-03, NFR-07, NFR-09	JobIngestionService, API LinkedIn / Fiverr / Upwork, JobRouter
FR-05	Sistem mengekstrak kualifikasi teknis, perkiraan durasi, dan lingkup pekerjaan dari lowongan eksternal lewat Llama 3.1 pada Groq (target kurang dari 3 detik per entri). Hasil JSON disimpan di `extracted_skills`. Gagal atau kena kuota: coba ulang maksimal 3 kali, lalu status `failed`.	NFR-01, NFR-03, NFR-07, NFR-09	LLMExtractionService, Groq API, BackgroundTasks
FR-06	Sistem menghitung skor kecocokan semantik antara embedding portofolio GitHub mahasiswa dan lowongan dengan cosine similarity, menyimpannya, dan menyajikan daftar rekomendasi serta rincian skor per skill.	NFR-01, NFR-02, NFR-07	Matching Engine, pgvector (HNSW)

FR	Perilaku backend	NFR terkait	Komponen dan teknologi
FR-07	Backend menyediakan skor dan `score_breakdown` per skill. Penggambaran radar chart, overlay kualifikasi lowongan, dan drill-down dilakukan di klien.	NFR-04, NFR-08	MatchRouter, MatchingService, Recharts (klien)
FR-08	Mahasiswa melihat rekomendasi lowongan terurut dari skor tertinggi, menyimpan lowongan (bookmark) lewat `POST /jobs/{job_id}/bookmark`, dan diarahkan ke tautan pendaftaran (`external_url`) di LinkedIn, Fiverr, atau Upwork.	NFR-02, NFR-04, NFR-08	JobRouter, MatchRouter, BookmarkService
FR-09	Notifikasi lowongan baru dengan kecocokan tinggi dikirim real-time lewat WebSocket dan disimpan di basis data. Target: kurang dari 1 menit setelah hasil pencocokan tersimpan.	NFR-02, NFR-06	WebSocketManager, NotificationService, Nginx
FR-10	Scheduler latar belakang menyinkronkan lowongan dari LinkedIn, Fiverr, dan Upwork secara berkala (atau lewat `POST /jobs/sync`) dan mengubah lowongan yang sudah kedaluwarsa menjadi status `expired`.	NFR-07, NFR-09	JobIngestionService, scheduler (cron / background worker), JobRouter
FR-11	Admin melihat daftar akun, mengubah status aktivasi, dan mengubah role pengguna.	NFR-05	AdminUserRouter, RBACGuard, AdminUserService
FR-12	Admin melihat statistik pengguna per role, jumlah lowongan per platform (LinkedIn, Fiverr, Upwork), ringkasan pencocokan, dan interaksi mahasiswa.	NFR-05, NFR-07	StatisticsRouter, StatisticsService

3. Autentikasi dan Keamanan

Seluruh akses ke API bisnis, kecuali registrasi dan login, dilindungi token JWT dan pemeriksaan peran. Identitas pengguna selalu diambil dari token, bukan dari parameter URL atau isi permintaan, sehingga pengguna tidak dapat meminta data milik orang lain hanya dengan mengganti id. JWT adalah token tanda pengenal yang ditandatangani server, sehingga server tidak perlu menyimpan sesi.

Tabel 5. Autentikasi dan keamanan

Bagian	Perilaku
Registrasi	`POST /api/v1/auth/register` menerima email dan password. Email diperiksa lebih dulu di tabel `users`; bila sudah ada, sistem menjawab 409 Conflict. Bila belum, password di-hash dan akun mahasiswa disimpan. Profil mahasiswa (`mahasiswa_profiles`) dibuat ketika username GitHub pertama kali diisi lewat `PUT /profiles/me`. Role admin tidak dapat dipilih lewat registrasi publik.
Login	`POST /api/v1/auth/login` mencari akun berdasarkan email dan memverifikasi password. User tidak ditemukan dan password salah sama-sama dijawab 401 dengan pesan yang seragam, sehingga tidak membocorkan apakah suatu email terdaftar. Akun dengan `is_active = false` dijawab 403 Forbidden.
Hashing password	Argon2id dengan salt acak serta parameter memori dan komputasi terstandar. Yang tersimpan di kolom `password_hash` hanya hasil hash (NFR-05).
Token	Login berhasil menerbitkan JWT yang memiliki masa berlaku (expiry). Token ini wajib disertakan pada seluruh endpoint terlindungi.
RBAC	Dua role: mahasiswa dan admin. Role mitra tidak lagi dipakai karena lowongan berasal dari platform eksternal yang diagregasi sistem. Pemeriksaan role dilakukan sebelum permintaan diproses; role yang tidak berhak dijawab 403.
Kepemilikan data	Data profil, rekomendasi, match, bookmark, dan notifikasi hanya dikembalikan untuk pengguna yang sedang login, karena identitas diambil dari JWT.
Validasi input	Input yang tidak sesuai ditolak dengan 422 Unprocessable Entity. Input teks disanitasi untuk mencegah XSS, dan akses basis data memakai ORM (SQLAlchemy) dengan query terparameter untuk mencegah SQL Injection.
Validasi username GitHub	Username GitHub divalidasi formatnya dan dicek keberadaannya lewat GitHub API sebelum diproses; username yang tidak ditemukan dijawab 404. Tidak ada unggahan berkas, sehingga pemeriksaan tipe dan ukuran berkas tidak lagi diperlukan.
Kunci API eksternal	`GROQ_API_KEY` serta kunci API LinkedIn, Upwork, dan Fiverr dibaca dari environment variable (`.env`) di server, tidak disimpan di repositori kode sumber.
Transmisi data	Komunikasi klien dan server memakai HTTPS/TLS yang diterminasi di Nginx, termasuk koneksi WebSocket (WSS).
WebSocket	Koneksi ke `/ws/notifications` diautentikasi dengan JWT. Token yang tidak valid atau sudah kedaluwarsa membuat koneksi ditolak.
Sinkronisasi identitas GitHub	Identitas profil diselaraskan dengan akun GitHub lewat penautan akun (account linking). `github_username` bersifat UNIQUE sehingga satu akun GitHub hanya terhubung ke satu profil. Foto KTM, citra wajah, dan data biometrik tidak lagi dikumpulkan.

Alur login dan registrasi secara rinci digambarkan pada diagram sequence FR-01 di Bab 13.

4. Alur Fungsional Utama

Bab ini menjelaskan bagaimana permintaan mengalir melalui backend, dimulai dari pola pemrosesan asinkron yang menjadi dasar fitur-fitur berat, lalu alur tiap kelompok fitur. Diagram sequence dan activity yang lebih rinci untuk setiap FR dimuat pada Bab 13.

4.1 Pola Pemrosesan Asinkron

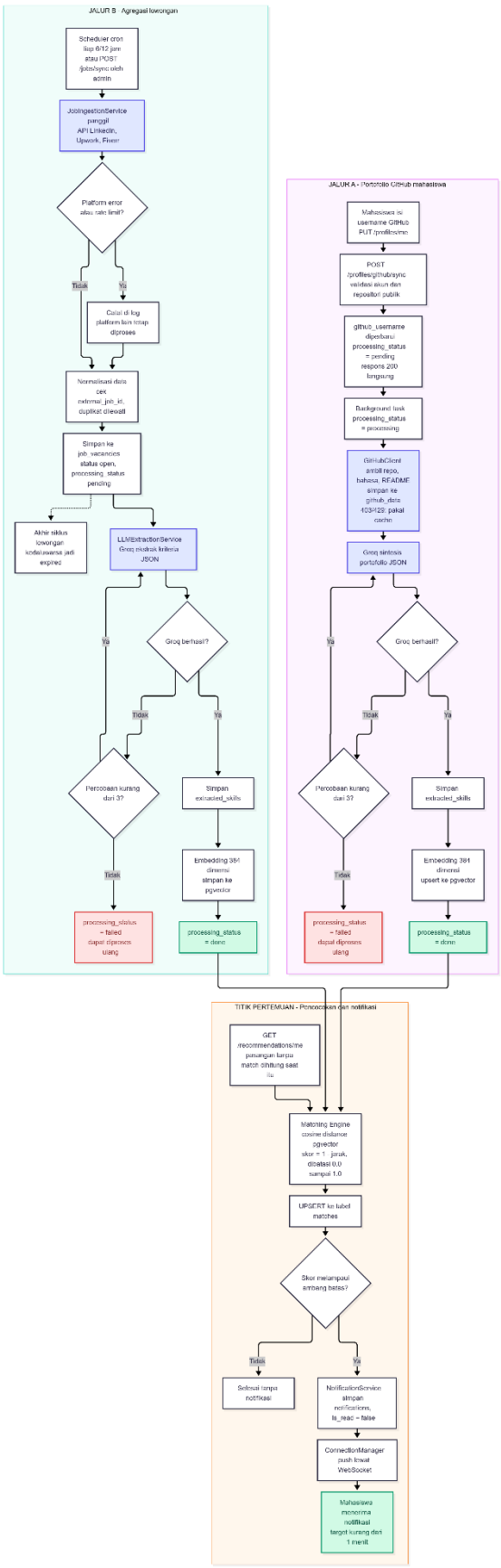
Backend memakai dua jalur pemrosesan yang berjalan di latar belakang, lalu bertemu pada tahap pencocokan.

Jalur A dimulai ketika mahasiswa memasukkan username GitHub. `processing_status` diset `pending`, kemudian tugas latar belakang memanggil GitHub REST API untuk mengambil repositori publik, persentase bahasa pemrograman, dan `README.md` (target pengambilan dan sintesis kurang dari 10 detik per akun). Data itu diteruskan ke Groq untuk disusun menjadi portofolio kompetensi terstruktur dalam JSON. Teks hasil sintesis dikonversi menjadi embedding oleh Sentence Transformers dan disimpan ke pgvector, lalu status menjadi `done`.

Jalur B dimulai oleh scheduler. JobIngestionService memanggil API LinkedIn, Fiverr, dan Upwork secara berkala. Setiap lowongan baru disimpan ke `job_vacancies` dengan status `pending`, kemudian tugas latar belakang meminta Groq mengekstrak kriteria kompetensi dari deskripsinya (target kurang dari 3 detik per entri). Hasilnya dijadikan embedding dan disimpan, lalu status menjadi `done`.

Pada titik pertemuan, Matching Engine menghitung cosine similarity, hasilnya disimpan ke tabel `matches`, NotificationService membuat notifikasi untuk pasangan yang skornya melampaui ambang batas, dan WebSocket mendorongnya ke mahasiswa. Seluruh rangkaian ini ditargetkan selesai kurang dari 1 menit sejak hasil pencocokan tersimpan (FR-09, NFR-02).

Bila Groq mengembalikan error atau terkena rate limit, sistem mencoba ulang maksimal tiga kali. Jika tetap gagal, status diubah menjadi `failed` sehingga dapat diproses ulang, dan fitur lain tidak terganggu. Kegagalan atau batas kuota pada salah satu API platform lowongan dicatat di log dan tidak menghentikan pemrosesan platform lain (NFR-09).



Gambar 2. Alur pemrosesan asinkron backend ProjectMatch AI

4.2 Registrasi, Login, dan Penautan GitHub (FR-01, FR-02)

Pengguna mendaftar lewat `POST /auth/register`, lalu masuk lewat `POST /auth/login` dan menerima JWT. Frontend mengarahkan pengguna ke dashboard sesuai role. Mahasiswa kemudian memperbarui profil dan username GitHub lewat `PUT /profiles/me`, lalu memicu pemrosesan portofolio lewat `POST /profiles/github/sync`.

Pada sinkronisasi GitHub, backend memeriksa token, format username, serta keberadaan akun dan jumlah repositori publiknya lewat GitHub API. Jika akun tidak memiliki repositori publik, backend memberi respons peringatan agar mahasiswa membuka minimal satu repositori publik sebelum embedding dihitung. Jika valid, `github_username` diperbarui, `processing_status` diset `pending`, dan tugas pengambilan data diantrekan. Respons 200 langsung dikembalikan tanpa menunggu proses selesai. Frontend lalu memantau kemajuan dengan memanggil `GET /profiles/me` secara berkala sampai status `done` atau `failed`.

4.3 Pembuatan Portofolio dari GitHub (FR-03)

Tugas latar belakang mengubah status menjadi `processing`, lalu GitHubClient mengambil daftar repositori publik, language breakdown, dan `README.md` repositori teratas. Hasilnya disimpan ke `github_data` (repositori teratas, total commit, dan languages breakdown). Jika GitHub membalas 403 atau 429 (rate limit), proses memakai `github_data` yang sudah tersimpan sehingga tidak gagal.

PortfolioSynthesizerService mengirim agregasi teks itu ke Groq dengan prompt khusus untuk mengekstrak kompetensi riil, framework, tools, dan spesialisasi dalam format JSON, dengan percobaan ulang maksimal tiga kali. Setelah JSON didapat, `extracted_skills` diperbarui, teks digabung menjadi masukan embedding, vektor 384 dimensi dibuat dan di-upsert ke tabel `embeddings`, lalu status menjadi `done` dan pencocokan dipicu. Bila Groq gagal tiga kali, status menjadi `failed`.

4.4 Agregasi Lowongan dan Ekstraksi Kriteria (FR-04, FR-05)

Sinkronisasi lowongan dijalankan terjadwal oleh scheduler atau dipicu admin lewat `POST /jobs/sync`. JobIngestionService memanggil API LinkedIn (lowongan Part-Time atau Contract), Upwork (Fixed-Price atau Hourly), dan Fiverr (buyer request atau project brief). Setiap entri dicek terhadap `external_job_id` untuk mencegah duplikasi, lalu disimpan ke `job_vacancies` lengkap dengan `platform_source`, `job_type`, dan `external_url`, berstatus `open` dengan `processing_status = pending`, dan ekstraksi LLM diantrekan. Bila satu platform gagal atau terkena rate limit, galat dicatat di log dan platform lain tetap diproses. Daftar lowongan yang tampil ke pengguna lewat `GET /jobs` hanya yang berstatus `open`.

Di latar belakang, LLMExtractionService mengirim deskripsi lowongan ke Groq untuk dinormalisasi menjadi kualifikasi teknis, kategori keahlian, perkiraan durasi, lingkup pekerjaan, dan tipe kontrak. Hasil JSON valid disimpan ke `extracted_skills`, teksnya diubah

menjadi embedding, lalu status menjadi `done`. Bila Groq mengembalikan 429, timeout, atau JSON yang tidak dapat di-parse, percobaan diulang sampai tiga kali. Jika tetap gagal, status menjadi `failed` dan lowongan dapat diproses ulang. Karena `extracted_skills` disimpan permanen, LLM hanya dipanggil satu kali per lowongan.

4.5 Pencocokan dan Visualisasi (FR-06, FR-07)

Ketika mahasiswa membuka rekomendasi (`GET /recommendations/me`), backend memeriksa dulu apakah `processing_status` mahasiswa sudah `done`. Jika belum, sistem menjawab bahwa profil masih diproses. Jika sudah, untuk setiap lowongan berstatus `open` backend mencari match yang sudah tersimpan dan memakainya langsung tanpa menghitung ulang. Bila belum ada, Matching Engine meminta pgvector menghitung jarak cosine antara embedding mahasiswa dan lowongan.

Skor dihitung sebagai 1 dikurangi jarak cosine, dibatasi pada rentang 0.0 sampai 1.0, lalu disimpan dengan UPSERT ke tabel `matches` (satu baris per pasangan mahasiswa dan lowongan). Match baru yang skornya melampaui ambang batas memicu `notifyMatch` ke NotificationService. Daftar rekomendasi dikembalikan dalam urutan skor.

Untuk radar chart, frontend memanggil `GET /matches/{job_id}/me` dan menerima `similarity_score` serta `score_breakdown` per skill. Bila match belum ada, backend menjawab 404 dan frontend menampilkan pesan bahwa skor belum tersedia. Penggambaran radar, tombol overlay kualifikasi lowongan, dan drill-down per sumbu skill sepenuhnya dikerjakan di klien dengan Recharts.

4.6 Dasbor Lowongan, Bookmark, dan Pengalihan Lamaran (FR-08)

Mahasiswa membuka daftar lowongan lewat `GET /jobs` (dengan filter seperti platform dan remote) atau daftar rekomendasi terurut skor lewat `GET /recommendations/me`, lalu membuka detail beserta tautan lamaran lewat `GET /jobs/{id}`. Untuk menyimpan lowongan favorit, mahasiswa memanggil `POST /jobs/{job_id}/bookmark`; bila lowongan sudah disimpan, backend menjawab 409. Pendaftaran tidak diproses di dalam sistem: frontend mengarahkan (redirect) mahasiswa ke `external_url` di LinkedIn, Fiverr, atau Upwork, sehingga tidak ada lagi lamaran internal, peringkat kandidat, maupun perubahan status lamaran.

4.7 Notifikasi Real-time (FR-09)

Browser membuka koneksi `wss://.../ws/notifications` dengan JWT. Nginx meneruskan header Upgrade dan Connection ke WebSocketManager, yang memverifikasi token. Token valid membuat koneksi diterima dan didaftarkan di ConnectionManager; token tidak valid membuat koneksi ditutup.

Ketika ada kejadian baru, misalnya ditemukan lowongan dengan skor kecocokan tinggi, NotificationService lebih dulu menyimpan notifikasi ke tabel `notifications` dengan `is_read = false`, lalu mendorongnya lewat ConnectionManager. Pengguna yang online menerima

notifikasi langsung (toast dan badge). Pengguna yang offline tetap memiliki notifikasi di basis data dan mengambilnya lewat `GET /notifications` saat masuk kembali. Selama koneksi aktif, klien dan server bertukar heartbeat, dan klien menyambung ulang secara otomatis bila koneksi putus. Notifikasi yang dibaca ditandai lewat `PATCH /notifications/{id}/read`.

4.8 Penjadwalan dan Sinkronisasi Lowongan (FR-10)

Scheduler latar belakang (cron atau background worker) menjalankan siklus sinkronisasi berkala, misalnya batch tiap 6 atau 12 jam, untuk memperbarui ketersediaan lowongan dari LinkedIn, Fiverr, dan Upwork. Setiap siklus memanggil JobIngestionService untuk menarik lowongan, mengabaikan entri yang `external_job_id`-nya sudah ada, lalu mengantrekan ekstraksi LLM dan embedding bagi lowongan baru. Admin dapat memicu siklus yang sama secara manual lewat `POST /jobs/sync`.

Pada akhir siklus, lowongan yang sudah kedaluwarsa atau tidak lagi tersedia di platform asal diubah menjadi `expired` sehingga tidak tampil di daftar lowongan dan tidak dihitung pada rekomendasi. Bila satu platform membalas galat atau melampaui kuota, kegagalan itu dicatat di log dan siklus tetap berlanjut untuk platform lain. Jumlah lowongan per platform dapat dipantau admin lewat statistik (Bagian 4.9).

4.9 Administrasi Akun dan Statistik (FR-11, FR-12)

Pada manajemen akun, permintaan `GET /admin/users` melewati verifikasi token dan RBACGuard. Role selain admin dijawab 403. Admin dapat mengubah `is_active` atau role lewat `PATCH /admin/users`; input tidak valid dijawab 422 dan akun yang tidak ditemukan dijawab 404.

Pada statistik, `GET /admin/statistics` hanya dapat diakses admin. StatisticsService menjalankan empat agregasi secara paralel: jumlah pengguna per role, jumlah lowongan per platform (LinkedIn, Fiverr, Upwork), ringkasan pencocokan, dan interaksi mahasiswa seperti bookmark. Hasilnya digabung menjadi satu laporan dan ditampilkan sebagai kartu dan grafik di dasbor admin.

5. Pipeline AI: Ekstraksi, Embedding, dan Pencocokan

5.1 Prinsip Rancangan

AI Semantic Matching Engine tidak membandingkan kata kunci, tetapi membandingkan makna. Teks portofolio dari repositori GitHub mahasiswa dan kriteria lowongan sama-sama diubah menjadi vektor, lalu kemiripan keduanya diukur dengan cosine similarity. Dengan cara ini, deskripsi yang memakai istilah berbeda tetapi bermakna dekat tetap dapat dicocokkan. Untuk menjaga mutu input, LLM dipakai sebagai penyusun struktur dan pengekstrak kriteria, bukan sebagai penentu skor. Skor sepenuhnya berasal dari perhitungan vektor.

5.2 Tahapan Pipeline

Pipeline terdiri dari lima tahap.

1. **Pengambilan data sumber.** GitHubClient mengambil repositori publik, language breakdown, dan `README.md` mahasiswa lewat GitHub REST API, lalu menyimpannya di `github_data` sebagai cache. JobIngestionService menarik lowongan dari API LinkedIn, Fiverr, dan Upwork dan memfilter duplikat lewat `external_job_id`. Target sintesis portofolio kurang dari 10 detik per akun (NFR-03).
2. **Sintesis portofolio dan ekstraksi kriteria.** Groq API (model `Llama-3.1-8b-instant`) menerima agregasi teks GitHub atau deskripsi lowongan dan mengembalikan JSON. Untuk portofolio, JSON berisi kompetensi riil, framework, tools, dan spesialisasi. Untuk lowongan, JSON berisi kualifikasi kunci, kategori keahlian, perkiraan durasi, dan tipe kontrak, dengan target kurang dari 3 detik per entri. Hasilnya disimpan di kolom `extracted_skills` pada `mahasiswa_profiles` atau `job_vacancies`.
3. **Embedding.** Sentence Transformers dengan model multibahasa 384 dimensi (misalnya `paraphrase-multilingual-MiniLM-L12-v2`) mengubah teks hasil sintesis menjadi vektor. Vektor disimpan di tabel `embeddings` bersama `model_name`. Setiap baris hanya milik salah satu: mahasiswa atau lowongan.
4. **Pencocokan.** Matching Engine memakai operator cosine distance pgvector, menghitung skor sebagai 1 dikurangi jarak, membatasinya pada 0.0--1.0, lalu menyimpan `similarity_score` dan `score_breakdown` di tabel `matches`. Pencarian vektor memakai indeks HNSW dengan operator `vector_cosine_ops`.
5. **Notifikasi.** Bila skor melampaui ambang batas, NotificationService menyimpan notifikasi dan ConnectionManager mendorongnya ke mahasiswa lewat WebSocket.

5.3 Pemicu Pencocokan

Pencocokan dipicu pada dua titik. Pertama, setelah embedding selesai dibuat dan status menjadi `done`, baik untuk profil mahasiswa maupun lowongan baru, backend memicu Matching Engine (terlihat pada pertemuan dua jalur di Gambar 2). Kedua, saat mahasiswa

membuka rekomendasi, pasangan mahasiswa dan lowongan yang belum punya match dihitung pada saat itu juga, sedangkan yang sudah tersimpan dipakai ulang. Karena pasangan (`mahasiswa_id`, `job_vacancy_id`) bersifat unik, pencocokan ulang hanya memperbarui baris yang sudah ada.

5.4 Pemisahan Status

Tabel 6. Pemisahan status pada sistem

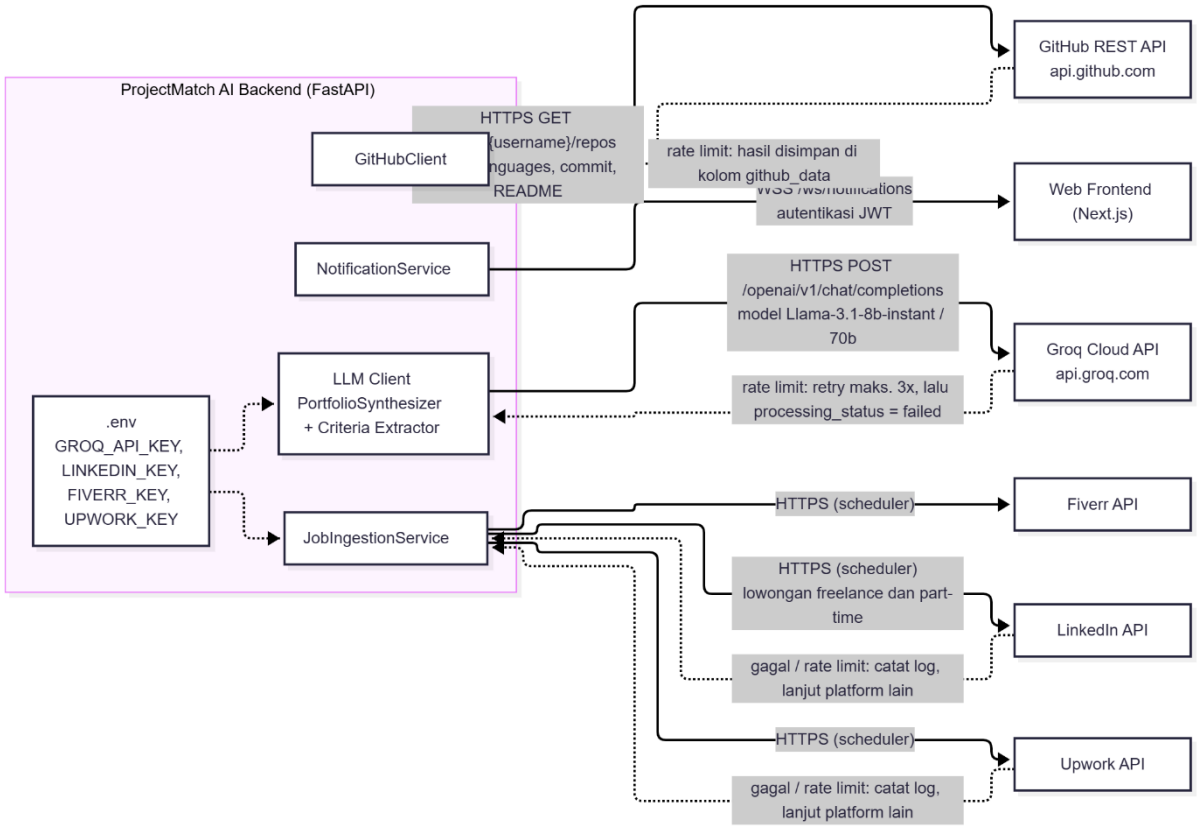
Jenis	Nilai	Makna
`processing_status` (mahasiswa dan lowongan)	`pending`, `processing`, `done`, `failed`	Status proses latar belakang: sintesis portofolio GitHub, ekstraksi LLM lowongan, dan embedding. `failed` dapat diproses ulang.
`status` lowongan	`open`, `expired`, `closed`	Hanya lowongan `open` yang tampil di daftar lowongan dan dihitung pada rekomendasi. `expired` untuk lowongan kedaluwarsa hasil pembersihan scheduler.
`platform_source` dan `job_type` (lowongan)	`linkedin`, `fiverr`, `upwork`, `manual`; `freelance`, `part_time`	Klasifikasi asal platform dan jenis kontrak lowongan; bukan status proses.
`type` notifikasi	`match_result`, `system`	Jenis notifikasi yang dikirim ke pengguna.

5.5 Penanganan Kegagalan

Kegagalan pada layanan eksternal tidak boleh merambat ke fitur lain. Karena itu, kegagalan Groq ditangani dengan percobaan ulang maksimal tiga kali, status `failed`, dan pencatatan log. Kegagalan GitHub ditangani dengan memakai `github_data` yang sudah tersimpan. Akun GitHub tanpa repositori publik dihentikan sejak sinkronisasi dengan respons peringatan. Kegagalan atau rate limit pada API LinkedIn, Upwork, atau Fiverr dicatat di log dan tidak menghentikan platform lain, sementara lowongan yang sudah tersimpan tetap dipakai. Karena embedding yang sudah tersimpan tidak dihapus, pencocokan untuk data yang sudah selesai diproses tetap berjalan (NFR-09).

6. Integrasi Layanan Pihak Ketiga

Backend berintegrasi dengan lima layanan eksternal (Groq, GitHub, LinkedIn, Upwork, dan Fiverr) serta menyediakan satu kanal real-time sendiri (WebSocket), di samping REST API yang menjadi antarmuka utama dengan frontend.



Gambar 3. Peta integrasi backend ProjectMatch AI

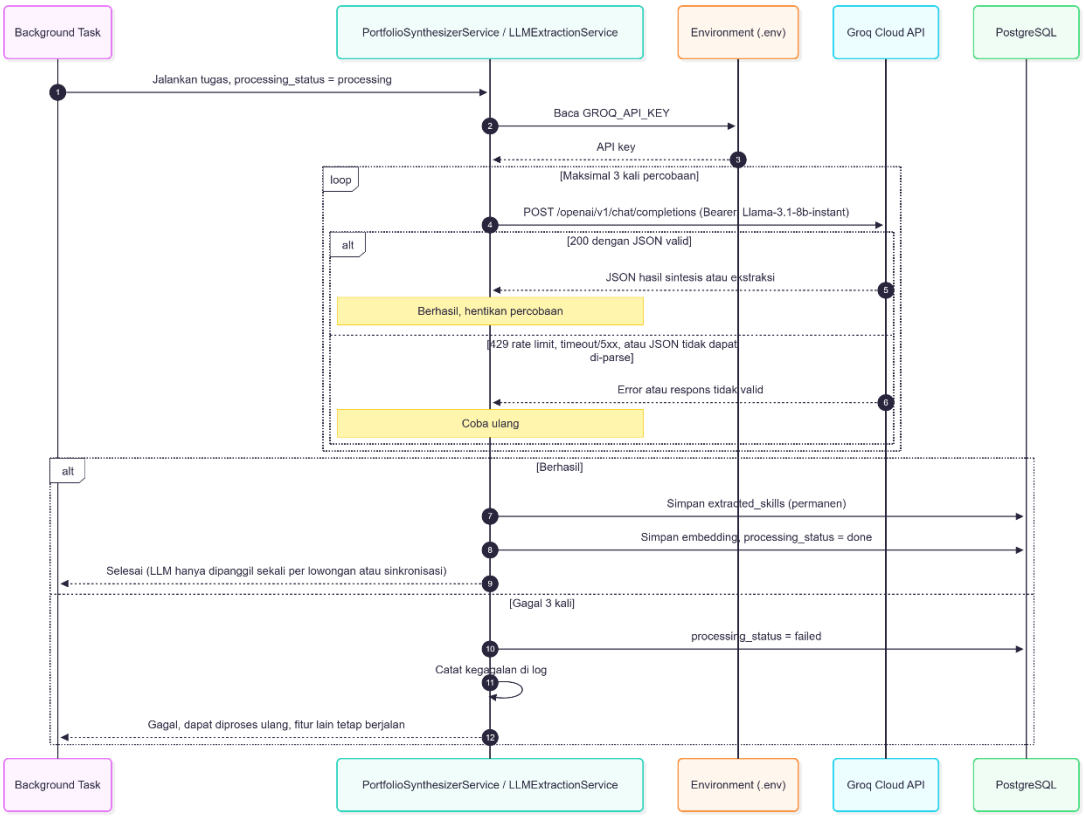
Tabel 7. Antarmuka integrasi backend

Antarmuka	Arah	Protokol dan endpoint	Autentikasi	Penanganan kegagalan
FastAPI REST API	Frontend ke backend	HTTPS, awalan `/api/v1`	JWT dan RBAC, kecuali registrasi dan login	Kode galat HTTP baku (401, 403, 404, 409, 422)
Groq Cloud API	Backend ke layanan eksternal	HTTPS POST `https://api.groq.com/openai/v1/chat/completions`, model `Llama-3.1-8b-instant`	API key dari `.env`	Coba ulang maks. 3 kali, lalu `failed` dan dapat diproses ulang
GitHub REST API	Backend ke layanan eksternal	HTTPS GET `/users/{username}/repos`, `/repos/{owner}/{repo}/languages`, dan `/repos/{owner}/{repo}/readme` pada `api.github.com`	Publik	Hasil di-cache di `github_data`; saat rate limit memakai data tersimpan

Antarmuka	Arah	Protokol dan endpoint	Autentikasi	Penanganan kegagalan
LinkedIn Job Search API	Backend ke layanan eksternal	HTTPS (LinkedIn API atau RapidAPI Job Search); lowongan Part-Time atau Contract	API key dari `.env`	Polling terjadwal; galat atau rate limit dicatat di log dan platform lain tetap diproses
Upwork API	Backend ke layanan eksternal	HTTPS (OAuth 2.0, GraphQL Job Search API); lowongan Fixed-Price atau Hourly	OAuth 2.0; kredensial dari `.env`	Polling terjadwal; galat atau rate limit dicatat di log dan platform lain tetap diproses
Fiverr API / RapidAPI Fiverr	Backend ke layanan eksternal	HTTPS; permintaan proyek pembeli (buyer request atau project brief) atau lowongan jasa terkait	API key dari `.env`	Polling terjadwal; galat atau rate limit dicatat di log dan platform lain tetap diproses
WebSocket	Backend ke frontend	`wss://.../ws/notifications` lewat Nginx	JWT	Notifikasi tetap disimpan di basis data bila pengguna offline; klien menyambung ulang otomatis

6.1 Integrasi Groq Cloud API

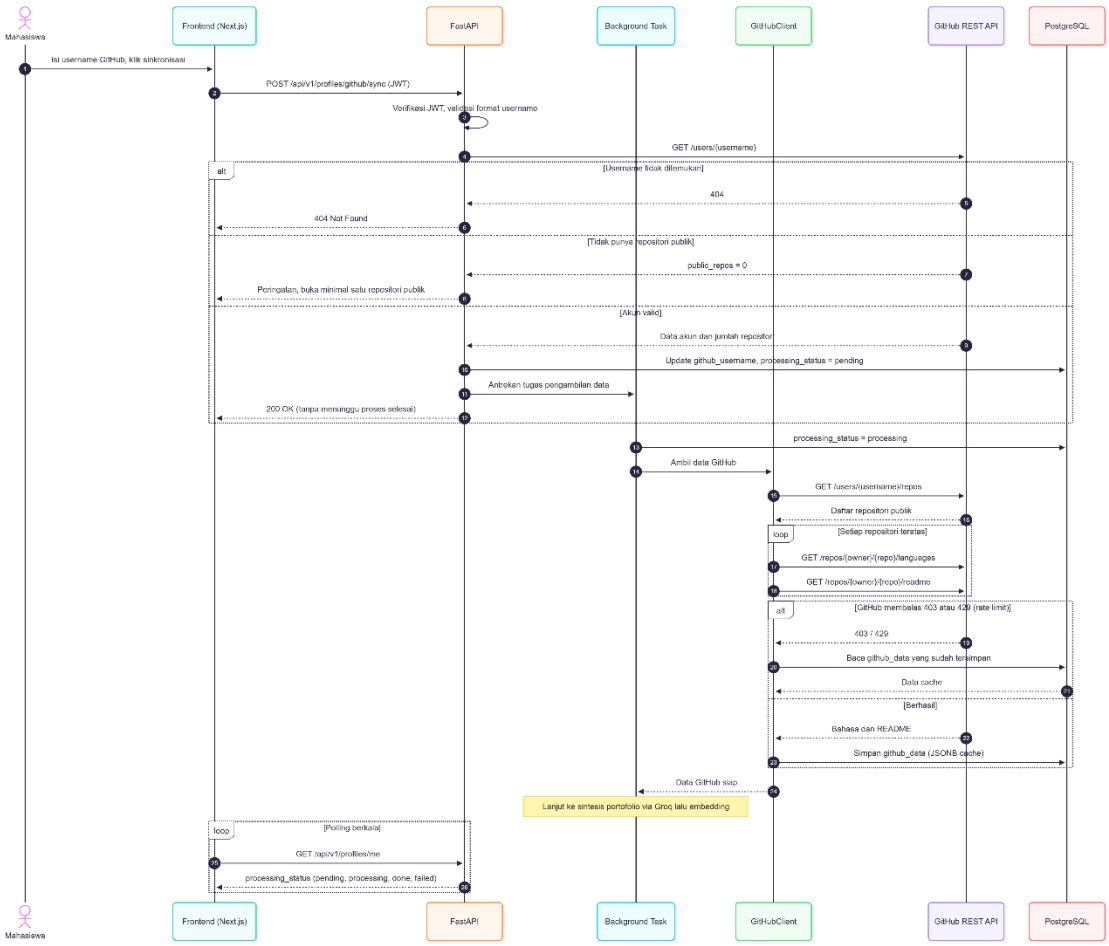
Pemanggilan Groq dilakukan dari tugas latar belakang oleh PortfolioSynthesizerService, yang merangkum data mentah GitHub menjadi resume kompetensi, dan oleh LLMExtractionService, yang mengekstrak kriteria dari deskripsi lowongan freelance atau part-time. Sebelum memanggil, layanan membaca `GROQ_API_KEY` dari environment variable. Permintaan dikirim dengan header Authorization Bearer. Respons dibedakan menjadi empat kemungkinan: 200 dengan JSON valid, 429 karena rate limit, timeout atau 5xx karena galat server, dan respons yang tidak dapat di-parse sebagai JSON. Tiga kemungkinan terakhir memicu percobaan ulang sampai maksimal tiga kali. Bila berhasil, `extracted_skills` diperbarui secara permanen dan status menjadi `done`, sehingga LLM hanya dipanggil satu kali per lowongan dan per sinkronisasi profil GitHub. Bila gagal, status menjadi `failed`, kegagalan dicatat di log, dan fitur lain tetap berjalan.



Gambar 4. Sequence diagram integrasi Groq API

6.2 Integrasi GitHub REST API

Backend mengambil username dari `github_username` yang sudah divalidasi formatnya. Pada setiap sinkronisasi, GitHubClient memanggil `GET /users/{username}/repos` untuk daftar repositori publik, lalu `GET /repos/{owner}/{repo}/languages` dan `GET /repos/{owner}/{repo}/readme` untuk repositori teratas. Username yang tidak ditemukan dijawab 404 dan dilaporkan ke pengguna. Metadata, language breakdown, dan isi README disimpan ke kolom JSONB `github_data` sebagai cache untuk mencegah pembatasan kuota. Bila GitHub membalas 403 atau 429, proses memakai data tersimpan dan tidak digagalkan.



Gambar 5. Sequence diagram integrasi GitHub API

6.3 Integrasi API Lowongan Eksternal (LinkedIn, Upwork, Fiverr)

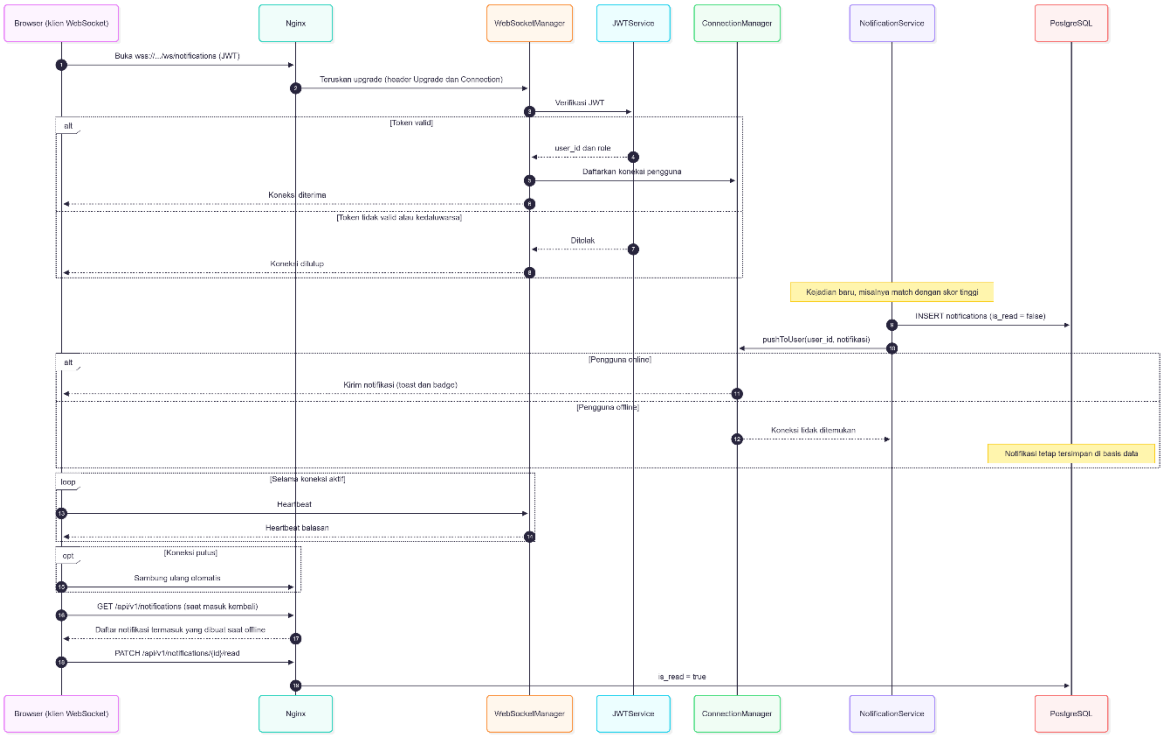
JobIngestionService memanggil ketiga API platform dari worker terjadwal atau dari endpoint `POST /jobs/sync`. LinkedIn Job Search API (atau RapidAPI Job Search) dipakai untuk lowongan bertipe Part-Time atau Contract, dengan data judul, perusahaan, deskripsi, Apply URL, serta lokasi atau status remote. Upwork API (OAuth 2.0 dan GraphQL Job Search) dipakai untuk lowongan Fixed-Price atau Hourly, dengan data kategori, skill yang dibutuhkan, feedback klien, anggaran, dan tautan proyek. Fiverr API atau RapidAPI Fiverr dipakai untuk permintaan proyek pembeli (buyer request atau project brief) dan lowongan jasa terkait.

Tiap entri dinormalisasi ke skema `job_vacancies` dengan `platform_source`, `job_type`, dan `external_url`, lalu dicek terhadap `external_job_id` untuk mencegah duplikasi. Untuk menghormati batas kuota, penarikan dilakukan secara batch terjadwal (misalnya tiap 6 atau 12 jam) dan hasilnya disimpan di basis data lokal. Bila satu API gagal atau mencapai rate limit, galat dicatat di log dan siklus tetap memproses platform lain (NFR-09).

6.4 Kanal WebSocket

Kanal WebSocket dikelola oleh endpoint FastAPI dan `ConnectionManager`. Nginx dikonfigurasi untuk meneruskan upgrade WebSocket. Setelah JWT diverifikasi oleh

JWTService, koneksi pengguna didaftarkan sehingga NotificationService dapat memanggil `pushToUser`. Alur lengkapnya dijelaskan pada Bagian 4.7.



Gambar 6. Sequence diagram integrasi WebSocket

7. Peran dan Hak Akses

SKPL mendefinisikan aktor manusia dan sistem, sedangkan RBAC pada backend mengikuti dua role: mahasiswa dan admin. Role mitra dan dosen peneliti tidak lagi dipakai karena lowongan berasal dari platform eksternal (LinkedIn, Fiverr, Upwork) yang diagregasi otomatis oleh Sistem lewat scheduler, bukan diinput oleh pengguna.

Tabel 8. Peran dan hak akses

Peran	Hak utama	Pembatasan
Mahasiswa	Mengelola profil dan username GitHub, memicu sinkronisasi portofolio, melihat daftar dan rekomendasi lowongan freelance dan part-time beserta skor dan radar chart, menyimpan lowongan (bookmark), membuka tautan lamaran di platform asal, serta menerima notifikasi.	Hanya mengakses data miliknya. Tidak dapat mengakses endpoint `/admin` maupun `POST /jobs/sync`.
Administrator	Mengelola akun, status aktivasi, dan role pengguna, memicu sinkronisasi lowongan secara manual, serta melihat statistik agregasi lowongan per platform dan pencocokan.	Akses endpoint `/admin` dan `POST /jobs/sync` hanya untuk role ini. Role lain dijawab 403.

Pemeriksaan hak akses dilakukan di backend pada setiap permintaan, bukan hanya dengan menyembunyikan menu di antarmuka. Skenario uji TC-11 memverifikasi hal ini dengan mencoba mengakses endpoint admin memakai akun mahasiswa dan mengharapkan respons 403.

8. Aturan Data dan Konsistensi

Aturan berikut dijaga di level basis data maupun di level aplikasi agar data tetap konsisten, terutama karena pemrosesan berjalan asinkron dan bisa dipicu berulang kali.

Tabel 9. Aturan data dan konsistensi

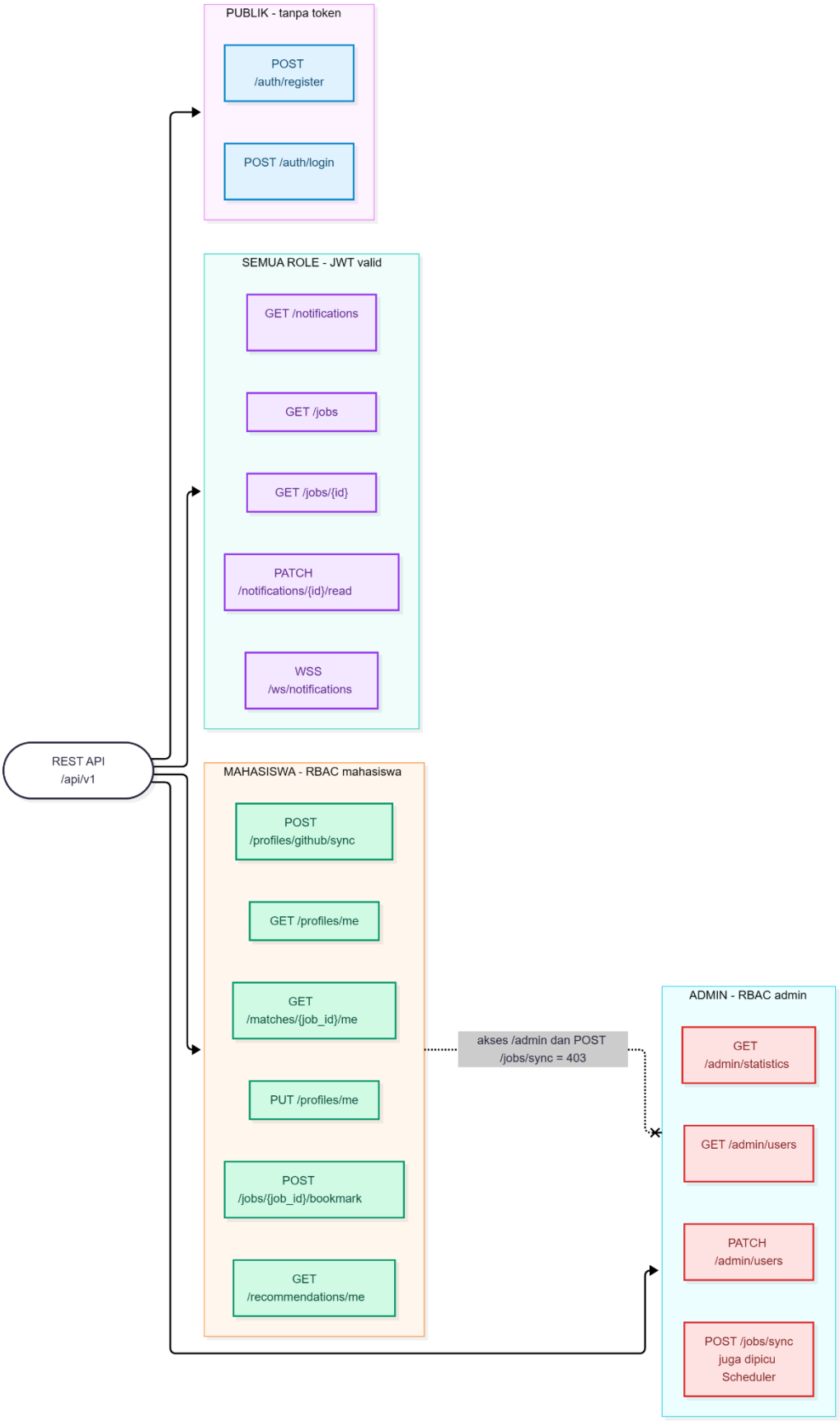
Kode	Aturan	Penerapan
D01	Email dan NIM tidak boleh ganda.	`users.email` UNIQUE dan `mahasiswa_profiles.nim` UNIQUE; registrasi dengan email yang sudah ada dijawab 409.
D02	Satu akun memiliki paling banyak satu profil.	`user_id` bersifat UNIQUE pada `mahasiswa_profiles`.
D03	Password tidak pernah disimpan dalam bentuk asli.	Kolom `password_hash` berisi hasil Argon2id.
D04	Satu embedding hanya milik satu entitas.	Constraint CHECK memastikan tepat satu dari `mahasiswa_id` atau `job_vacancy_id` terisi; keduanya UNIQUE sehingga embedding diperbarui, bukan digandakan.
D05	Satu pencocokan per pasangan mahasiswa dan lowongan.	Pasangan (`mahasiswa_id`, `job_vacancy_id`) UNIQUE pada `matches`; pencocokan ulang melakukan UPSERT.
D06	Skor kecocokan selalu valid.	CHECK 0.0--1.0 pada `similarity_score`, dengan pembatasan (clamp) nilai di backend sebelum disimpan.
D07	Mahasiswa tidak dapat menyimpan bookmark yang sama dua kali.	Pasangan (`mahasiswa_id`, `job_vacancy_id`) UNIQUE pada `job_bookmarks`; bookmark ganda dijawab 409.
D08	Hanya lowongan terbuka yang ditawarkan.	`GET /jobs` dan perhitungan rekomendasi memfilter `status` = 'open'.
D09	Proses yang gagal dapat diulang.	`processing_status` bertipe ENUM dengan nilai `failed`; profil atau lowongan yang gagal dapat diproses ulang.
D10	Identitas pelaku diambil dari token.	`user_id` dan role dibaca dari JWT; kepemilikan profil, bookmark, dan notifikasi diperiksa di query atau service.
D11	Satu akun GitHub hanya terhubung ke satu profil.	`github_username` UNIQUE dan NOT NULL pada `mahasiswa_profiles`.
D12	Waktu perubahan data tercatat otomatis.	Trigger `updated_at` pada tabel profil mahasiswa, lowongan, embedding, dan match.
D13	Rahasia tidak masuk ke repositori.	`GROQ_API_KEY`, kunci API LinkedIn, Upwork, dan Fiverr, kredensial basis data, dan secret JWT berada di `.env` pada server.
D14	Lowongan tidak digandakan saat penarikan API.	Pasangan (`platform_source`, `external_job_id`) UNIQUE pada `job_vacancies`; penarikan ulang memperbarui, bukan menggandakan.

9. Endpoint Inti Backend

Seluruh API bisnis memakai awalan `/api/v1`. Setiap endpoint terlindungi memeriksa token, role, dan, bila relevan, kepemilikan data. Pemetaan endpoint per role ditunjukkan pada Gambar 7.

Tabel 10. Endpoint inti backend

Modul	Metode dan endpoint	Fungsi	Akses (FR)
Auth	`POST /auth/register`	Registrasi akun baru	Publik (FR-01)
Auth	`POST /auth/login`	Login dan penerbitan token JWT	Publik (FR-01)
Profil	`GET` dan `PUT /profiles/me`	Melihat dan memperbarui profil sendiri, termasuk username GitHub (tanpa input berkas)	Mahasiswa (FR-02)
Profil	`POST /profiles/github/sync`	Memvalidasi akun GitHub dan memicu pengambilan data, sintesis portofolio, dan embedding di latar belakang	Mahasiswa (FR-02, FR-03)
Lowongan	`GET /jobs`	Daftar lowongan freelance dan part-time berstatus open (filter: platform, remote, dll.)	Semua role (FR-04, FR-08)
Lowongan	`GET /jobs/{id}`	Detail lowongan eksternal dan tautan lamaran	Semua role (FR-08)
Lowongan	`POST /jobs/sync`	Memicu penarikan data dari LinkedIn, Fiverr, dan Upwork	Admin atau Scheduler (FR-04, FR-10)
Lowongan	`POST /jobs/{job_id}/bookmark`	Menyimpan lowongan favorit untuk dilamar mahasiswa	Mahasiswa (FR-08)
Pencocokan	`GET /recommendations/me`	Rekomendasi lowongan freelance dan part-time yang cocok dengan portofolio GitHub mahasiswa yang sedang login	Mahasiswa (FR-06)
Pencocokan	`GET /matches/{job_id}/me`	Detail skor dan breakdown per skill untuk radar chart overlay	Mahasiswa (FR-06, FR-07)
Notifikasi	`GET /notifications`	Daftar notifikasi pengguna	Semua role (FR-09)
Notifikasi	`PATCH /notifications/{id}/read`	Menandai notifikasi sudah dibaca	Semua role (FR-09)
Notifikasi	`WSS /ws/notifications`	Kanal real-time notifikasi lowongan baru yang cocok, diautentikasi JWT	Semua role (FR-09)
Admin	`GET` dan `PATCH /admin/users`	Melihat akun serta mengubah status aktivasi dan role	Admin (FR-11)
Admin	`GET /admin/statistics`	Analitik jumlah lowongan per platform (LinkedIn, Fiverr, Upwork), statistik kecocokan, dan interaksi mahasiswa	Admin (FR-12)



Gambar 7. Peta endpoint REST API per role

9.1 Kode Respons yang Digunakan

Tabel 11. Kode respons HTTP yang muncul pada alur backend

Kode	Situasi
200 OK	Permintaan berhasil, misalnya sinkronisasi GitHub diterima atau daftar berhasil diambil.
201 Created	Akun atau bookmark berhasil dibuat.
401 Unauthorized	Kredensial login tidak valid atau token tidak valid atau kedaluwarsa.
403 Forbidden	Role tidak berhak atau akun dinonaktifkan.
404 Not Found	Data tidak ditemukan, misalnya username GitHub tidak ada, match belum tersedia, atau lowongan tidak ada.
409 Conflict	Email sudah terdaftar atau lowongan sudah disimpan sebagai bookmark.
422 Unprocessable Entity	Validasi input gagal.

10. Non-Fungsional dan Cara Verifikasi

Tabel 12. Kebutuhan non-fungsional dan cara verifikasinya

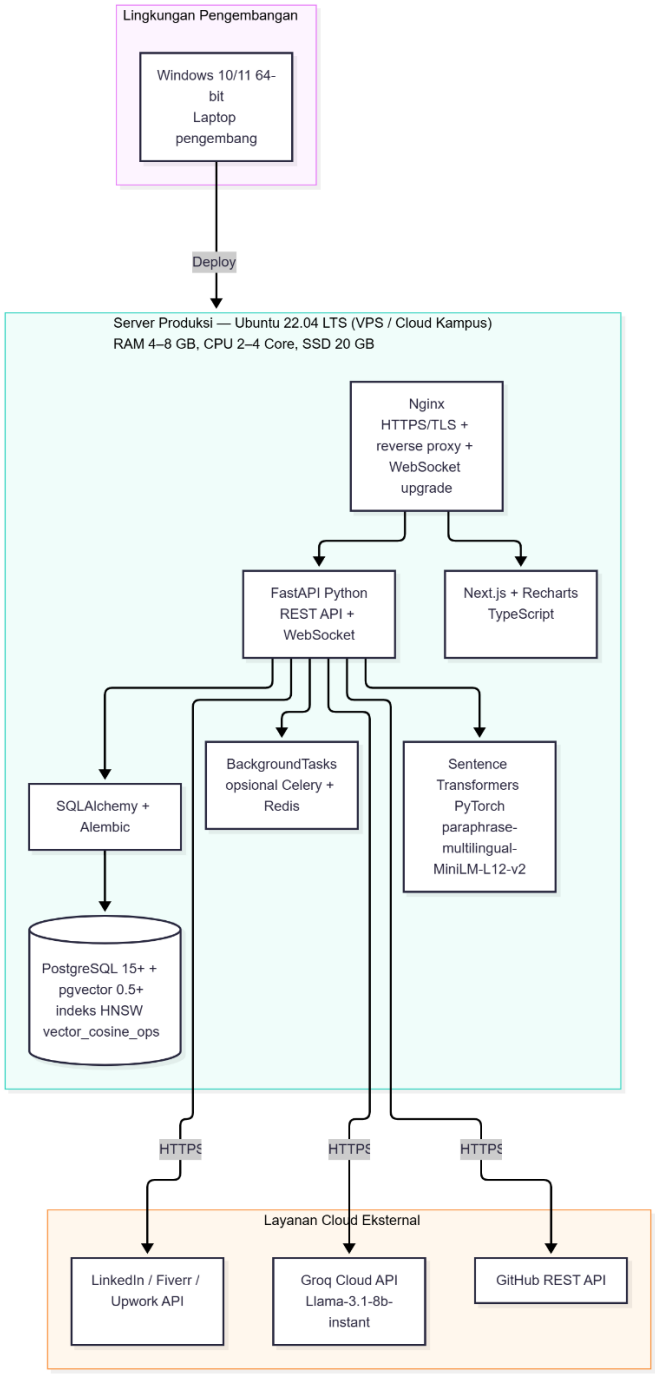
Kode	Ketentuan (SKPL)	Penerapan di backend	Verifikasi
NFR-01	Akurasi semantic matching portofolio GitHub dengan lowongan eksternal minimal 80% pada UAT	Embedding multibahasa 384 dimensi, cosine similarity, ekstraksi kriteria JSON terstruktur oleh LLM, teks portofolio disusun dari README dan metadata repositori sebelum diolah	TC-04: skor dibandingkan penilaian pakar pada sampel portofolio GitHub dan lowongan uji
NFR-02	Rekomendasi lowongan kurang dari 1 menit; notifikasi lowongan cocok kurang dari 1 menit setelah match tersimpan	Hasil match disimpan dan dipakai ulang, pencarian vektor dengan indeks HNSW, proses berat di latar belakang, notifikasi lewat WebSocket	TC-10: waktu daftar rekomendasi; TC-06: waktu notifikasi diterima
NFR-03	Pengambilan dan sintesis portofolio GitHub kurang dari 10 detik per akun; penguraian satu lowongan eksternal oleh LLM kurang dari 3 detik	GitHubClient dengan cache `github_data`, Groq (`Llama-3.1-8b-instant`) untuk sintesis dan ekstraksi, proses dijalankan di latar belakang	TC-09: waktu pengambilan dan sintesis portofolio GitHub; TC-12: agregasi dan penguraian lowongan eksternal
NFR-04	Kepuasan pengguna minimal 4,0 dari 5,0; SUS minimal 68	Respons galat yang konsisten, status pemrosesan yang dapat dipantau frontend	Kuesioner SUS pada responden saat UAT
NFR-05	Argon2id, JWT dengan expiry, RBAC, HTTPS/TLS, perlindungan SQL Injection dan XSS, API key (LinkedIn, Upwork, Fiverr, Groq) di .env	Lihat Bab 3	TC-02: login gagal; TC-11: akses lintas role ditolak 403
NFR-06	Uptime minimal 99% (24/7)	Nginx sebagai pintu masuk, status proses disimpan di basis data sehingga proses gagal dapat diulang	Pemantauan ketersediaan layanan selama masa uji coba (usulan, belum ada test case di SKPL)
NFR-07	Skalabilitas; proses berat asinkron	BackgroundTasks (opsional Celery + Redis), hasil match disimpan, agregasi statistik di basis data	Pengujian dengan jumlah data yang ditingkatkan bertahap (usulan, belum ada test case di SKPL)
NFR-08	Dapat diakses lintas perangkat lewat web responsive Next.js	REST API dan WebSocket yang tidak bergantung pada perangkat tertentu	Pengujian antarmuka pada beberapa perangkat dan browser modern (usulan)

Kode	Ketentuan (SKPL)	Penerapan di backend	Verifikasi
NFR-09	Kegagalan atau rate limit pada Groq atau salah satu API platform lowongan tidak menghentikan fitur lain	Retry maksimal 3 kali ke Groq, status `failed` dapat diproses ulang, galat per platform dicatat dan platform lain tetap diproses, lowongan dan embedding tersimpan tetap dipakai	TC-08: Groq gagal atau melebihi kuota; simulasi galat atau rate limit pada salah satu API platform (usulan)

Angka-angka pada kolom ketentuan adalah target dari SKPL, bukan hasil pengukuran. Hasil pengujian dicatat pada laporan pengujian, bukan pada dokumen rancangan ini.

11. Penempatan Komponen dan Infrastruktur

Seluruh komponen aplikasi dijalankan pada satu server produksi Ubuntu 22.04 LTS, berupa VPS atau layanan cloud kampus. Perangkat pengembang memakai Windows 10/11 dan menjalankan migrasi skema dengan Alembic ke basis data. Pada server, browser pengguna terhubung lewat HTTPS dan WSS ke Nginx, yang meneruskan permintaan ke proses FastAPI (Uvicorn). Proses itu memuat GitHubClient, JobIngestionService, Sentence Transformers, REST API, dan WebSocket. Basis data PostgreSQL diakses lewat koneksi internal TCP 5432. Panggilan ke Groq, GitHub, LinkedIn, Upwork, dan Fiverr keluar lewat HTTPS.



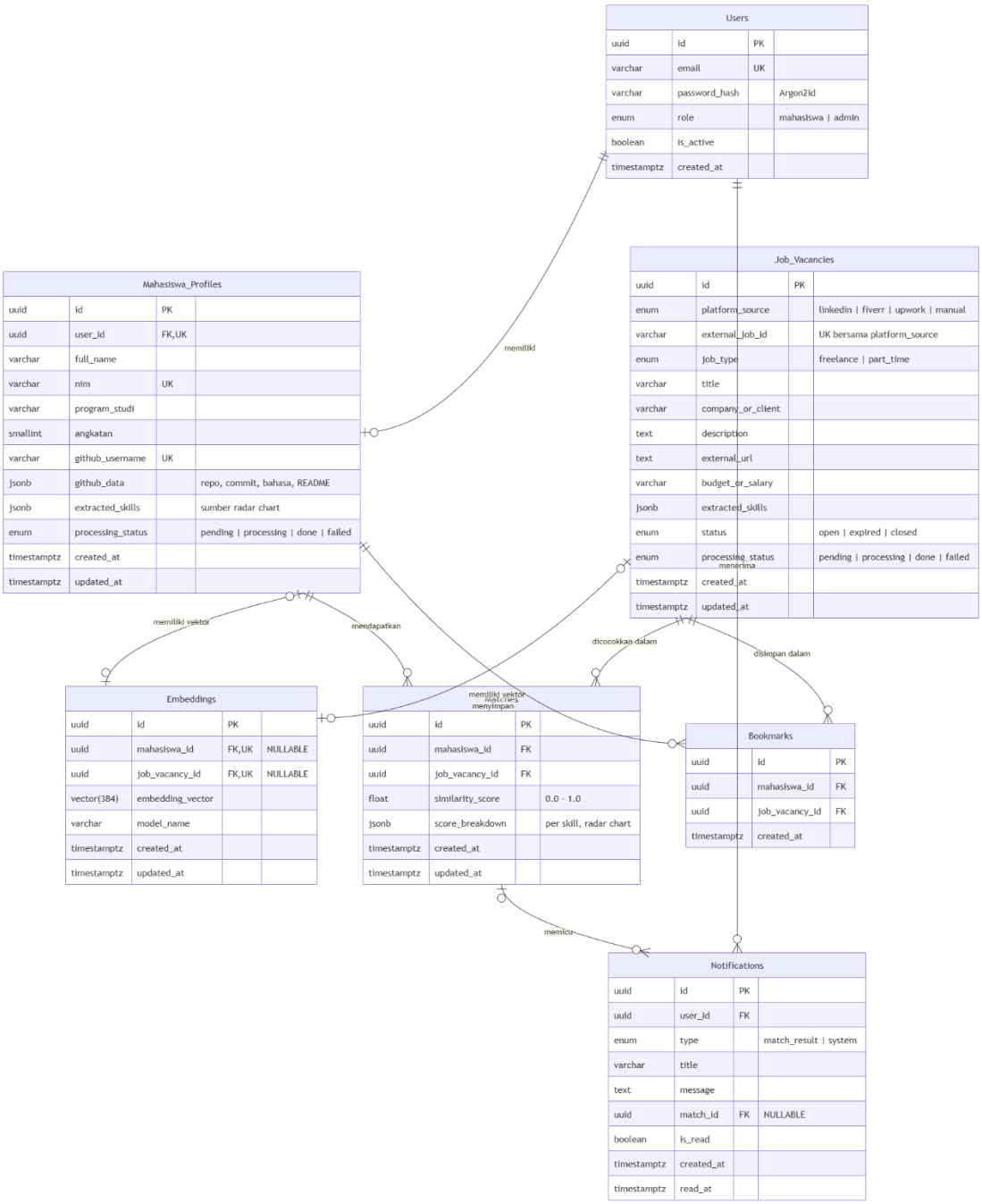
Gambar 8. Deployment diagram ProjectMatch AI

Tabel 13. Penempatan komponen

Komponen	Penempatan
Klien	Browser pengguna: Next.js dengan Recharts dan klien WebSocket.
Nginx	Pada server produksi. Menangani HTTPS/TLS, reverse proxy ke FastAPI, dan upgrade WebSocket.
Aplikasi FastAPI (Uvicorn)	Pada server produksi. Satu proses yang memuat router, GitHubClient, JobIngestionService, Sentence Transformers, REST API, dan WebSocket.
Pemrosesan latar belakang	FastAPI BackgroundTasks sebagai baseline, ditambah scheduler (cron atau background worker) untuk sinkronisasi lowongan berkala. Celery Worker dengan Redis bersifat opsional bila beban membutuhkannya.
PostgreSQL 15+ dengan pgvector 0.5+	Pada server yang sama atau layanan basis data kampus. Diakses internal lewat TCP 5432.
Penyimpanan data	Hanya basis data PostgreSQL dan cache metadata (`github_data`). Tidak ada penyimpanan berkas PDF, video, atau citra e-KYC.
Konfigurasi rahasia	File `.env` pada server berisi `GROQ_API_KEY`, kunci API LinkedIn, Upwork, dan Fiverr, kredensial basis data, dan secret JWT.
Layanan eksternal	Groq Cloud API, GitHub REST API, serta API LinkedIn, Upwork, dan Fiverr, dipanggil lewat HTTPS keluar.
Perangkat keras server	RAM 4--8 GB, CPU 2--4 core, SSD 20 GB. Ringan karena inferensi LLM dialihkan ke Groq.

12. Model Data

Basis data terdiri dari tujuh tabel utama: `users`, `mahasiswa_profiles`, `job_vacancies`, `embeddings`, `matches`, `job_bookmarks`, dan `notifications`. Skema fisik dirancang untuk PostgreSQL 15 atau lebih baru dengan pgvector 0.5 atau lebih baru. Skrip DDL lengkap (tipe ENUM, tabel, indeks, dan trigger `updated_at`) tersedia pada berkas `projectmatch_schema.sql` yang dirujuk SKPL dan perlu diperbarui agar sesuai dengan perubahan skema pada bab ini.



Gambar 9. Entity Relationship Diagram (ERD) ProjectMatch AI

Tabel 14. Ringkasan entitas dan relasi

Entitas	Fungsi	Atribut utama	Relasi penting
`users`	Akun seluruh pengguna	`id`, `email`, `password_hash`, `role`, `is_active`, `created_at`	Satu user mahasiswa memiliki satu profil. Juga menjadi penerima notifikasi.
`mahasiswa_profiles`	Profil dan portofolio GitHub mahasiswa	`nim`, `github_username`, `github_data`, `extracted_skills`, `processing_status`	Terhubung ke `users`; memiliki satu embedding, banyak match, dan banyak bookmark.
`job_vacancies`	Lowongan freelance dan part-time hasil agregasi LinkedIn, Fiverr, dan Upwork	`platform_source`, `external_job_id`, `job_type`, `title`, `company_or_client`, `description`, `external_url`, `budget_or_salary`, `extracted_skills`, `status`, `processing_status`	Memiliki satu embedding, banyak match, dan banyak bookmark. Pasangan (`platform_source`, `external_job_id`) unik.
`embeddings`	Vektor makna portofolio mahasiswa atau lowongan	`embedding_vector` (VECTOR 384), `model_name`, `mahasiswa_id`, `job_vacancy_id`	Tepat satu dari `mahasiswa_id` atau `job_vacancy_id` terisi. Diindeks HNSW dengan cosine.
`matches`	Hasil pencocokan	`similarity_score`, `score_breakdown`	Menghubungkan satu mahasiswa dan satu lowongan (pasangan unik).
`job_bookmarks`	Lowongan favorit mahasiswa	`mahasiswa_id`, `job_vacancy_id`, `created_at`	Menghubungkan mahasiswa dan lowongan (pasangan unik); menggantikan lamaran internal.
`notifications`	Notifikasi pengguna	`type`, `title`, `message`, `is_read`, `read_at`, `match_id`	Dimiliki satu user; opsional merujuk match.

13. Diagram UML

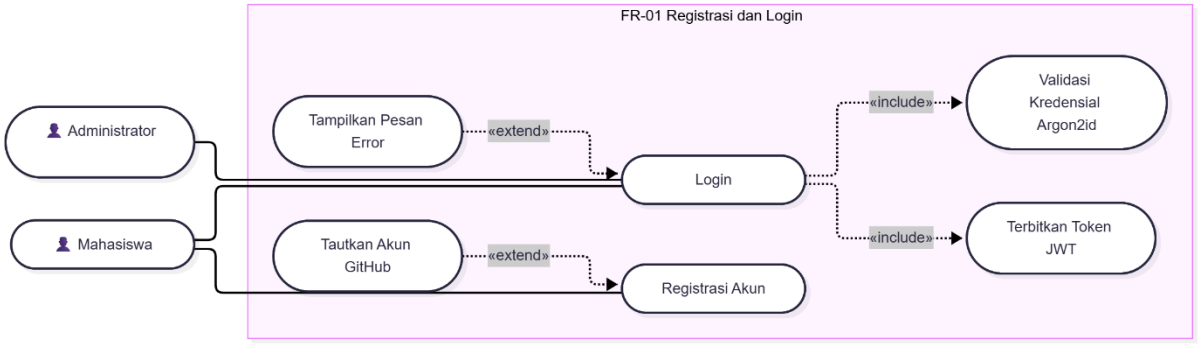
Bab ini memuat empat jenis diagram UML untuk setiap kebutuhan fungsional: use case diagram, activity diagram, class diagram, dan sequence diagram. Diagram disusun per FR sehingga setiap fitur dapat ditelusuri dari aktor yang terlibat sampai pertukaran pesan antarkomponen backend. Sequence diagram memakai fragmen alt, opt, loop, dan par untuk menunjukkan percabangan dan perulangan.

13.1 Use Case Diagram

Use case diagram menunjukkan aktor dan fungsi yang dapat mereka jalankan pada tiap fitur.

a. Use Case Diagram FR-01: Registrasi & Login

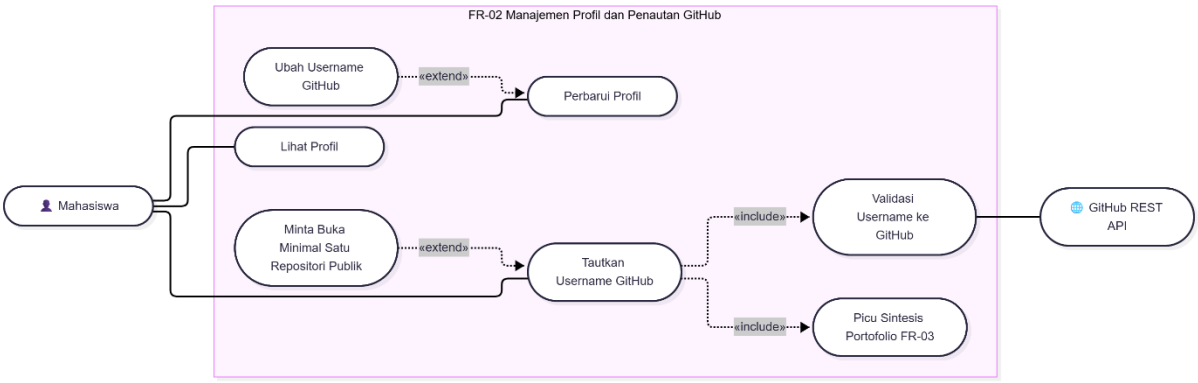
Aktor: Semua Aktor. Pengguna dari semua role mendaftar dan masuk ke sistem; login mencakup verifikasi password dan penerbitan JWT.



Gambar 10. Use Case Diagram FR-01: Registrasi & Login

b. Use Case Diagram FR-02: Manajemen Profil & Penautan GitHub

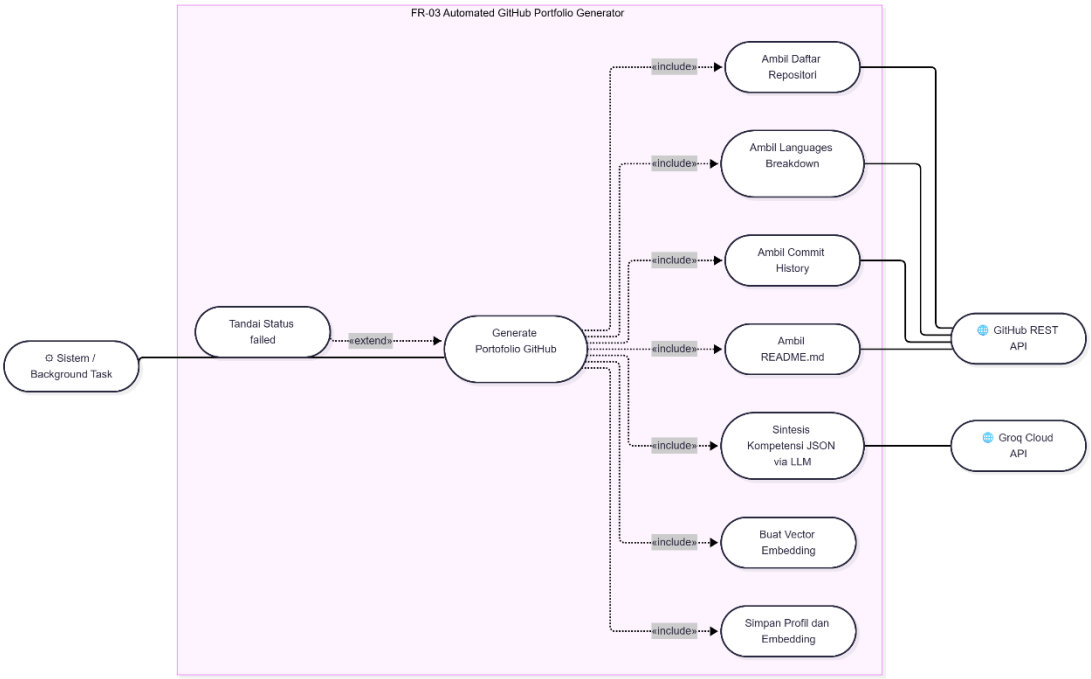
Aktor: Mahasiswa. Mahasiswa menautkan username GitHub pada profilnya dan memicu sinkronisasi portofolio.



Gambar 11. Use Case Diagram FR-02: Manajemen Profil & Penautan GitHub

c. Use Case Diagram FR-03: Automated GitHub Portfolio Generator

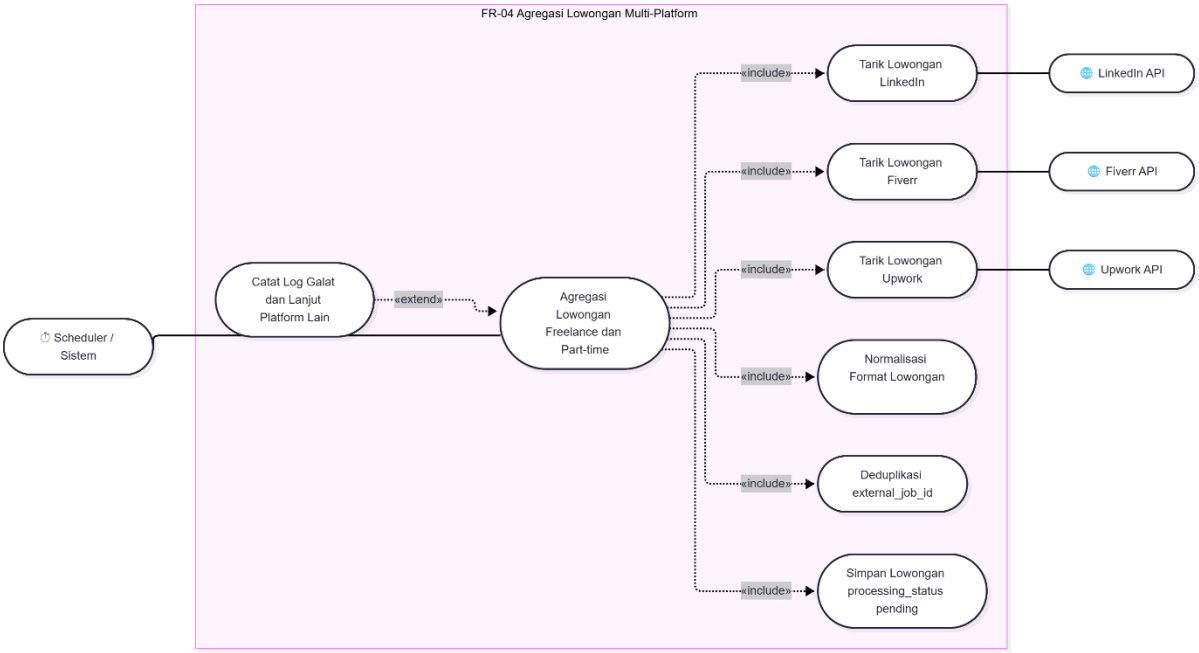
Aktor: Sistem. Sistem mengambil repositori, bahasa pemrograman, riwayat commit, dan README dari GitHub, menyusun portofolio kompetensi terstruktur, dan membuat embedding portofolio mahasiswa.



Gambar 12. Use Case Diagram FR-03: Automated GitHub Portfolio Generator

d. Use Case Diagram FR-04: Agregasi Lowongan Pekerjaan Multi-Platform

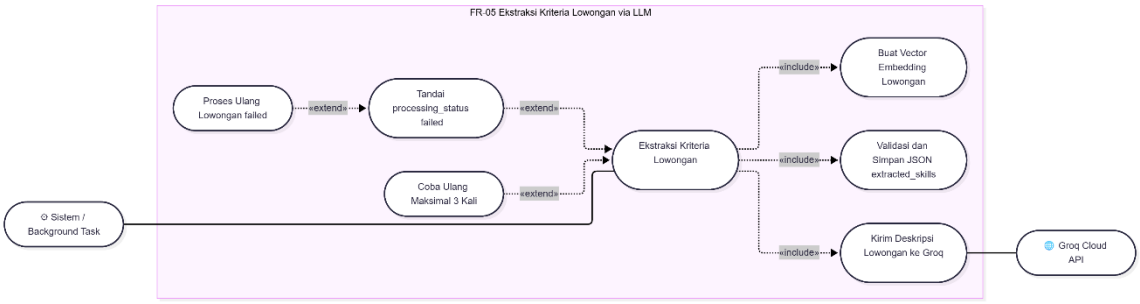
Aktor: Sistem. Sistem menarik dan mengagregasi lowongan freelance dan part-time dari LinkedIn, Fiverr, dan Upwork; sinkronisasi juga dapat dipicu admin.



Gambar 13. Use Case Diagram FR-04: Agregasi Lowongan Pekerjaan Multi-Platform

e. Use Case Diagram FR-05: Ekstraksi Kriteria Lowongan Otomatis (LLM)

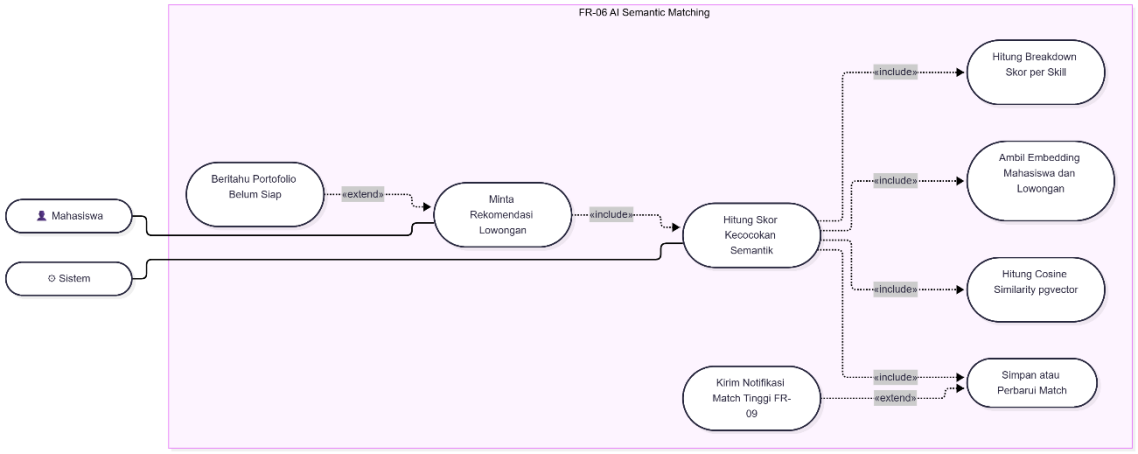
Aktor: Sistem. Sistem mengekstrak kriteria kompetensi, durasi, dan lingkup pekerjaan dari lowongan eksternal lewat Groq API dan menyimpannya.



Gambar 14. Use Case Diagram FR-05: Ekstraksi Kriteria Lowongan Otomatis (LLM)

f. Use Case Diagram FR-06: AI Semantic Matching

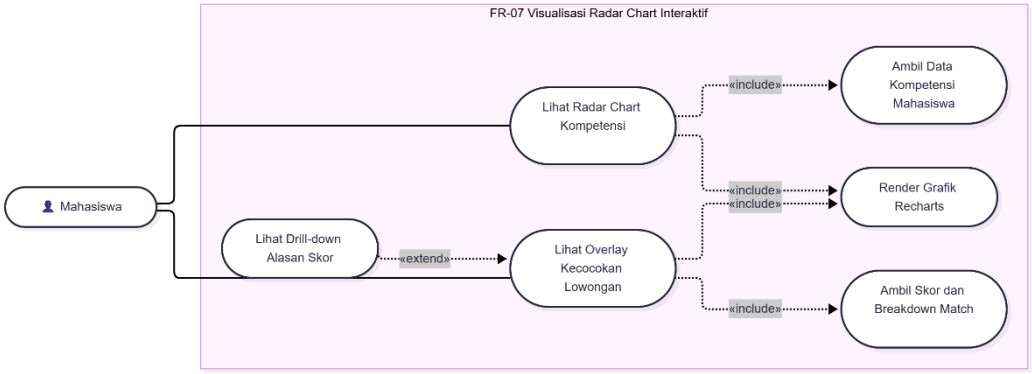
Aktor: Sistem (dipicu Mahasiswa). Sistem menghitung skor kecocokan antara portofolio GitHub mahasiswa dan lowongan serta menyajikan rekomendasi.



Gambar 15. Use Case Diagram FR-06: AI Semantic Matching

g. Use Case Diagram FR-07: Visualisasi Radar Chart Interaktif

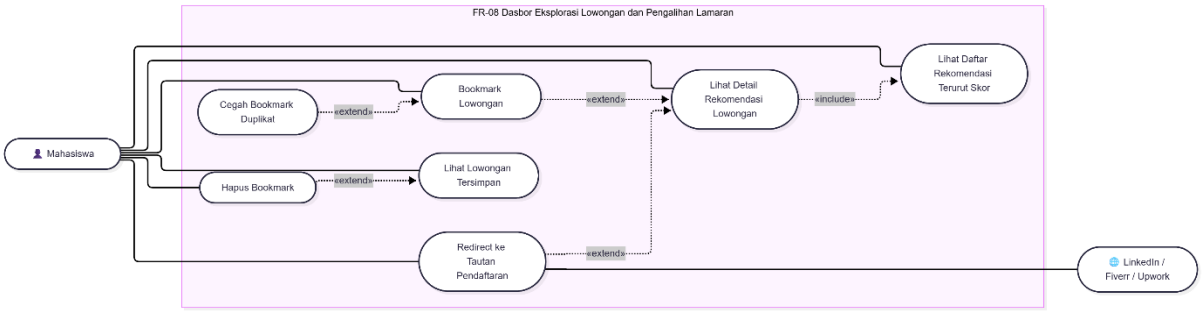
Aktor: Mahasiswa. Mahasiswa melihat radar chart kompetensi dengan overlay kualifikasi lowongan dan drill-down.



Gambar 16. Use Case Diagram FR-07: Visualisasi Radar Chart Interaktif

h. Use Case Diagram FR-08: Dasbor Eksplorasi Lowongan & Pengalihan Lamaran

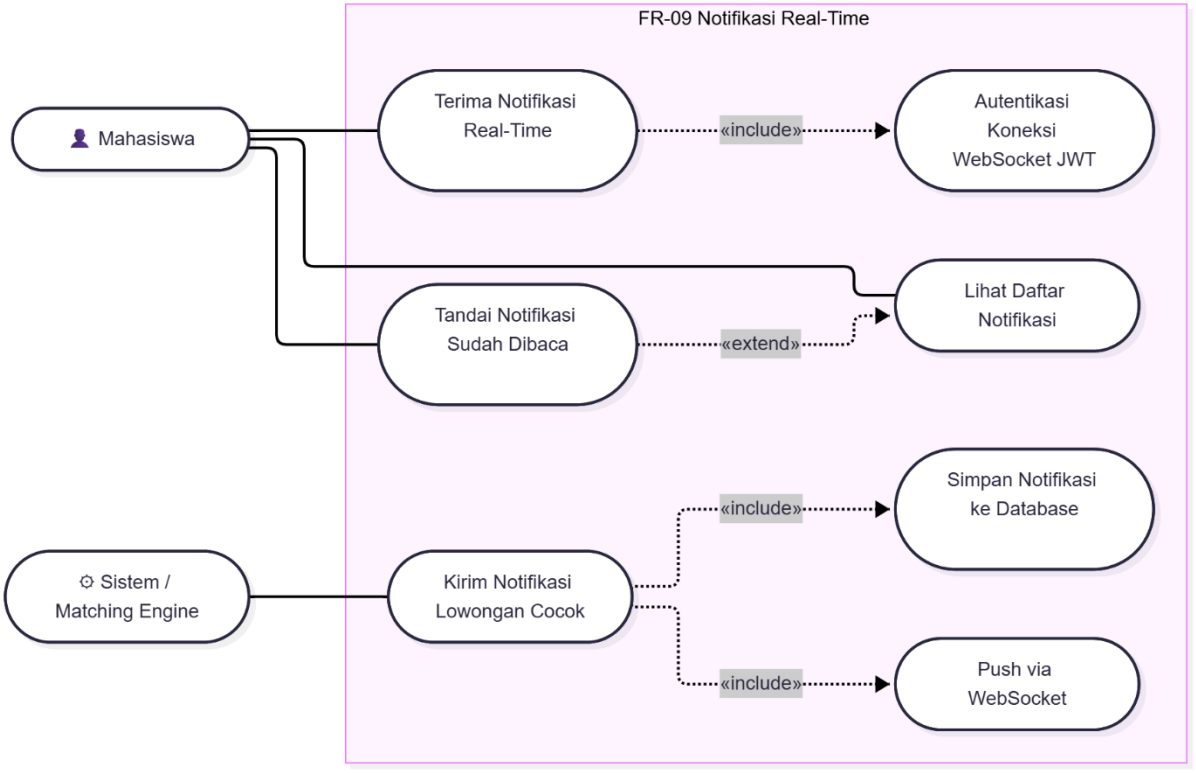
Aktor: Mahasiswa. Mahasiswa melihat rekomendasi lowongan terurut skor, menyimpan lowongan (bookmark), dan diarahkan ke tautan pendaftaran di platform asal.



Gambar 17. Use Case Diagram FR-08: Dasbor Eksplorasi Lowongan & Pengalihan Lamaran

i. Use Case Diagram FR-09: Notifikasi Real-time

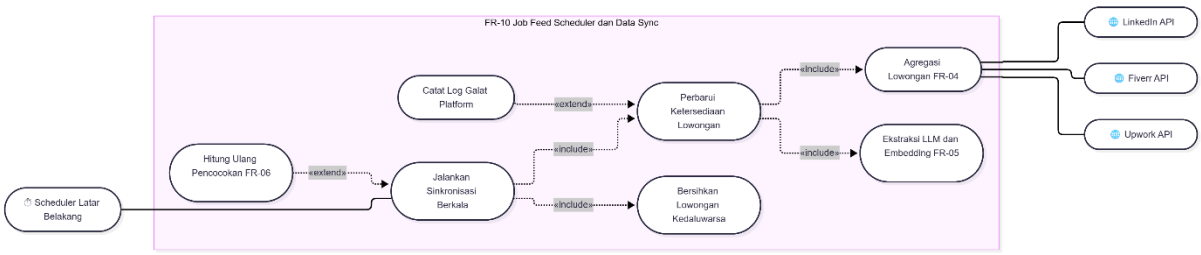
Aktor: Mahasiswa. Mahasiswa menerima notifikasi real-time saat ditemukan lowongan baru yang cocok.



Gambar 18. Use Case Diagram FR-09: Notifikasi Real-time

j. Use Case Diagram FR-10: Automated Job Feed Scheduler & Data Sync

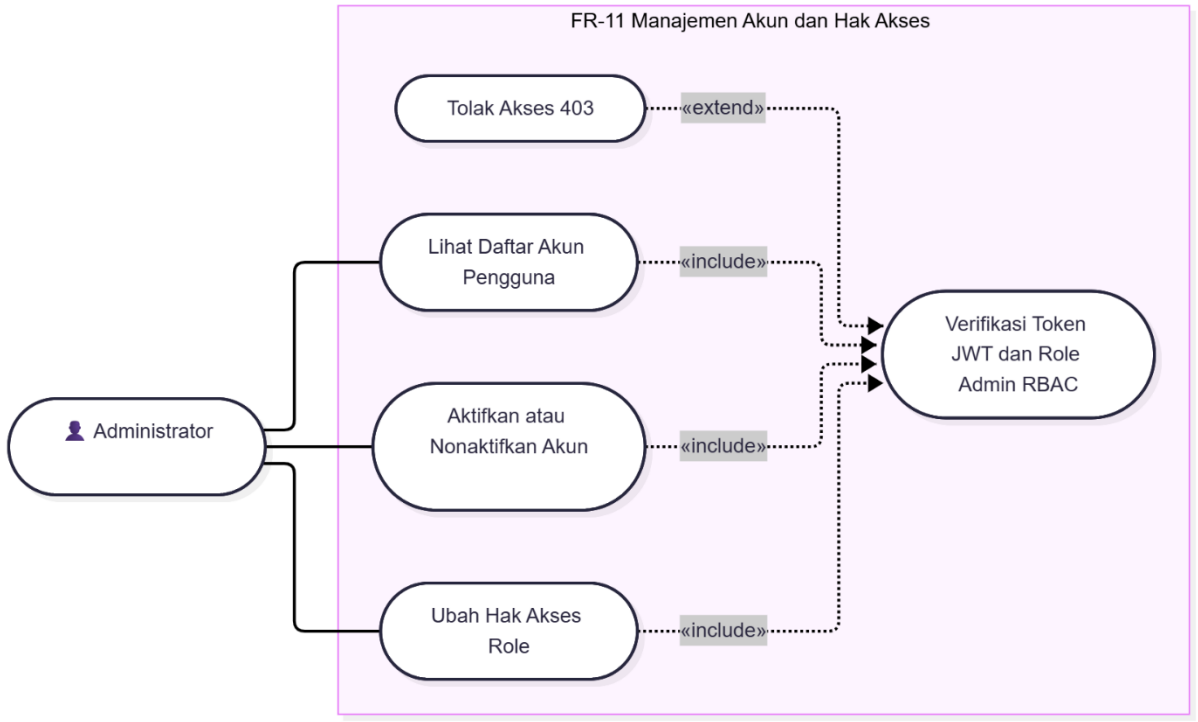
Aktor: Sistem (dipicu scheduler atau Administrator). Sistem menyinkronkan lowongan dari LinkedIn, Fiverr, dan Upwork secara berkala dan membersihkan lowongan yang sudah kedaluwarsa.



Gambar 19. Use Case Diagram FR-10: Automated Job Feed Scheduler & Data Sync

k. Use Case Diagram FR-11: Manajemen Akun & Hak Akses

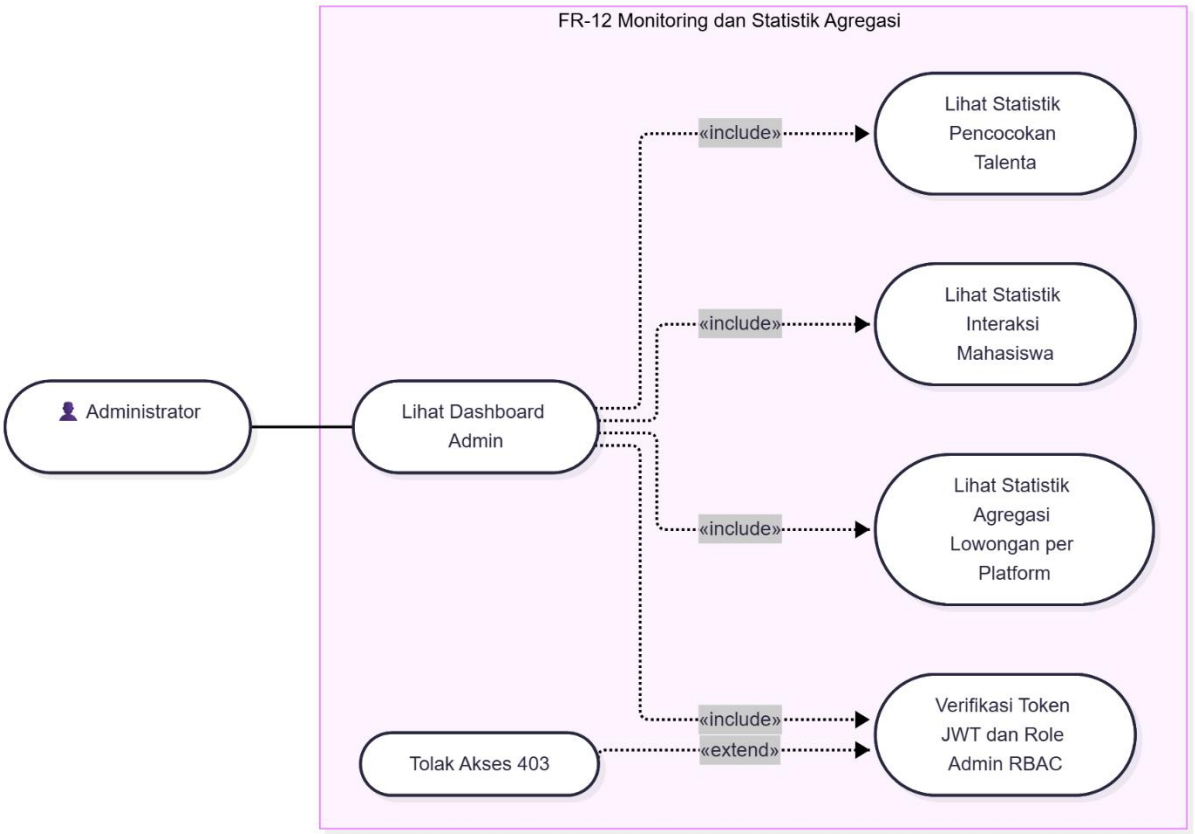
Aktor: Administrator. Administrator mengelola akun, status aktivasi, dan role pengguna.



Gambar 20. Use Case Diagram FR-11: Manajemen Akun & Hak Akses

l. Use Case Diagram FR-12: Monitoring & Statistik Agregasi

Aktor: Administrator. Administrator memantau statistik agregasi lowongan per platform, statistik pencocokan, dan interaksi mahasiswa.



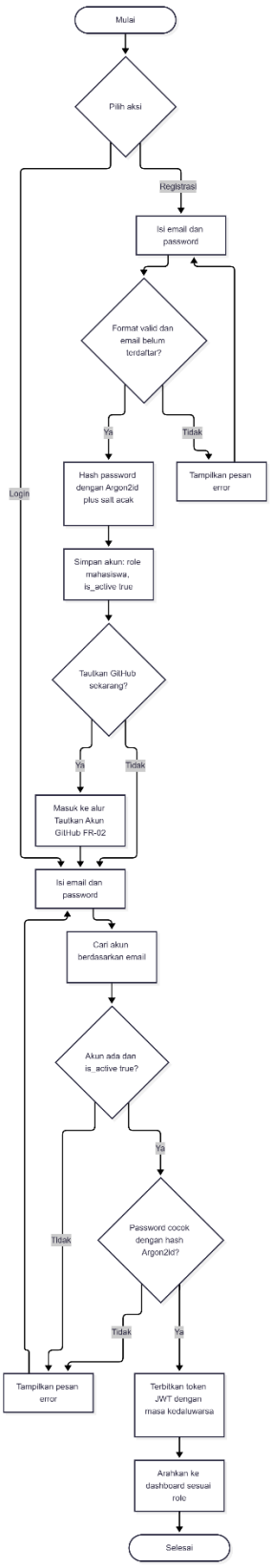
Gambar 21. Use Case Diagram FR-12: Monitoring & Statistik Agregasi

13.2 Activity Diagram

Activity diagram menunjukkan urutan aktivitas dan titik keputusan pada tiap fitur.

a. Activity Diagram FR-01: Registrasi & Login

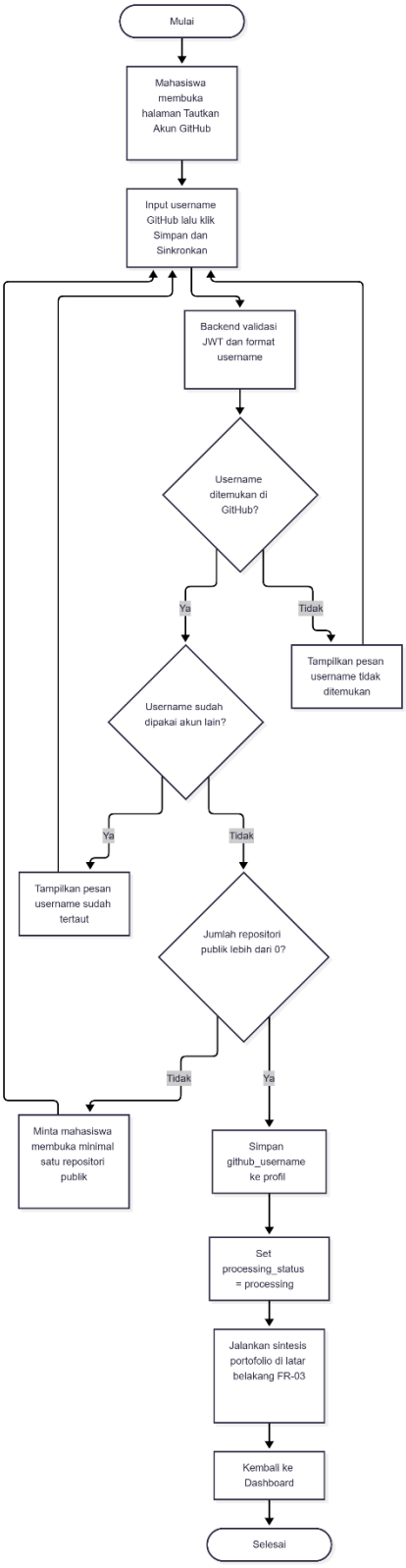
Diagram ini menggambarkan alur dari pengisian data registrasi atau login hingga token diterbitkan atau permintaan ditolak.



Gambar 22. Activity Diagram FR-01: Registrasi & Login

b. Activity Diagram FR-02: Manajemen Profil & Penautan GitHub

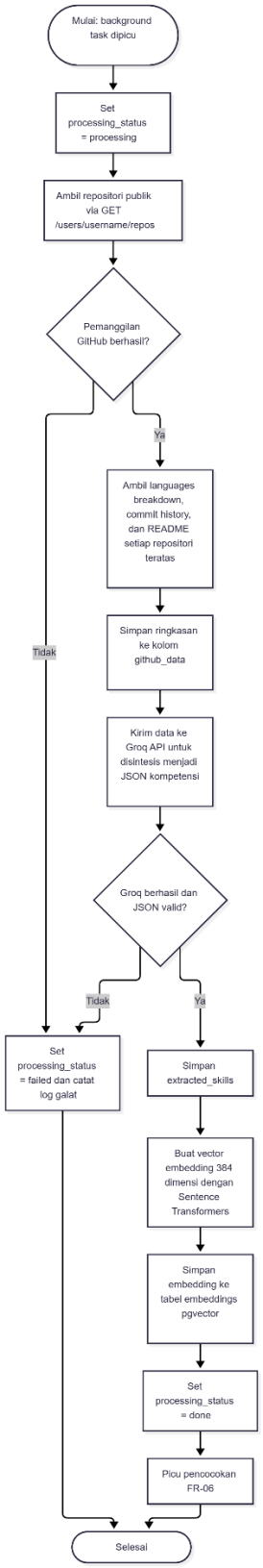
Diagram ini menggambarkan alur penautan username GitHub dan pemicuan sinkronisasi, termasuk validasi akun dan pemeriksaan jumlah repositori publik.



Gambar 23. Activity Diagram FR-02: Manajemen Profil & Penautan GitHub

c. Activity Diagram FR-03: Automated GitHub Portfolio Generator

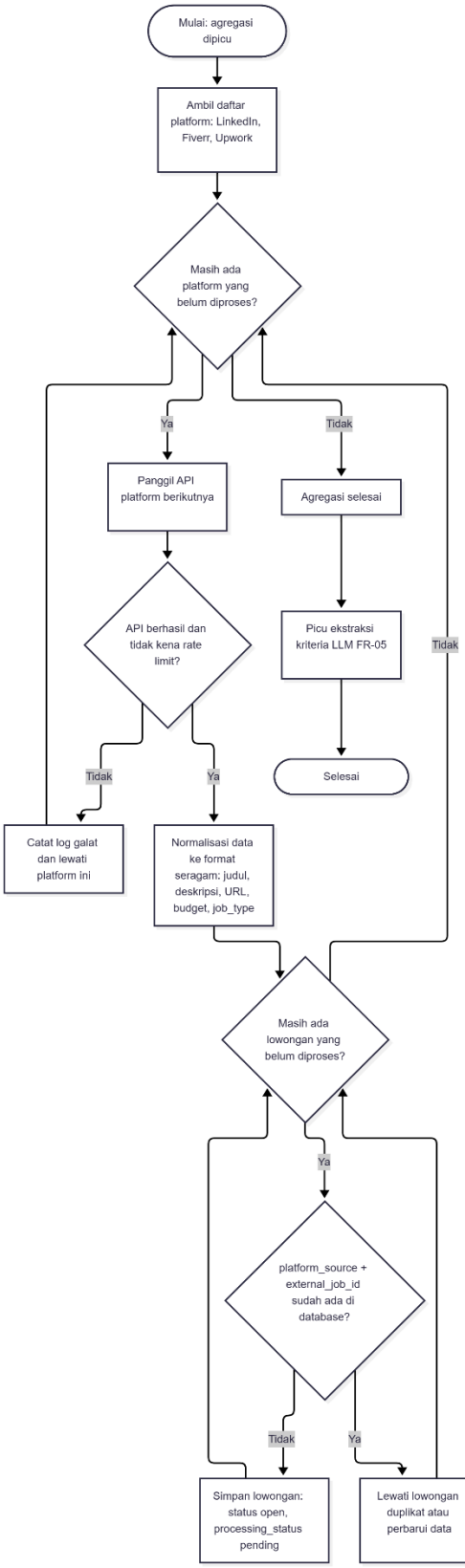
Diagram ini menggambarkan alur dari tugas latar belakang dimulai, pengambilan data GitHub, sintesis portofolio oleh Groq, hingga embedding tersimpan dan status menjadi `done` atau `failed`.



Gambar 24. Activity Diagram FR-03: Automated GitHub Portfolio Generator

d. Activity Diagram FR-04: Agregasi Lowongan Pekerjaan Multi-Platform

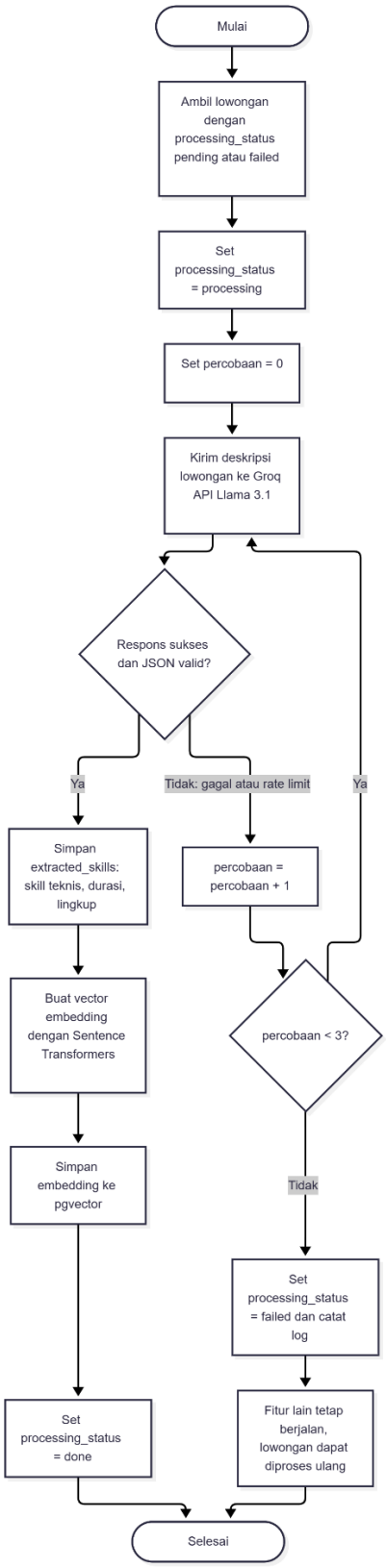
Diagram ini menggambarkan alur penarikan lowongan dari API platform, deduplikasi lewat `external_job_id`, hingga lowongan tersimpan dan ekstraksi diantrekan.



Gambar 25. Activity Diagram FR-04: Agregasi Lowongan Pekerjaan Multi-Platform

e. Activity Diagram FR-05: Ekstraksi Kriteria Lowongan Otomatis (LLM)

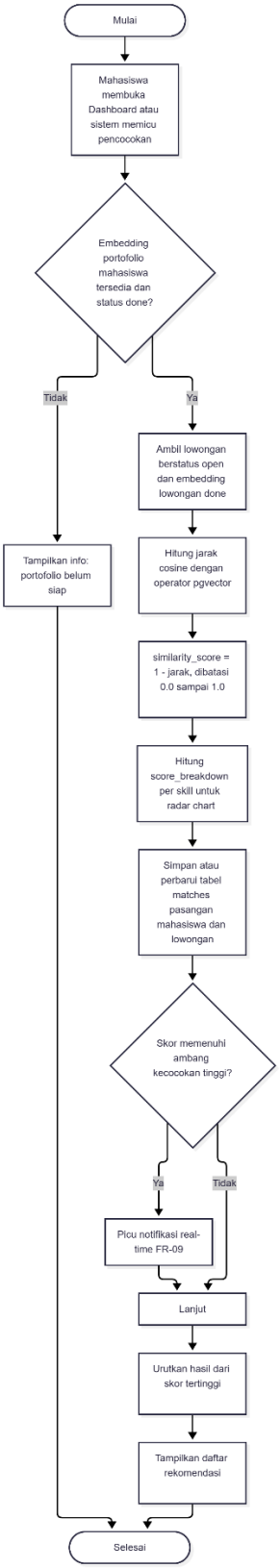
Diagram ini menggambarkan alur ekstraksi dengan percobaan ulang dan penandaan status `failed` bila gagal.



Gambar 26. Activity Diagram FR-05: Ekstraksi Kriteria Lowongan Otomatis (LLM)

f. Activity Diagram FR-06: AI Semantic Matching

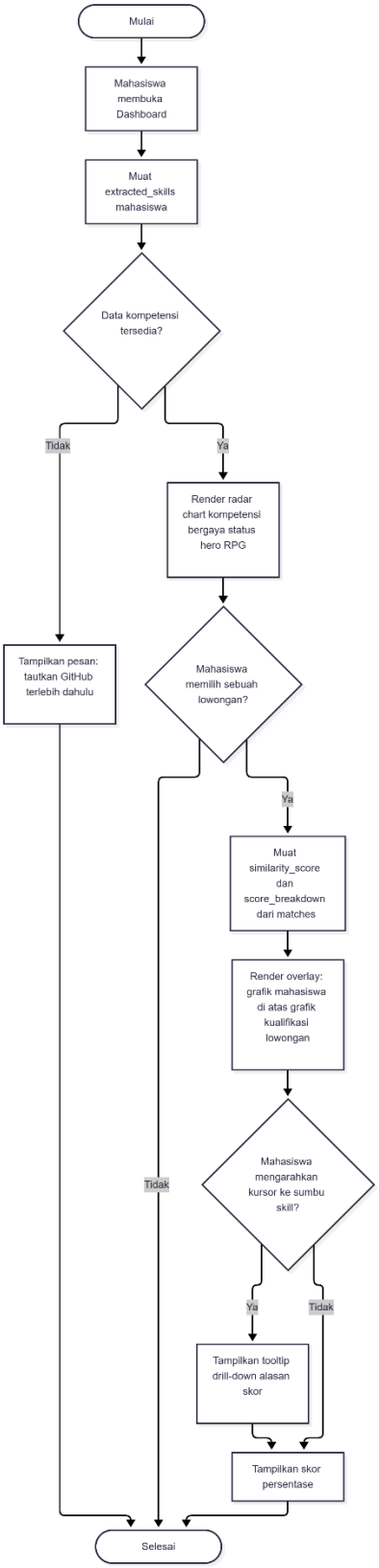
Diagram ini menggambarkan alur pengecekan embedding, perhitungan atau pemakaian ulang match, hingga daftar rekomendasi tampil.



Gambar 27. Activity Diagram FR-06: AI Semantic Matching

g. Activity Diagram FR-07: Visualisasi Radar Chart Interaktif

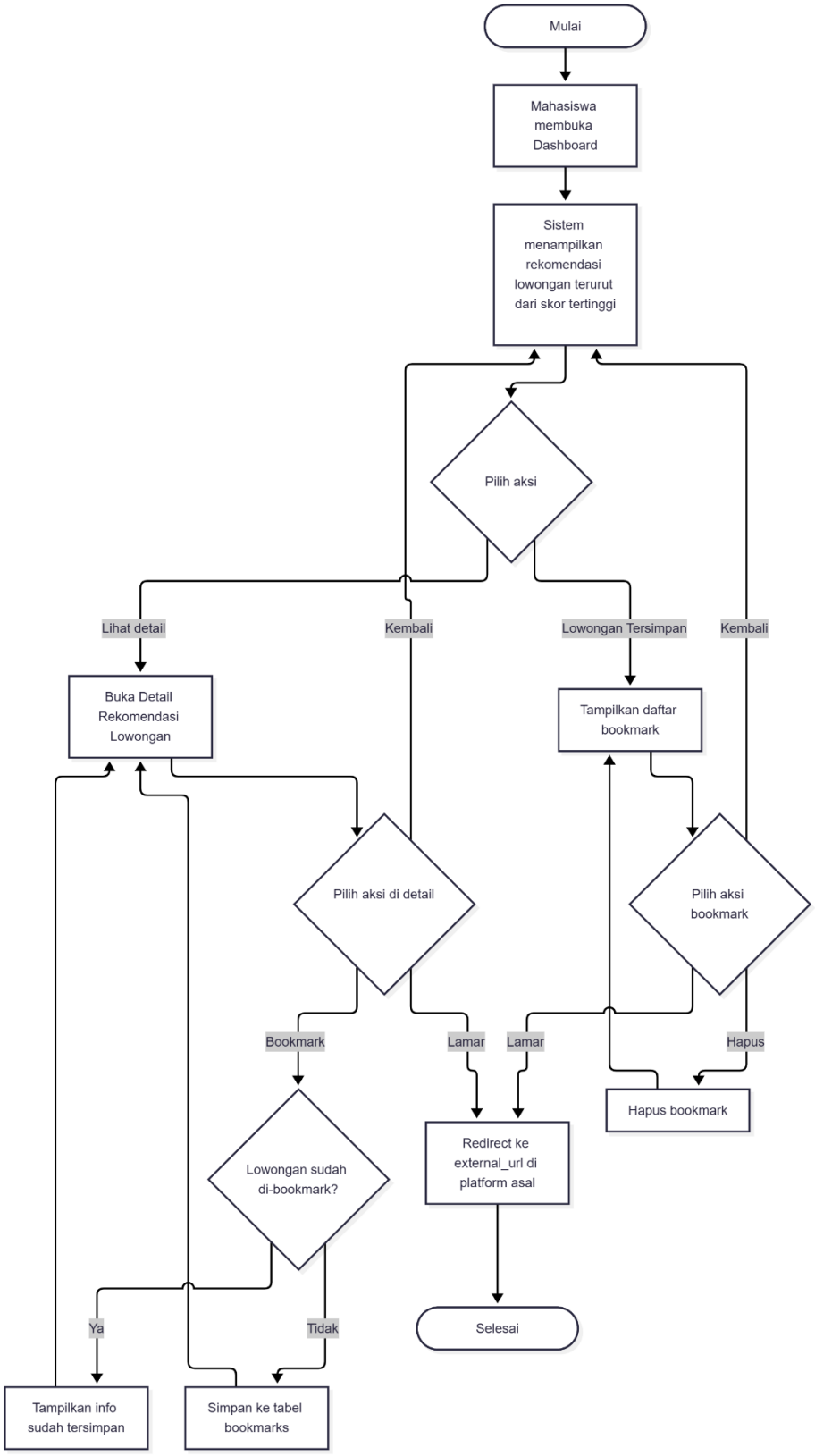
Diagram ini menggambarkan alur dari membuka dashboard, memuat data skor, hingga menampilkan overlay dan alasan skor.



Gambar 28. Activity Diagram FR-07: Visualisasi Radar Chart Interaktif

h. Activity Diagram FR-08: Dasbor Eksplorasi Lowongan & Pengalihan Lamaran

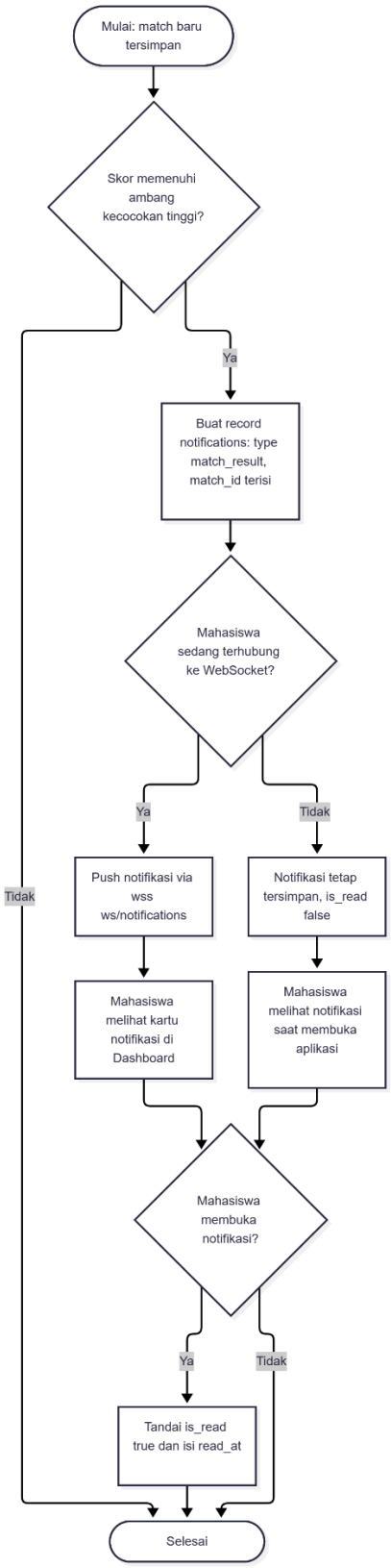
Diagram ini menggambarkan alur mahasiswa membuka daftar rekomendasi, menyimpan bookmark, hingga diarahkan ke tautan lamaran di platform asal.



Gambar 29. Activity Diagram FR-08: Dasbor Eksplorasi Lowongan & Pengalihan Lamaran

i. Activity Diagram FR-09: Notifikasi Real-time

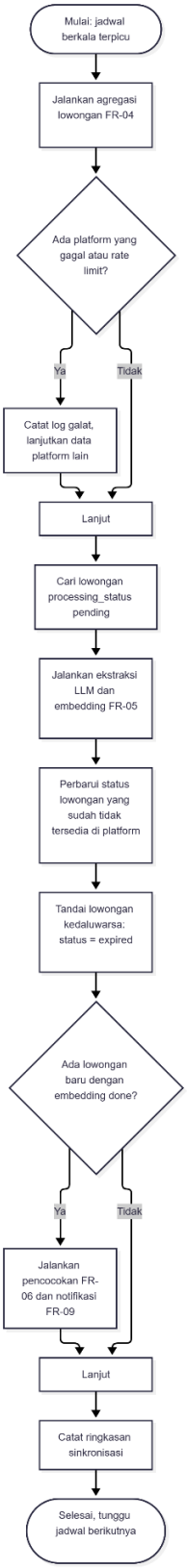
Diagram ini menggambarkan alur dari pembentukan notifikasi hingga diterima pengguna online atau tersimpan untuk pengguna offline.



Gambar 30. Activity Diagram FR-09: Notifikasi Real-time

j. Activity Diagram FR-10: Automated Job Feed Scheduler & Data Sync

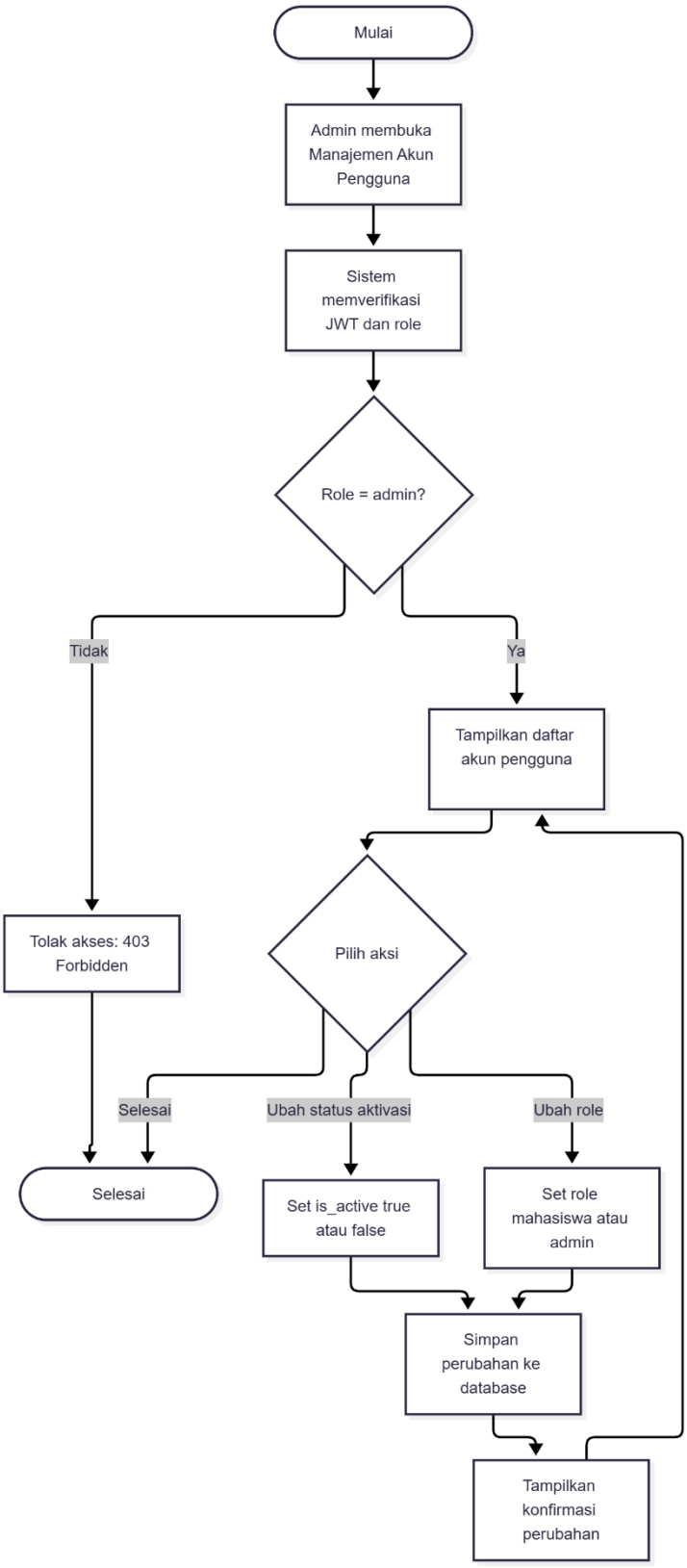
Diagram ini menggambarkan alur dari pemicu scheduler, penarikan lowongan dari tiap platform, pembaruan data, hingga lowongan kedaluwarsa dibersihkan.



Gambar 31. Activity Diagram FR-10: Automated Job Feed Scheduler & Data Sync

k. Activity Diagram FR-11: Manajemen Akun & Hak Akses

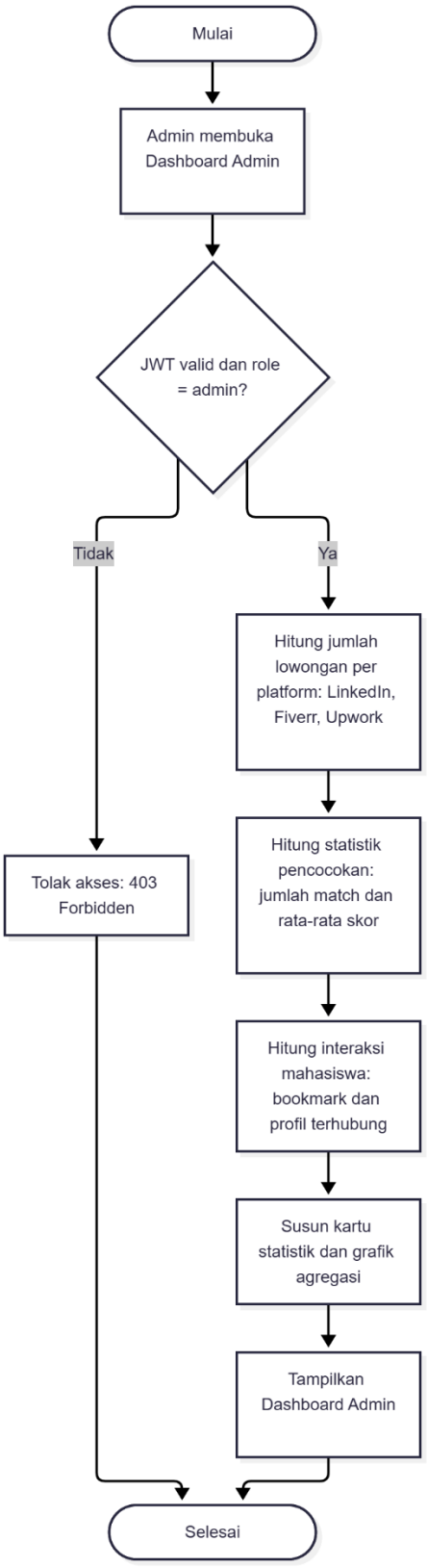
Diagram ini menggambarkan alur dari membuka manajemen akun hingga perubahan status atau role tersimpan.



Gambar 32. Activity Diagram FR-11: Manajemen Akun & Hak Akses

l. Activity Diagram FR-12: Monitoring & Statistik Agregasi

Diagram ini menggambarkan alur dari membuka dasbor statistik hingga kartu dan grafik ditampilkan.



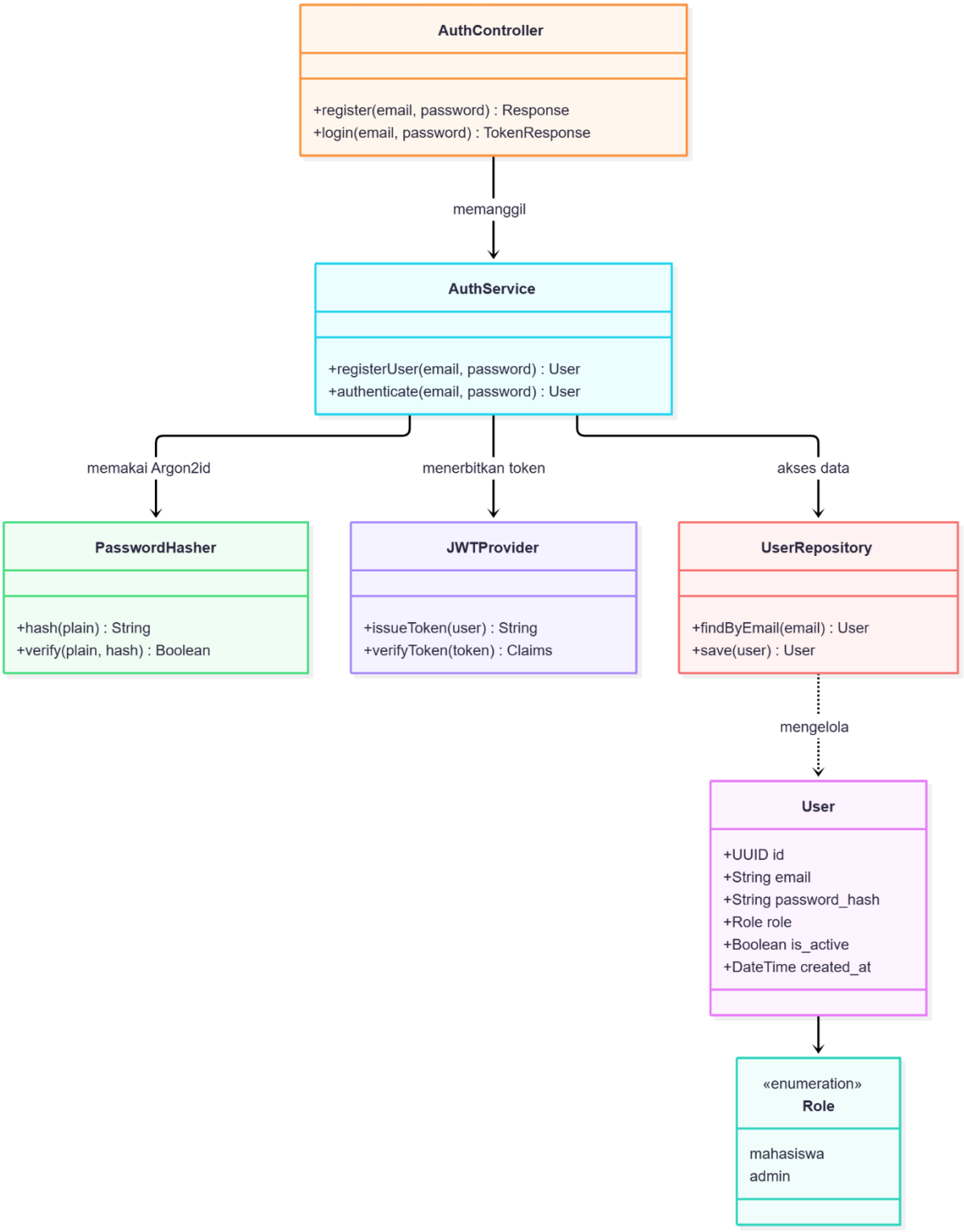
Gambar 33. Activity Diagram FR-12: Monitoring & Statistik Agregasi

13.3 Class Diagram

Class diagram menunjukkan kelas, atribut, operasi, dan hubungan antarkelas yang mendukung tiap fitur.

a. Class Diagram FR-01: Registrasi & Login

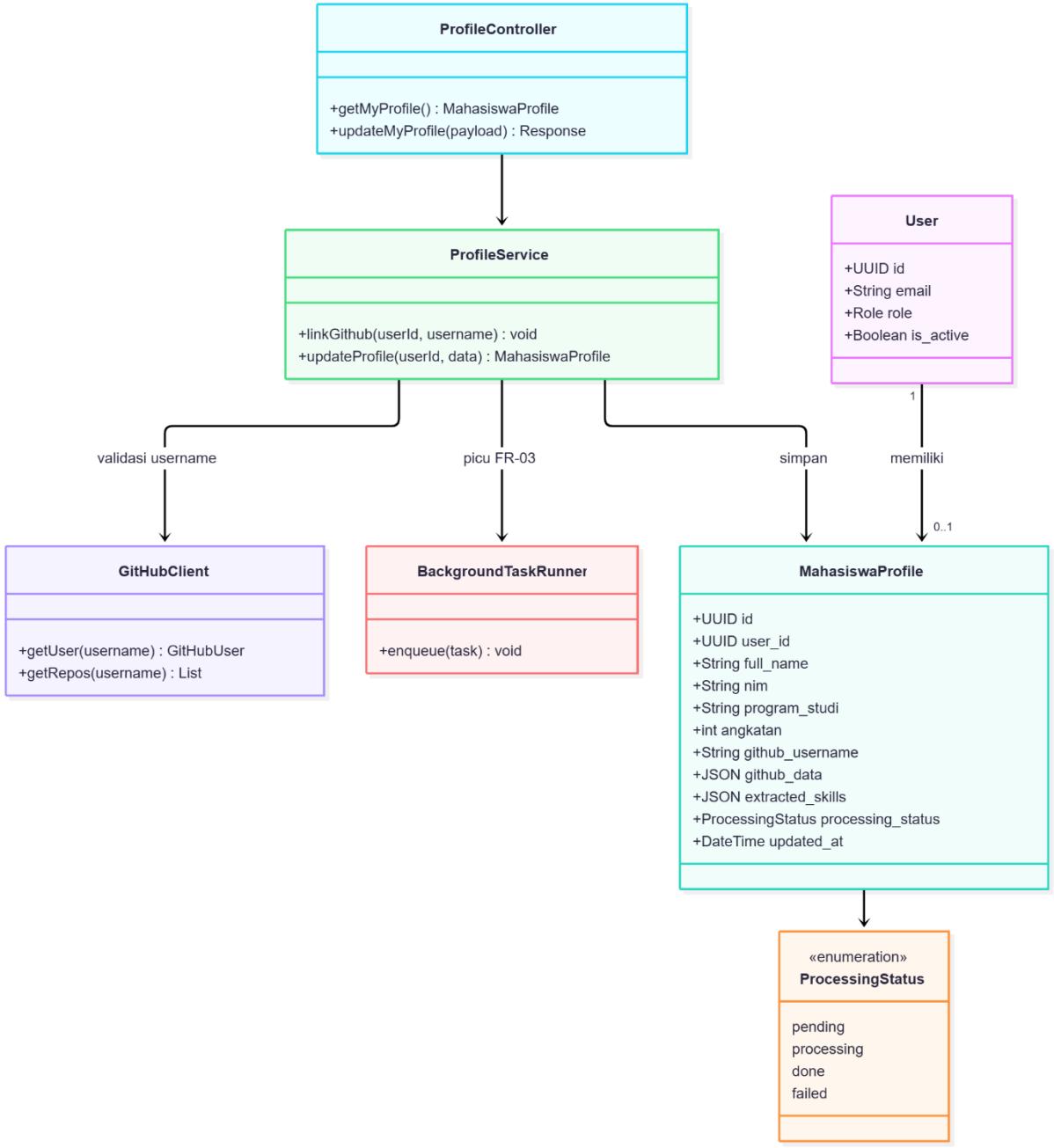
Diagram ini memuat kelas AuthService, PasswordHasher, TokenService, RBACGuard, User, MahasiswaProfile, serta enumerasi UserRole.



Gambar 34. Class Diagram FR-01: Registrasi & Login

b. Class Diagram FR-02: Manajemen Profil & Penautan GitHub

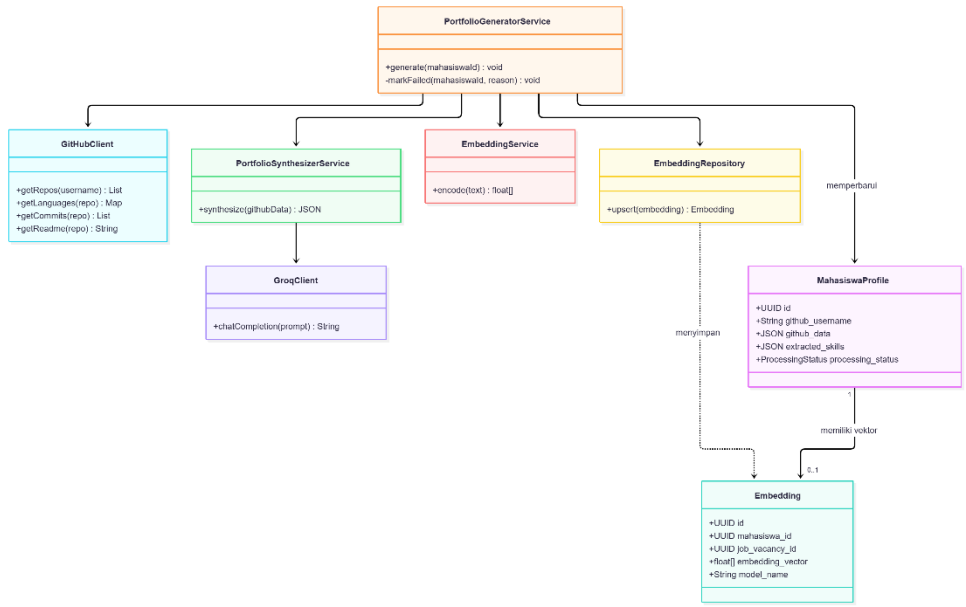
Diagram ini memuat kelas yang mengelola profil mahasiswa, penautan username GitHub, dan status pemrosesan portofolio.



Gambar 35. Class Diagram FR-02: Manajemen Profil & Penautan GitHub

c. Class Diagram FR-03: Automated GitHub Portfolio Generator

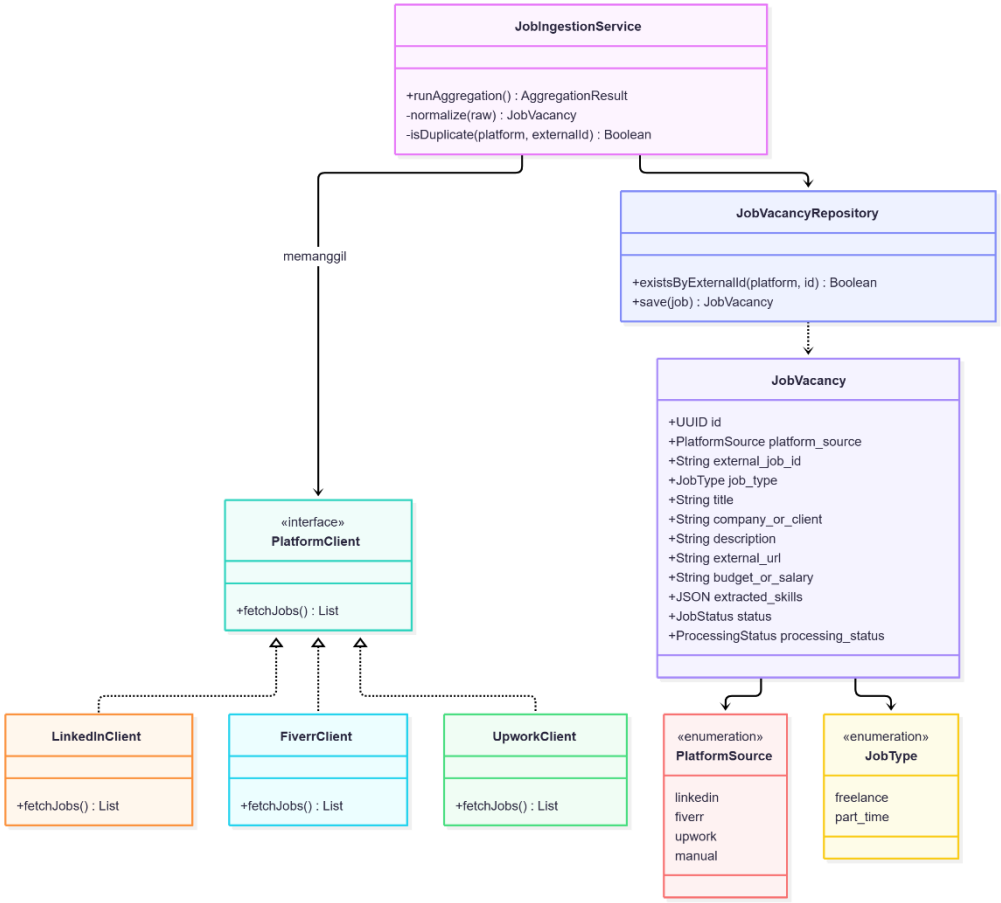
Diagram ini memuat kelas pemroses portofolio: GitHubClient, PortfolioSynthesizerService, klien Groq, layanan embedding, dan hubungannya dengan data profil serta embedding.



Gambar 36. Class Diagram FR-03: Automated GitHub Portfolio Generator

d. Class Diagram FR-04: Agregasi Lowongan Pekerjaan Multi-Platform

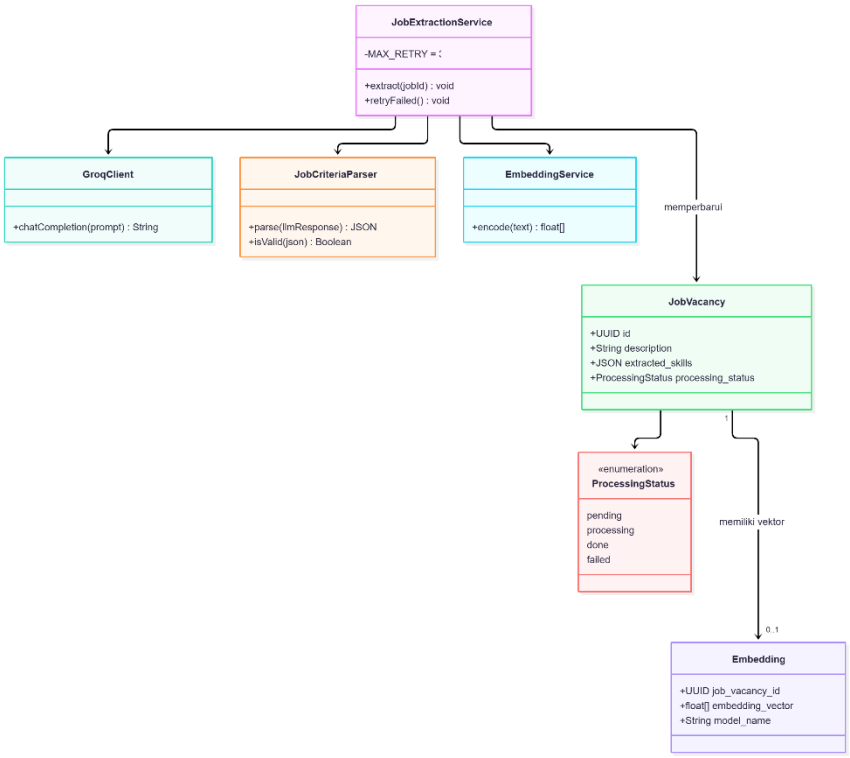
Diagram ini memuat kelas JobVacancy, JobIngestionService, klien API platform (LinkedIn, Fiverr, Upwork), dan layanan pengelola lowongan.



Gambar 37. Class Diagram FR-04: Agregasi Lowongan Pekerjaan Multi-Platform

e. Class Diagram FR-05: Ekstraksi Kriteria Lowongan Otomatis (LLM)

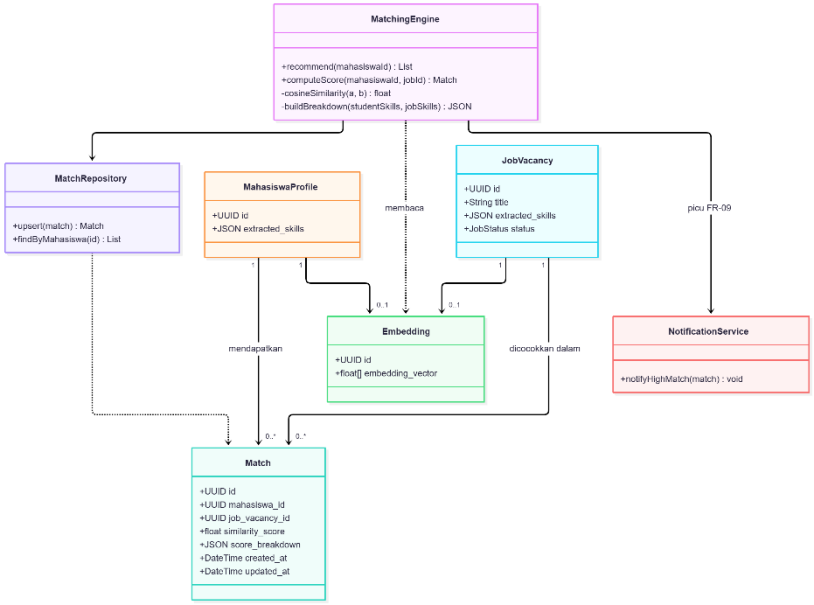
Diagram ini memuat kelas LLMExtractionService, klien Groq, layanan embedding, dan data lowongan yang diperbarui.



Gambar 38. Class Diagram FR-05: Ekstraksi Kriteria Lowongan Otomatis (LLM)

f. Class Diagram FR-06: AI Semantic Matching

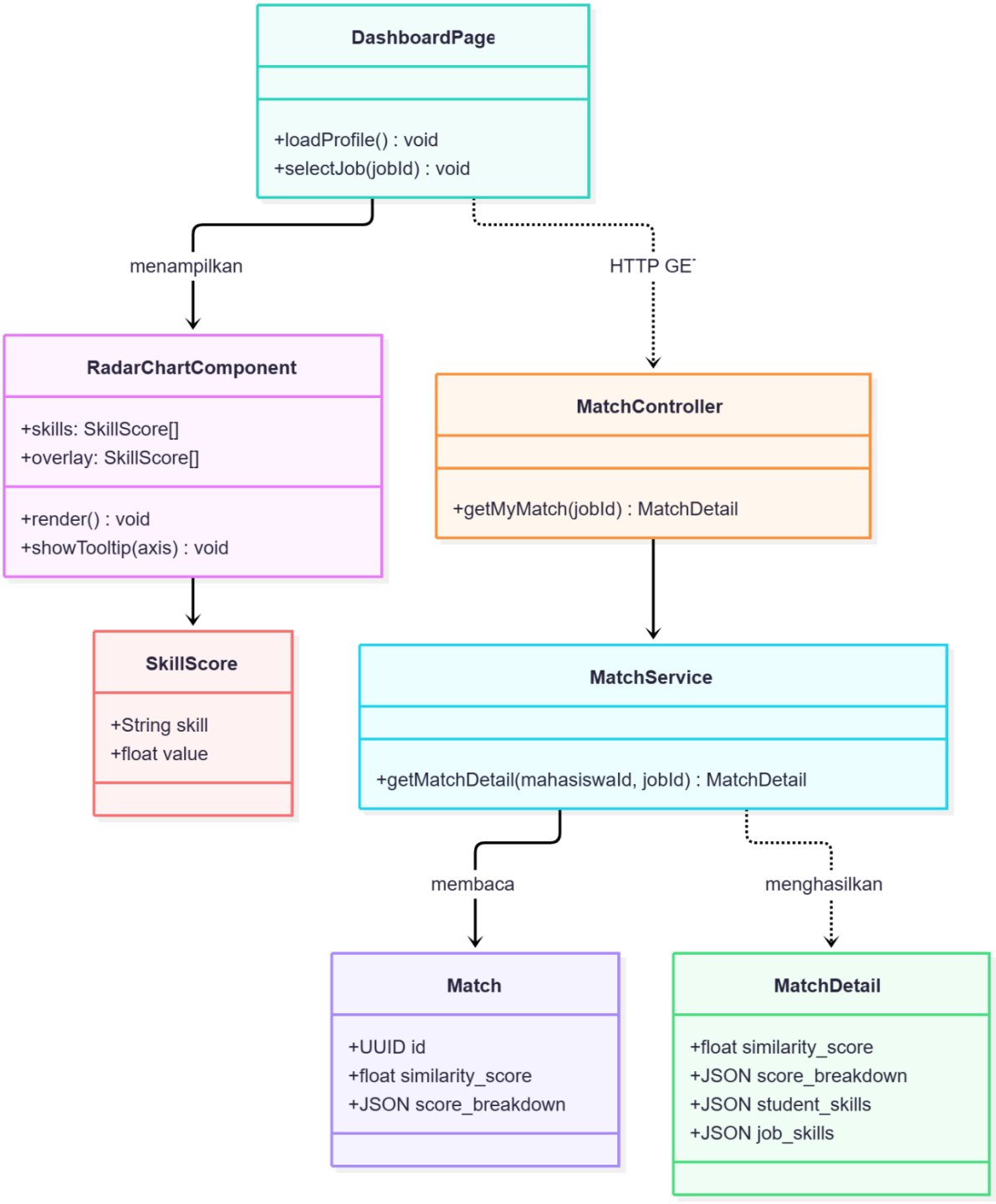
Diagram ini memuat kelas MatchingEngine, data Match, Embedding, dan layanan notifikasi yang dipanggil saat match baru tersimpan.



Gambar 39. Class Diagram FR-06: AI Semantic Matching

g. Class Diagram FR-07: Visualisasi Radar Chart Interaktif

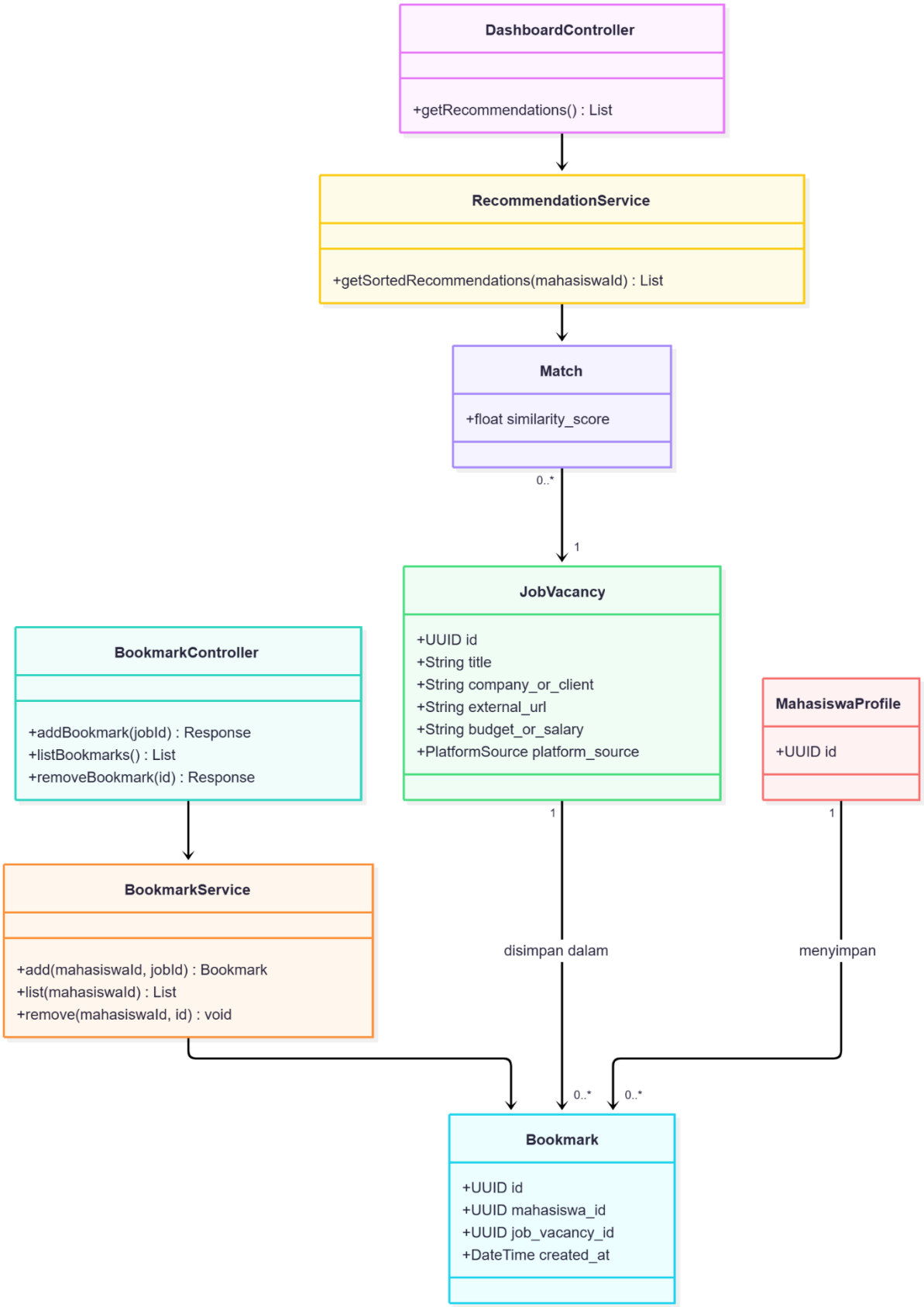
Diagram ini memuat kelas yang memodelkan hasil pencocokan dan komponen penyaji data radar chart.



Gambar 40. Class Diagram FR-07: Visualisasi Radar Chart Interaktif

h. Class Diagram FR-08: Dasbor Eksplorasi Lowongan & Pengalihan Lamaran

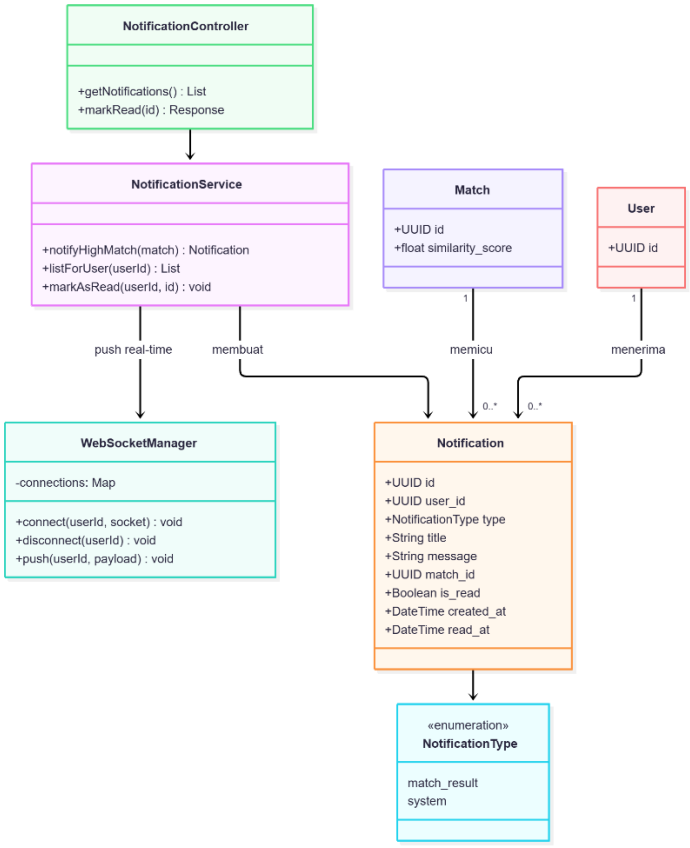
Diagram ini memuat kelas JobBookmark, Match, JobVacancy, profil mahasiswa, dan layanan pengelola bookmark.



Gambar 41. Class Diagram FR-08: Dasbor Eksplorasi Lowongan & Pengalihan Lamaran

i. Class Diagram FR-09: Notifikasi Real-time

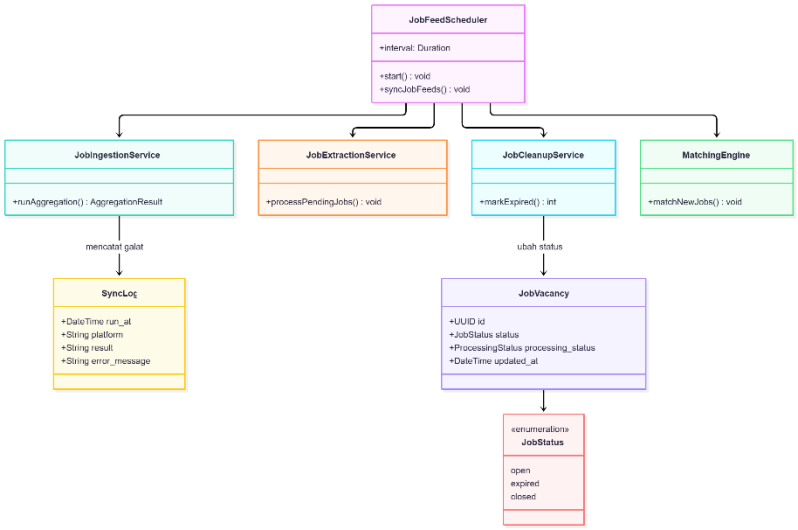
Diagram ini memuat kelas Notification, NotificationService, dan pengelola koneksi WebSocket.



Gambar 42. Class Diagram FR-09: Notifikasi Real-time

j. Class Diagram FR-10: Automated Job Feed Scheduler & Data Sync

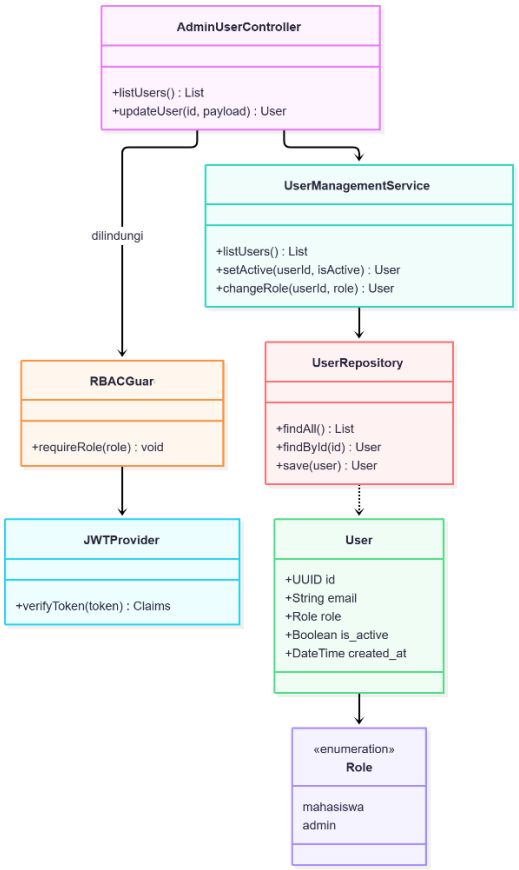
Diagram ini memuat kelas JobIngestionService, scheduler sinkronisasi, klien API platform, JobVacancy, serta pembersih lowongan kedaluwarsa.



Gambar 43. Class Diagram FR-10: Automated Job Feed Scheduler & Data Sync

k. Class Diagram FR-11: Manajemen Akun & Hak Akses

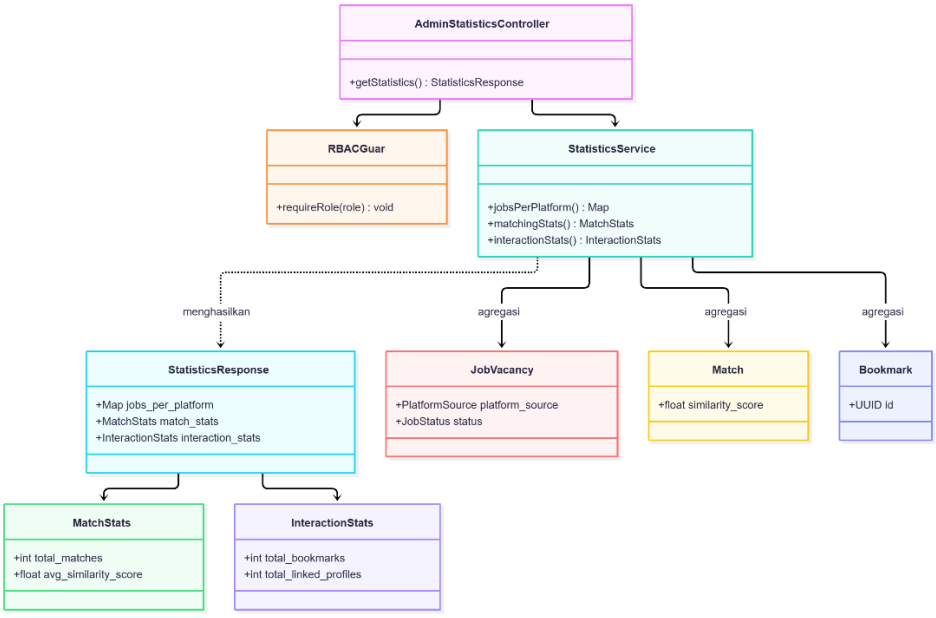
Diagram ini memuat kelas AdminUserService, RBACGuard, User, dan enumerasi UserRole.



Gambar 44. Class Diagram FR-11: Manajemen Akun & Hak Akses

l. Class Diagram FR-12: Monitoring & Statistik Agregasi

Diagram ini memuat kelas StatisticsService, laporan statistik, dan data yang diagregasi.



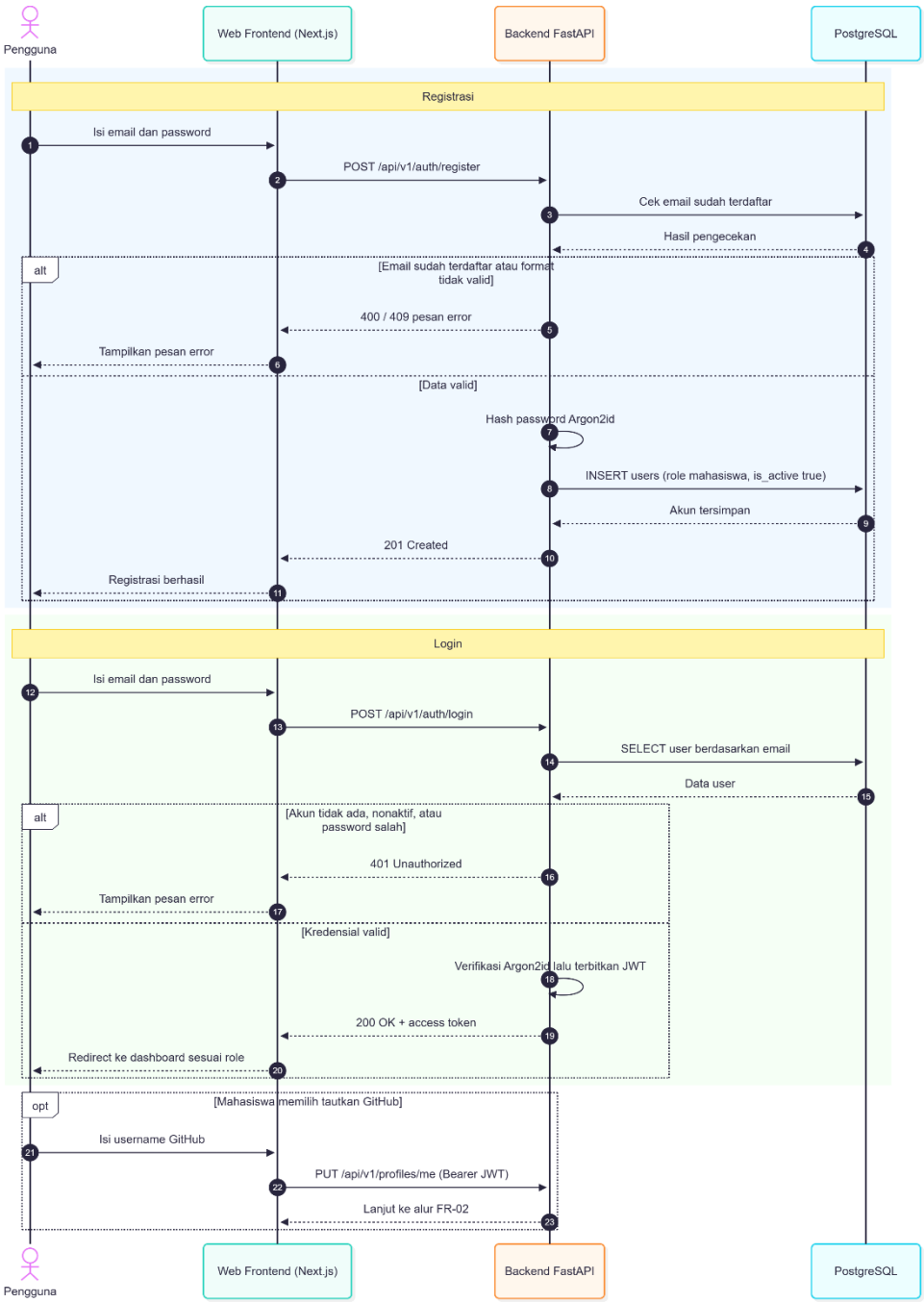
Gambar 45. Class Diagram FR-12: Monitoring & Statistik Agregasi

13.4 Sequence Diagram

Sequence diagram menunjukkan urutan pesan antara aktor, antarmuka, router, service, modul AI, dan basis data.

a. Sequence Diagram FR-01: Registrasi & Login

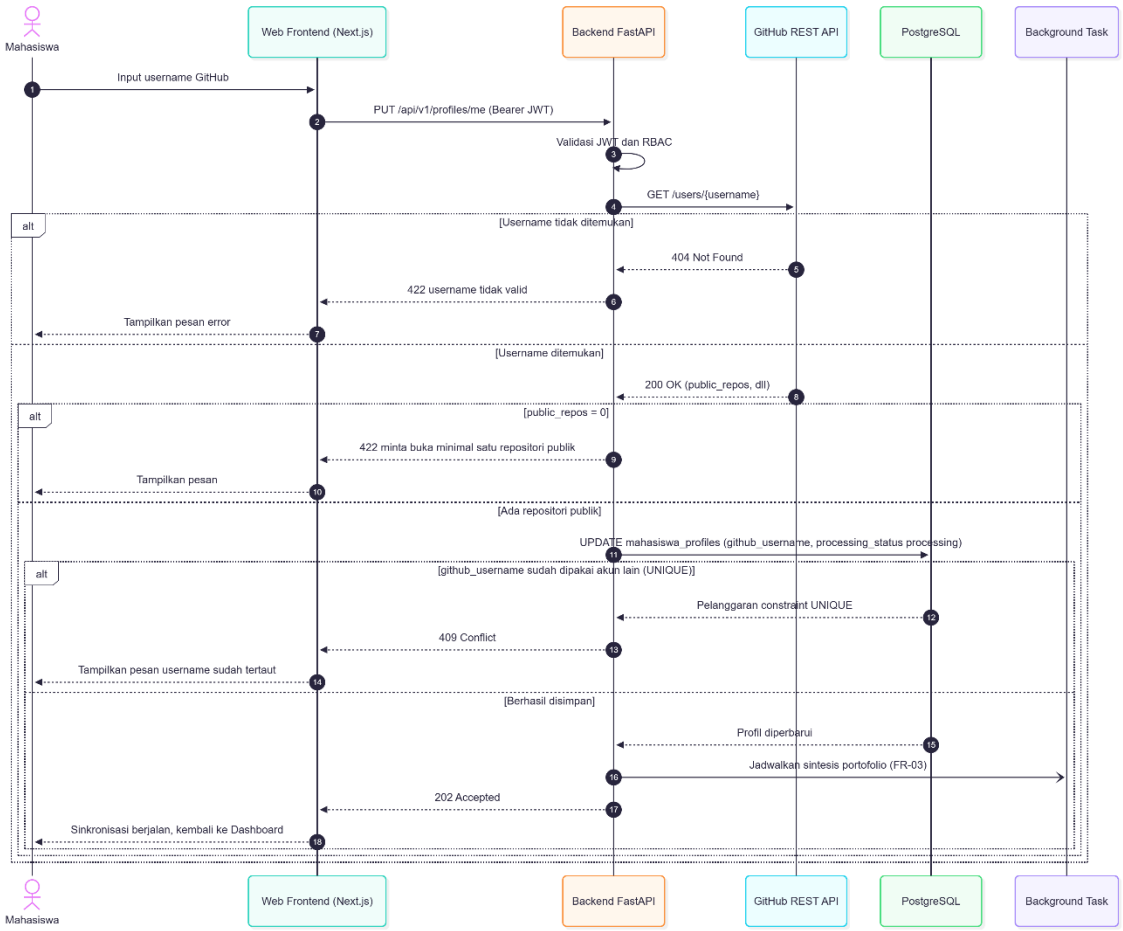
Diagram ini menggambarkan dua skenario, yaitu registrasi dan login. Fragmen alt memisahkan email yang sudah terdaftar (409), user tidak ditemukan atau password salah (401), akun nonaktif (403), dan login berhasil. Fragmen opt menunjukkan pembuatan profil awal sesuai role.



Gambar 46. Sequence Diagram FR-01: Registrasi & Login

b. Sequence Diagram FR-02: Manajemen Profil & Penautan GitHub

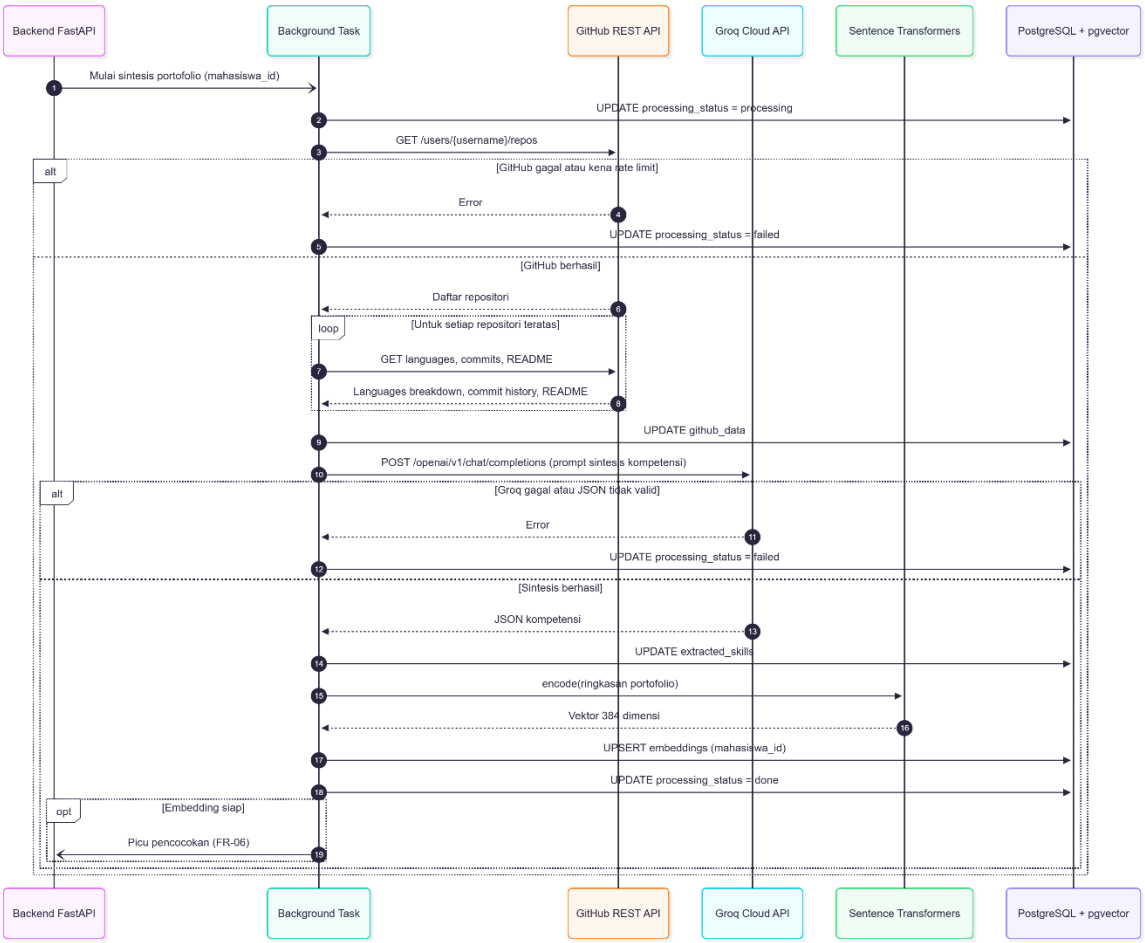
Diagram ini menggambarkan penautan username GitHub dengan fragmen alt untuk token tidak valid (401), username tidak ditemukan (404), dan akun tanpa repositori publik (respons peringatan). Fragmen opt menangani pengantrean sinkronisasi, dan fragmen loop menunjukkan polling `processing_status` oleh frontend.



Gambar 47. Sequence Diagram FR-02: Manajemen Profil & Penautan GitHub

c. Sequence Diagram FR-03: Automated GitHub Portfolio Generator

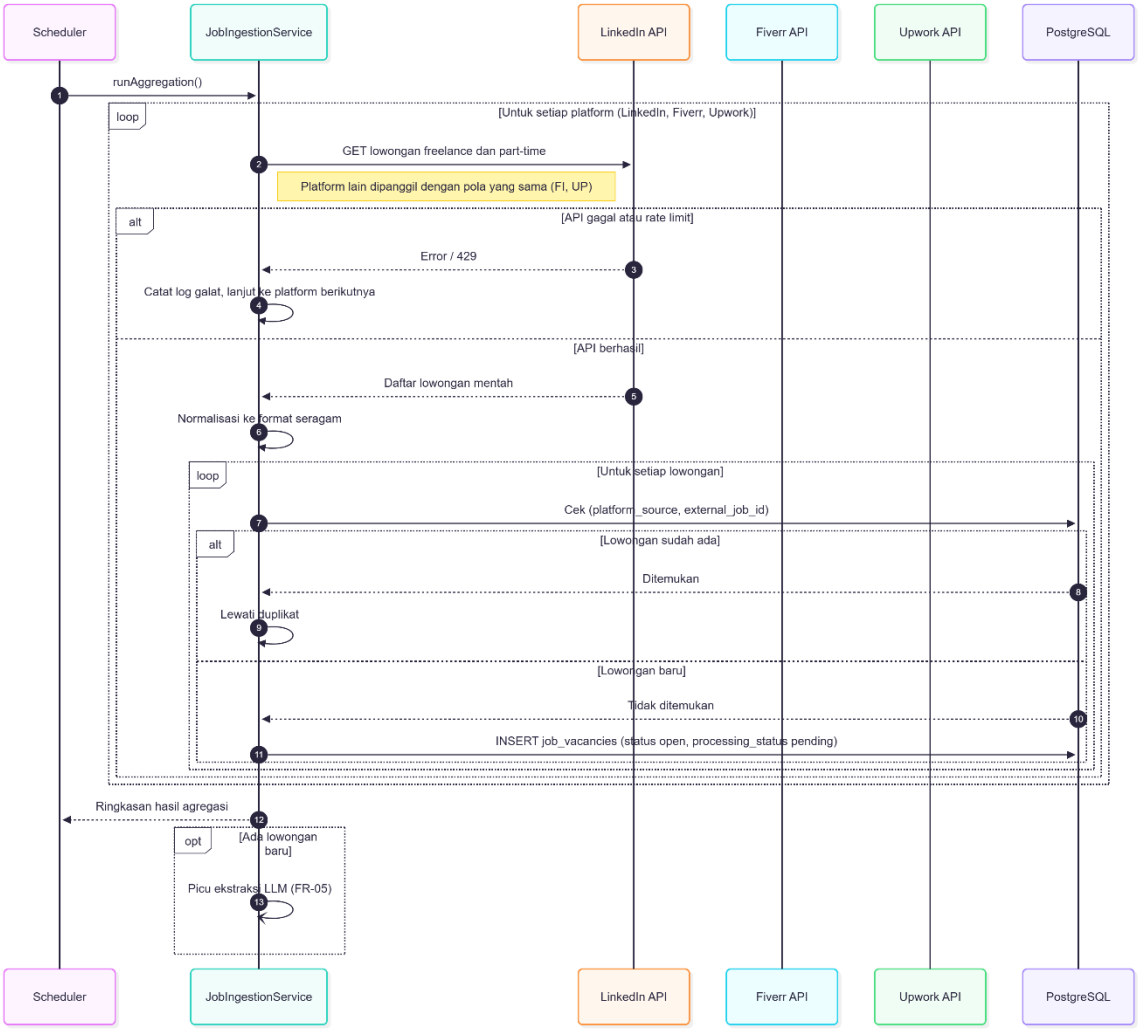
Diagram ini menggambarkan pemrosesan oleh BackgroundTask. Opt memuat pengambilan data GitHub (dengan cadangan `github_data` saat rate limit), alt menangani keberhasilan sintesis, loop menunjukkan percobaan ulang maksimal tiga kali ke Groq, dan opt terakhir memicu pencocokan setelah status `done`.



Gambar 48. Sequence Diagram FR-03: Automated GitHub Portfolio Generator

d. Sequence Diagram FR-04: Agregasi Lowongan Pekerjaan Multi-Platform

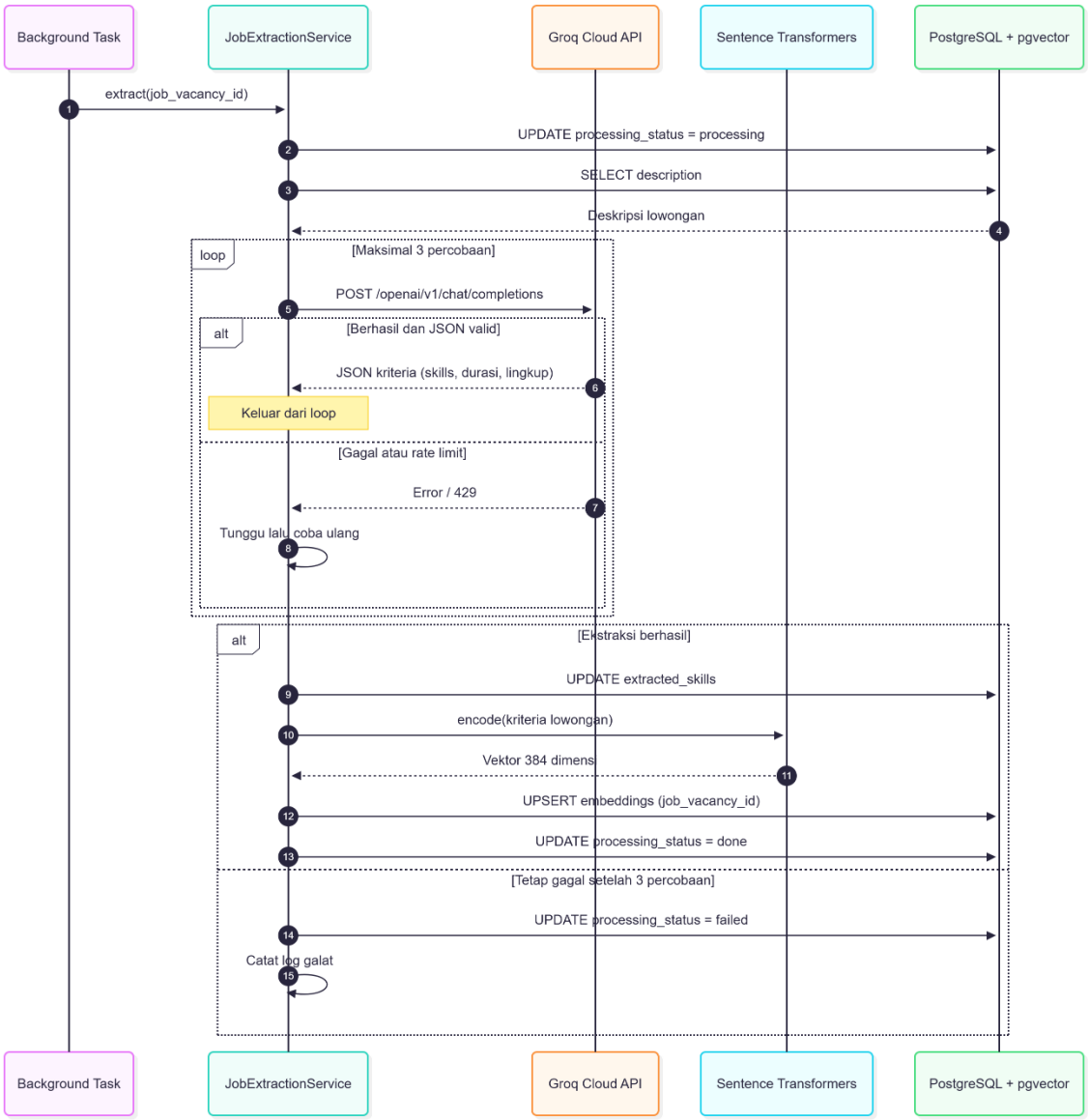
Diagram ini menggambarkan penarikan lowongan oleh JobIngestionService dengan fragmen loop pada tiap platform (LinkedIn, Fiverr, Upwork), alt untuk respons berhasil atau galat dan rate limit pada satu platform, opt untuk entri yang sudah ada (`external_job_id`), serta opt pengantrean ekstraksi LLM. Alt untuk role bukan admin (403) berlaku pada pemicu manual `POST /jobs/sync`.



Gambar 49. Sequence Diagram FR-04: Agregasi Lowongan Pekerjaan Multi-Platform

e. Sequence Diagram FR-05: Ekstraksi Kriteria Lowongan Otomatis (LLM)

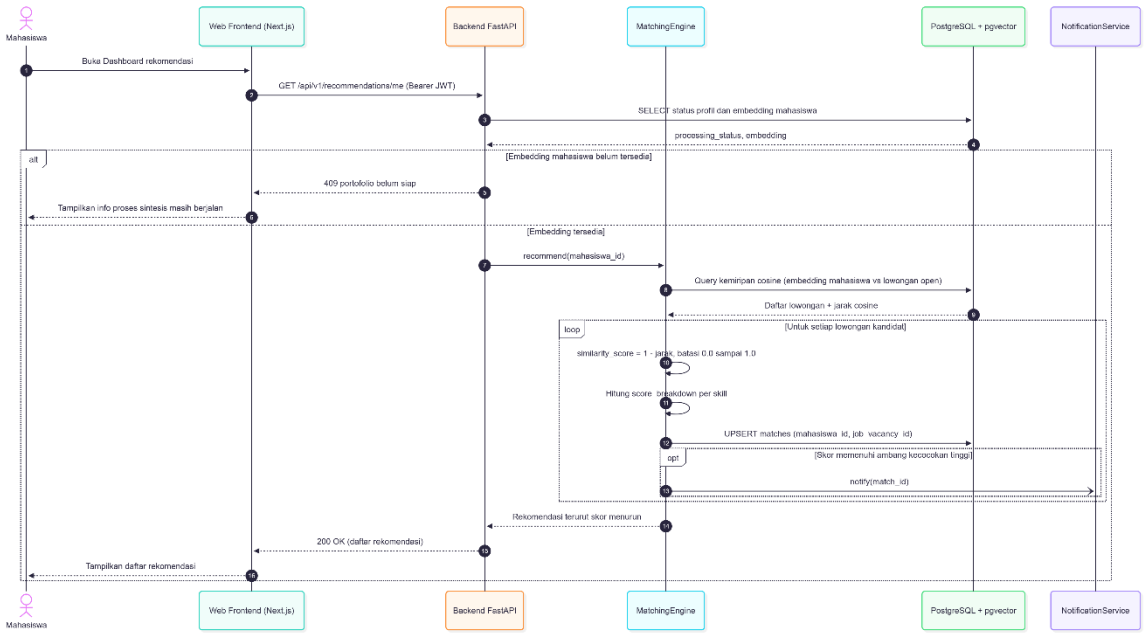
Diagram ini menggambarkan loop percobaan maksimal tiga kali dengan alt untuk respons 200 valid, 429 atau timeout, dan JSON tidak valid. Alt berikutnya memisahkan keberhasilan (simpan hasil, buat embedding, status `done`) dan kegagalan (status `failed`), disusul opt pemrosesan ulang.



Gambar 50. Sequence Diagram FR-05: Ekstraksi Kriteria Lowongan Otomatis (LLM)

f. Sequence Diagram FR-06: AI Semantic Matching

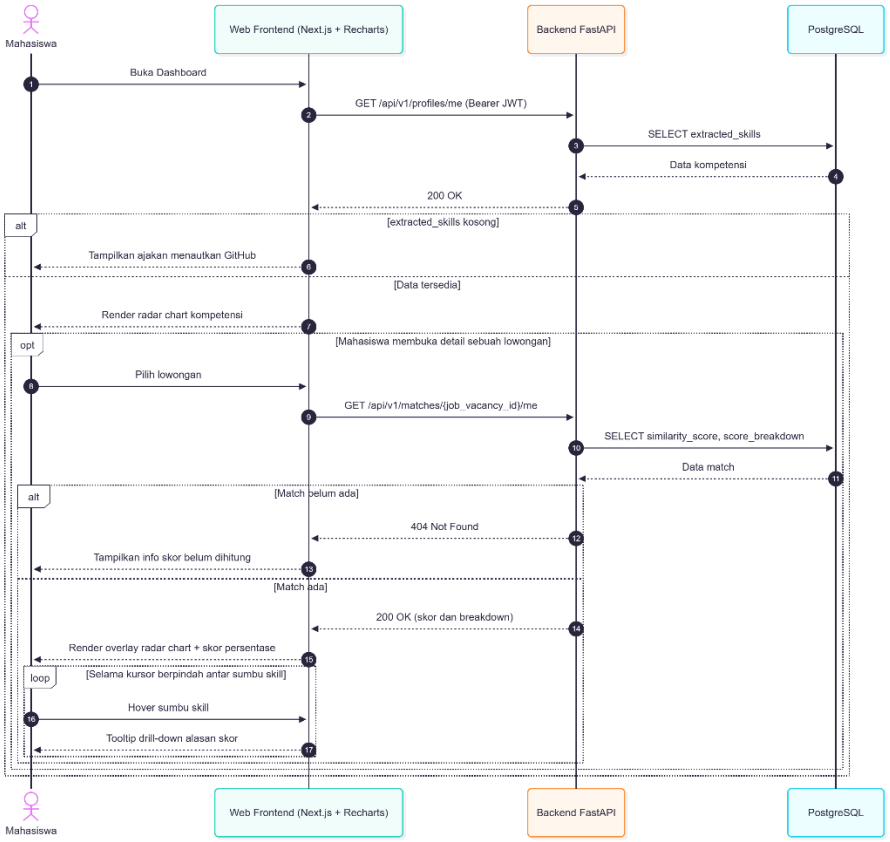
Diagram ini menggambarkan pembukaan rekomendasi dengan alt untuk profil yang belum `done`, loop pada setiap lowongan `open`, opt pemakaian match tersimpan, alt untuk skor valid atau perlu dibatasi (clamp), dan opt notifikasi untuk match baru. Bagian akhir menampilkan detail skor.



Gambar 51. Sequence Diagram FR-06: AI Semantic Matching

g. Sequence Diagram FR-07: Visualisasi Radar Chart Interaktif

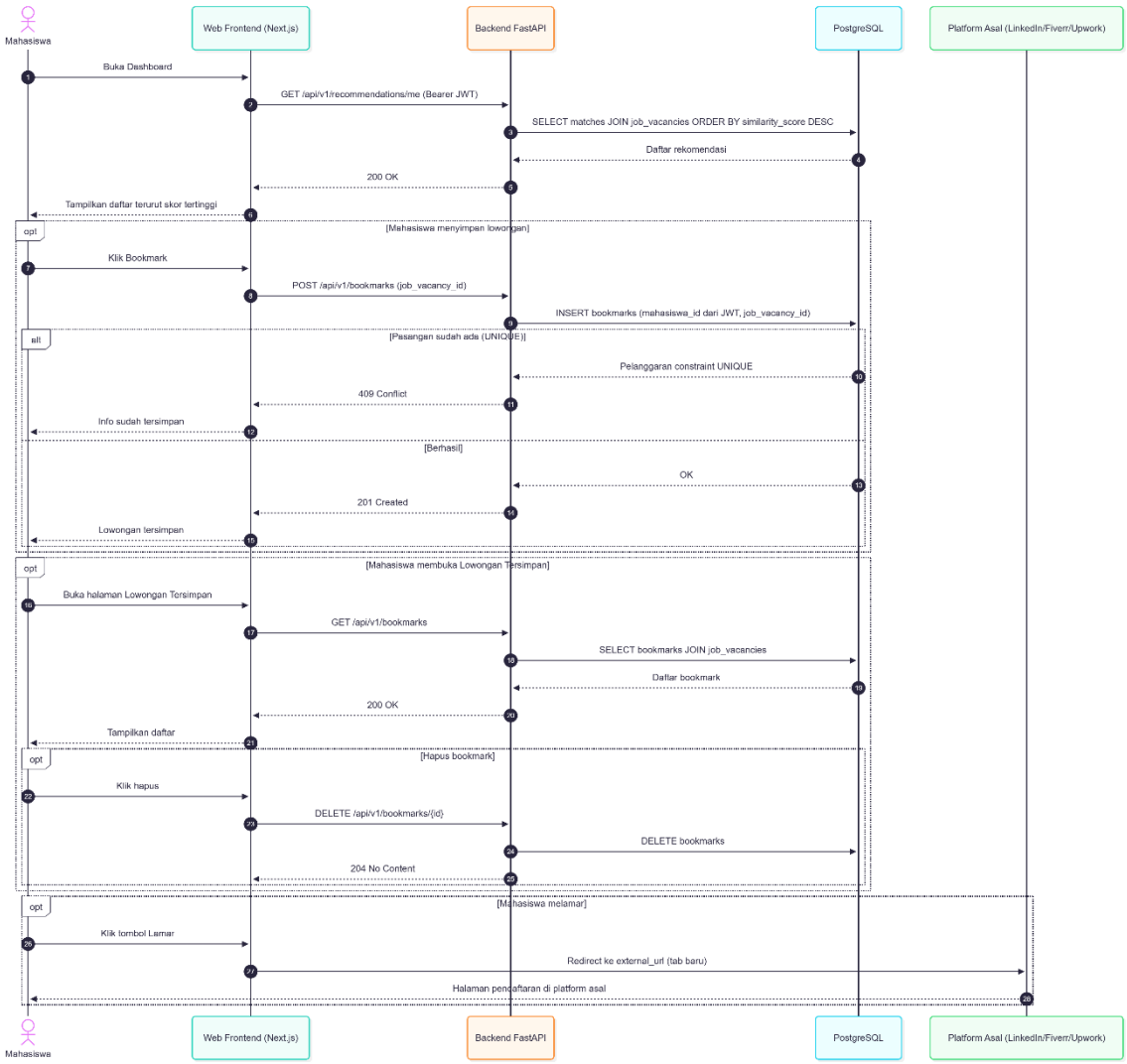
Diagram ini menggambarkan alt untuk token kedaluwarsa dan ketersediaan match (404 bila belum ada), opt pengaktifan overlay kualifikasi lowongan, dan loop drill-down pada tiap sumbu skill di klien.



Gambar 52. Sequence Diagram FR-07: Visualisasi Radar Chart Interaktif

h. Sequence Diagram FR-08: Dasbor Eksplorasi Lowongan & Pengalihan Lamaran

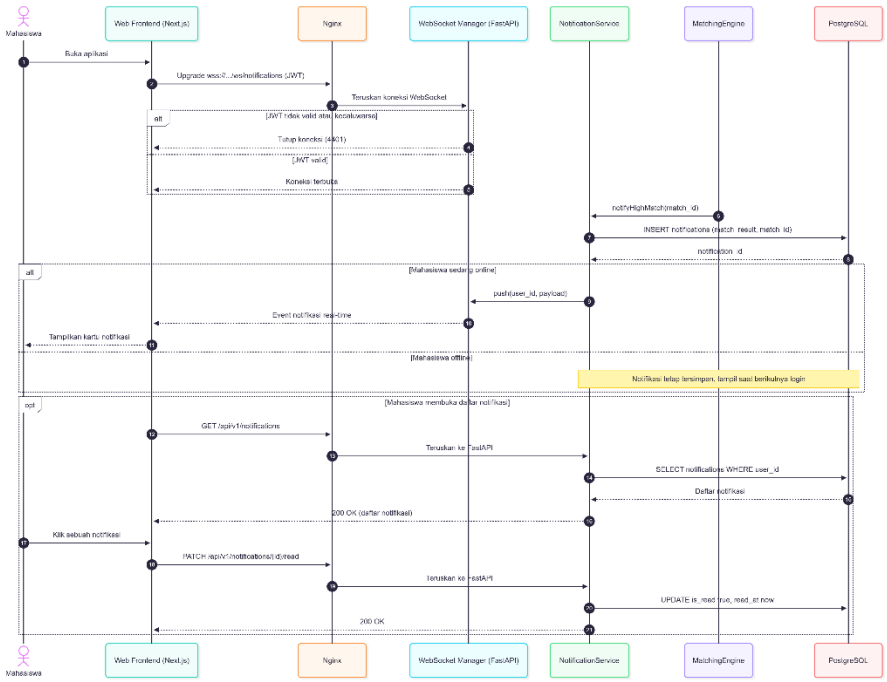
Diagram ini menggambarkan pembukaan daftar lowongan dan rekomendasi (loop pada lowongan), pembukaan detail lowongan, penyimpanan bookmark (alt untuk bookmark ganda 409), dan pengalihan ke `external_url` platform asal.



Gambar 53. Sequence Diagram FR-08: Dasbor Eksplorasi Lowongan & Pengalihan Lamaran

i. Sequence Diagram FR-09: Notifikasi Real-time

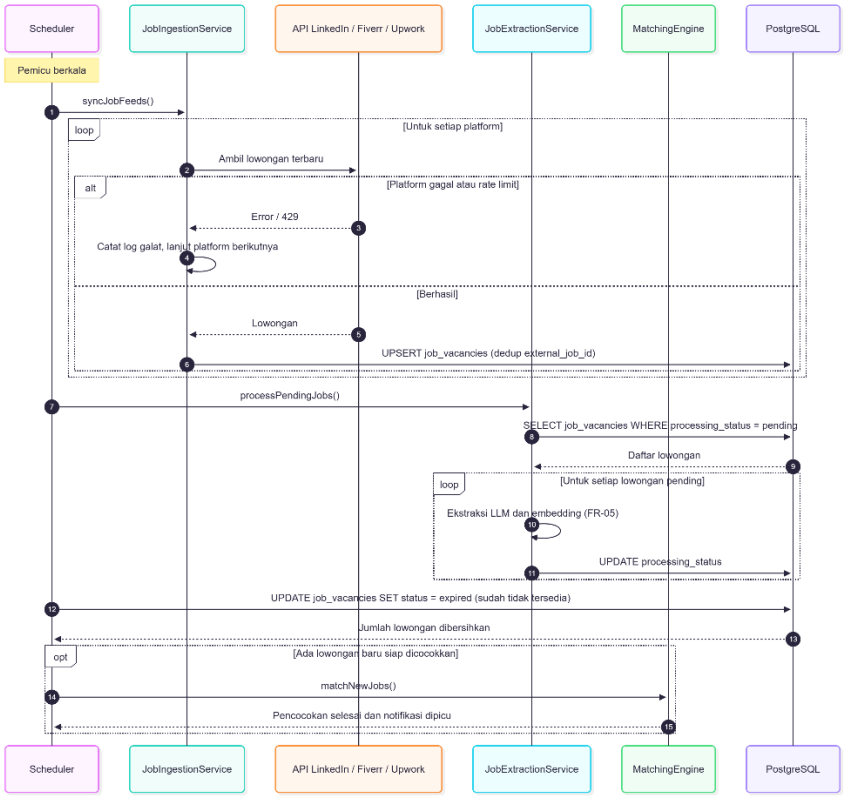
Diagram ini menggambarkan pembukaan koneksi WebSocket melalui Nginx dengan alt untuk JWT valid atau tidak, pengiriman notifikasi dengan alt untuk pengguna online atau offline, loop heartbeat, serta opt pengambilan dan penandaan notifikasi sebagai terbaca.



Gambar 54. Sequence Diagram FR-09: Notifikasi Real-time

j. Sequence Diagram FR-10: Automated Job Feed Scheduler & Data Sync

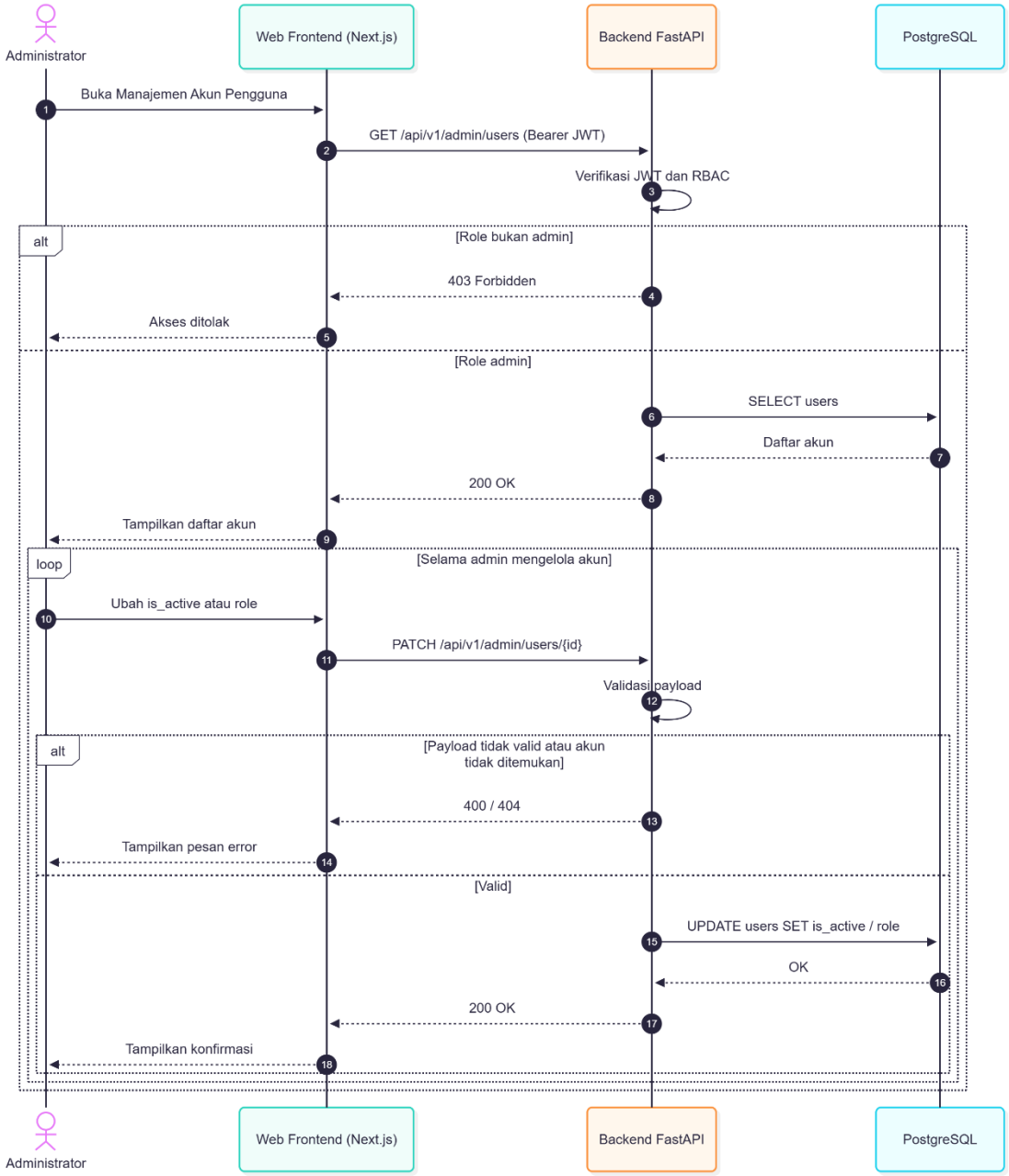
Diagram ini menggambarkan pemicu oleh scheduler atau admin, loop pada tiap platform (LinkedIn, Fiverr, Upwork) dengan alt untuk panggilan berhasil atau galat dan rate limit, opt untuk entri duplikat, dan opt pembersihan lowongan kedaluwarsa menjadi `expired`.



Gambar 55. Sequence Diagram FR-10: Automated Job Feed Scheduler & Data Sync

k. Sequence Diagram FR-11: Manajemen Akun & Hak Akses

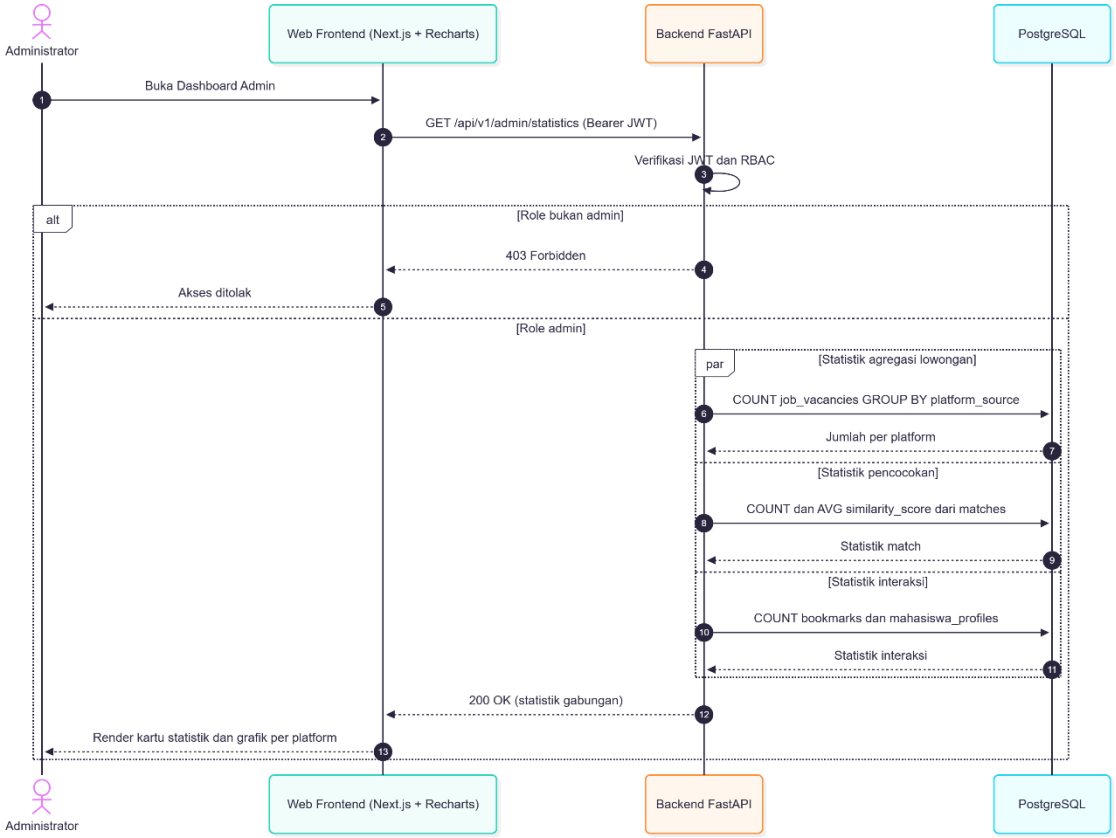
Diagram ini menggambarkan alt untuk role bukan admin (403), loop penampilan daftar akun, alt untuk input tidak valid (422), opt akun tidak ditemukan (404), serta alt untuk perubahan `is_active` atau role.



Gambar 56. Sequence Diagram FR-11: Manajemen Akun & Hak Akses

l. Sequence Diagram FR-12: Monitoring & Statistik Agregasi

Diagram ini menggambarkan alt untuk role bukan admin (403), fragmen par yang menjalankan empat agregasi sekaligus (pengguna per role, lowongan per platform, ringkasan match, interaksi bookmark), opt untuk data kosong, dan loop render kartu atau grafik.



Gambar 57. Sequence Diagram FR-12: Monitoring & Statistik Agregasi

14. Catatan Penyelarasan terhadap SKPL

Laporan ini disusun berdasarkan panduan perubahan SKPL dan arsitektur backend, yaitu transformasi menjadi platform berbasis repositori GitHub dan agregator lowongan freelance dan part-time. Dokumen SKPL akan direvisi menyusul, sehingga hal-hal berikut perlu diselaraskan agar dokumen-dokumen tersebut konsisten satu sama lain.

1. **Algoritma hashing password.** NFR-05, skema tabel `users`, Bab VIII, dan diagram FR-01 menyebut Argon2id. Namun, baris FR-01 pada Matriks Keterkaitan FR dan NFR masih menyebut bcrypt. Laporan ini memakai Argon2id, dan penyebutan bcrypt pada matriks sebaiknya diperbaiki.
2. **Pembatasan role saat registrasi.** Role mitra tidak lagi dipakai, sehingga registrasi publik hanya membuat akun mahasiswa, sedangkan role admin dan perubahan role dikelola lewat FR-11. Asumsi ini sebaiknya ditegaskan di SKPL.
3. **Pemilihan model Groq.** Bagian integrasi menyebut Llama-3.1-8b-instant atau 70b, sedangkan tabel infrastruktur menetapkan Llama-3.1-8b-instant. Laporan ini memakai 8b-instant sebagai model baku.
4. **Mekanisme pemrosesan asinkron.** SKPL menyebut FastAPI BackgroundTasks dengan Celery dan Redis sebagai opsi. Laporan ini memperlakukan BackgroundTasks dan scheduler (cron atau background worker) sebagai baseline, dan Celery dengan Redis sebagai pengembangan bila beban menuntut.
5. **Peran mitra dan lamaran internal.** Panduan perubahan tidak menyebut nasib role mitra, tabel `mitra_profiles`, dan tabel `applications`. Laporan ini menghapusnya karena tidak ada FR yang memakainya: lamaran dialihkan ke platform asal dan digantikan bookmark. Aktor, use case, dan skema pada SKPL perlu disesuaikan.
6. **Tabel bookmark.** Panduan menyebut `POST /jobs/{job_id}/bookmark` tetapi belum mendefinisikan tabel penyimpannya. Laporan ini menambahkan `job_bookmarks` (`mahasiswa_id`, `job_vacancy_id`, `created_at`) dengan pasangan unik. Tabel ini perlu dimasukkan ke skema di SKPL.
7. **Pemicu pencocokan.** FR-06 hanya menyebut perhitungan cosine similarity. Diagram menunjukkan bahwa pencocokan dipicu setelah embedding selesai dan juga dihitung saat rekomendasi dibuka bila match belum tersimpan. Hal ini sebaiknya dicatat di SKPL.
8. **Masa berlaku token.** SKPL mewajibkan JWT dengan masa kedaluwarsa tetapi belum menentukan lamanya, dan tidak membahas refresh token. Laporan ini tidak menetapkan angka.
9. **Test case untuk NFR-06, NFR-07, dan NFR-08.** Rencana pengujian pada SKPL belum memuat skenario khusus untuk ketersediaan, skalabilitas, dan portabilitas. Pada Bab 10, cara verifikasi ketiganya ditandai sebagai usulan.

10. **Atribut skema yang perlu dikonfirmasi.** Panduan tidak mencantumkan `processing_status` pada `job_vacancies`, padahal pipeline asinkron membutuhkannya sehingga laporan ini mempertahankannya. Panduan juga menetapkan `github_username` NOT NULL, sehingga baris `mahasiswa_profiles` dibuat saat username pertama kali diisi, bukan saat registrasi.
11. **Endpoint dan atribut yang dihapus.** `POST /profiles/resume`, `POST /profiles/verify-identity`, `GET /admin/students`, `PATCH /admin/students/{id}/validation`, serta endpoint proyek, kandidat, dan lamaran tidak dipakai lagi. Kolom `validation_status` dan `validated_by` ikut dihapus karena validasi akademik berbasis e-KYC tidak ada lagi.
12. **Ambang batas kecocokan.** Panduan menyebut notifikasi dikirim saat skor melampaui ambang batas tertentu tanpa menetapkan nilainya. Laporan ini tidak menetapkan angka.

15. Keterbatasan dan Mitigasi

Tabel 15. Keterbatasan sistem dan mitigasi pada rancangan

Keterbatasan atau risiko	Dampak	Mitigasi pada rancangan
Ketergantungan pada Groq Cloud API dan kuota Free Tier	Sintesis portofolio dan ekstraksi kriteria lowongan terhenti saat rate limit atau koneksi gagal	Retry backoff maksimal 3 kali, status `failed` yang dapat diproses ulang, `extracted_skills` disimpan permanen agar LLM hanya dipanggil satu kali per lowongan dan per sinkronisasi profil GitHub, fitur lain tetap berjalan, dan log kegagalan dicatat (NFR-09). Peningkatan kapasitas atau LLM lokal menjadi saran pengembangan lanjutan.
Batas kuota (rate limit) API LinkedIn, Upwork, dan Fiverr	Penarikan lowongan gagal saat request melebihi kuota	Polling terjadwal (batch tiap 6 atau 12 jam), lowongan disimpan di `job_vacancies`, deduplikasi lewat `external_job_id`, dan kegagalan satu platform dicatat tanpa menghentikan platform lain (NFR-09).
Ketersediaan dan ketentuan akses API platform pihak ketiga	Akses resmi API LinkedIn, Upwork, atau Fiverr dapat memerlukan persetujuan, kuota berbayar, atau tidak tersedia secara publik	Memastikan akses sebelum implementasi, memakai RapidAPI Job Search sebagai alternatif, dan menyediakan `platform_source` bernilai `manual` untuk lowongan mandiri bila diperlukan.
Batas rate limit GitHub REST API	Data repositori tidak dapat diambil ulang	Hasil disimpan di `github_data` sebagai cache dan dipakai saat GitHub membalas 403 atau 429.
Repositori GitHub mahasiswa sedikit atau kosong	LLM tidak dapat mengekstrak data kompetensi yang memadai	Backend memvalidasi akun; jika total repositori publik adalah 0, sistem memberi respons peringatan agar mahasiswa mengisi atau membuka minimal satu repositori publik sebelum embedding dihitung.
Beban komputasi embedding pada server berspesifikasi ringan	Waktu pemrosesan meningkat saat banyak permintaan	Inferensi LLM sudah dialihkan ke Groq, proses berat berjalan asinkron, dan Celery dengan Redis dapat ditambahkan bila perlu.
Pertumbuhan jumlah mahasiswa, lowongan, dan match	Pencarian dan perhitungan melambat	Indeks HNSW untuk pencarian vektor, penyimpanan dan pemakaian ulang hasil match, pembersihan lowongan `expired`, serta agregasi statistik di basis data (NFR-07).
Ketergantungan pada koneksi internet aktif	Backend tidak dapat mencapai Groq, GitHub, dan API platform lowongan tanpa koneksi	Termasuk batasan sistem pada SKPL. Fitur yang tidak membutuhkan layanan eksternal tetap dapat dipakai.

16. Penutup

16.1 Kesimpulan

Backend ProjectMatch AI dirancang sebagai aplikasi FastAPI tiga lapis yang mendukung dua belas kebutuhan fungsional dan sembilan kebutuhan non-fungsional pada SKPL, dengan portofolio mahasiswa bersumber dari repositori GitHub dan lowongan bersumber dari agregasi LinkedIn, Fiverr, dan Upwork. Keputusan penting dalam rancangan ini adalah memisahkan proses berat dari siklus permintaan pengguna lewat pemrosesan asinkron, memakai pgvector agar data relasional dan pencarian vektor berada dalam satu basis data, dan mengalihkan inferensi LLM ke Groq Cloud API sehingga server dapat berspesifikasi ringan. Ketahanan terhadap kegagalan layanan eksternal, termasuk kegagalan salah satu API platform lowongan, dijaga lewat percobaan ulang terbatas, status `failed` yang dapat diproses ulang, serta pemanfaatan data yang sudah tersimpan.

Rancangan ini baru sebatas sasaran. Akurasi pencocokan, waktu sintesis portofolio GitHub, waktu penguraian lowongan, dan uptime masih harus dibuktikan lewat pengujian sesuai rencana pada SKPL.

16.2 Saran Pengembangan

Sejalan dengan SKPL, pengembangan lanjutan yang dapat dipertimbangkan adalah peningkatan kapasitas API atau pemakaian LLM lokal bila volume permintaan sangat besar, dukungan multi-bahasa pada ekstraksi kriteria, serta penambahan kanal notifikasi di luar aplikasi web seperti WhatsApp Gateway.

DAFTAR PUSTAKA

Brooke, J. (1996). SUS: A 'quick and dirty' usability scale. Usability Evaluation in Industry, 189, 4–7.

IEEE. (2011). ISO/IEC/IEEE 29148:2011 — Systems and Software Engineering: Life Cycle Processes — Requirements Engineering. IEEE.

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. Proceedings of EMNLP-IJCNLP.

Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.